Mathematics

Arthur Li

May 7, 2026

Contents

1 Fundamentals of Higher Mathematics **1**
1.1 Single Variable Calculus . . . 1
1.2 Linear Algebra . . . 4
1.2.1 Vector Spaces . . . 4
1.2.2 Linear Maps . . . 8
1.2.3 Inner-Product Spaces . . . 17
1.2.4 Operators on Inner-Product Spaces . . . 23
1.2.5 Operators on Complex Vector Spaces . . . 32
1.2.6 Trace and Determinant . . . 38

2 Time Series Analysis **39**
2.1 Time Series Analysis . . . 39
2.1.1 Stationary Time Series . . . 39
2.1.2 Univariate Time Series Models . . . 39

3 Mathematical and Statistical Foundation **42**
3.1 Classical Machine Learning . . . 43
3.1.1 Ensemble Methods . . . 43
3.2 Deep Learning . . . 44
3.2.1 Deep Feedforward Networks . . . 44
3.2.2 Regularisation for Deep Learning . . . 51
3.2.3 Optimisation for Deep Learning . . . 56
3.2.4 Convolutional Neural Networks . . . 65

4 Time Series Analysis **67**
4.1 Time Series Analysis . . . 67
4.1.1 Stationary Time Series . . . 67
4.1.2 Univariate Time Series Models . . . 67

1 Fundamentals of Higher Mathematics

Generally covers four areas: single-variable calculus, linear algebra, multivariable calculus, and differential equations. As this is fundamentals, the focus is generally on computation instead of proofs.

1.1 Single Variable Calculus

Calculus is the branch of mathematics that studies change and accumulation. It has two main branches:

i. Differential Calculus: Concerned with the rate at which things change (e.g. velocity).

ii. Integral Calculus: Concerned with accumulation (e.g. area under a curve).

Calculus underpins physics, engineering, economics, biology, and more.

Definition 1.1.1. The *derivative* is defined as:

$$\lim_{\Delta x \to 0} \frac{\Delta f}{\Delta x} = \lim_{\Delta x \to 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x} = f'(x_0)$$

Definition 1.1.2. A function $f(x)$ is continuous at x_0 when

$$\lim_{x \to x_0} f(x) = f(x_0)$$

Definition 1.1.3. The *right-hand* and *left-hand* limits are as follows:

i. *Right-Hand Limit*: $\lim_{x \to x_0^+} f(x)$ means $\lim_{x \to x_0} f(x)$ for $x > x_0$

ii. *Left-Hand Limit*: $\lim_{x \to x_0^-} f(x)$ means $\lim_{x \to x_0} f(x)$ for $x < x_0$

Definition 1.1.4. A *removable discontinuity* is where

$$\lim_{x \to x_0^+} f(x) = \lim_{x \to x_0^-} f(x)$$

but not at $f(x_0)$, or if $f(x_0)$ is undefined.

Definition 1.1.5. A *jump discontinuity* is where $\lim_{x \to x_0^+} f(x)$ and $\lim_{x \to x_0^-} f(x)$ exists, but not equal.

Definition 1.1.6. A *infinite discontinuity* is where $\lim_{x \to x_0^+} f(x) \neq \lim_{x \to x_0^-} f(x) \in \{+\infty, -\infty\}$

Theorem 1.1.7. *Differentiability Implies Continuity*
If f is differentiable at x_0, then f is continuous at x_0.

Proof.

$$\lim_{x \to x_0} (f(x) - f(x_0)) = \lim_{x \to x_0} \left[\frac{f(x) - f(x_0)}{x - x_0} \right] (x - x_0) = f'(x_0) \cdot 0 = 0$$

□

Definition 1.1.8. *Product Formula*
The derivative of product of two functions.

$$(u \cdot v)' = u' \cdot v + u \cdot v'$$

Definition 1.1.9. *Quotient Formula*
The derivate of ratio of two differentiable functions. Let $v(x) \neq 0$, then

$$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$$

Definition 1.1.10. *Chain Rule*
The derivative of composition of two functions.

$$\frac{d}{dt} f(g(t)) = f'(g(t))g'(t)$$

May be used to differentiate implicitly defined functions.

Theorem 1.1.11. *Taylor's Theorem*
Let $k \geq 1$ be an integer and let function $f : \mathbb{R} \to \mathbb{R}$ be k-times differentiable at point $a \in \mathbb{R}$. Then there exists a function $h_k : \mathbb{R} \to \mathbb{R}$ such that

$$f(x) = \sum_{i=0}^{k} \frac{f^{(i)}(a)}{i!}(x-a)^i + h_k(x)(x-a)^k$$

where $\lim_{x \to a} h_k(x) = 0$ is the Peano form of the remainder.

Theorem 1.1.12. *Mean Value Theorem*
Let $f : [a, b] \to \mathbb{R}$ be a continuous function on closed interval $[a, b]$ and differentiable on open interval (a, b) where $a < b$. Then $\exists c \in (a, b)$ such that

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

Theorem 1.1.13. *First Fundamental Theorem of Calculus*
Let f be continuous real-valued function on closed interval $[a, b]$ Let F be function defined for all $x \in [a, b]$, by

$$F(x) = \int_a^x f(t)dt$$

Then F is uniformly continuous on $[a, b]$ and differentiable on open interval (a, b), and

$$F'(x) = f(x)$$

for all x in (a, b) so F is an antiderivative of f.

Remark 1.1.14. *Change of Variable*
If $f(x) = g(u(x))$, then $du = u'(x)dx$, hence

$$\int g(u)du = \int g(u(x))u'(x)dx = \int f(x)u'(x)dx$$

For definite integrals,

$$\int_{x_1}^{x_2} f(x)u'(x)dx = \int_{u_1}^{u_2} g(u)d(u)$$

where $u_1 = u(x_1), u_2 = u(x_2)$.

Theorem 1.1.15. *Second Fundamental Theorem of Calculus*
Let $f : \mathbb{R} \to \mathbb{R}$ on $[a, b]$ and F a continuous function on $[a, b]$ which is an antiderivative of f in (a, b),

$$F'(x) = f(x)$$

If f is Riemann integrable on $[a, b]$, then

$$\int_a^b f(x)dx = F(b) - F(a)$$

Remark 1.1.16. *Integration by Trigonometric Substitution*
Evaluate trigonometric integral, work backwards to original variable by using $\text{trig}(\text{trig}^{-1})$ with right-triangle.

i. $a^2 - x^2$ use $x = a \sin u$ and $dx = a \cos u \; du$
ii. $a^2 + x^2$ use $x = a \tan u$ and $dx = a \sec^2 u \; du$
iii. $x^2 - a^2$ use $x = a \sec u$ and $dx = a \sec u \tan u \; du$

Remark 1.1.17. *Systematic Integration of Rational Functions*
For a rational function P/Q (ratio of polynomials), by method of partial fractions, factor Q completely into linear and quadratic factors, and write P/Q as a sum of simpler terms.

i. If $Q(x) = (x-a)^m(x-b)^n$, then the expression is

$$\frac{A_1}{x-a} + \frac{A_2}{(x-a)^2} + \cdots + \frac{A_m}{(x-a)^m} + \frac{B_1}{x-b} + \frac{B_2}{(x-b)^2} + \cdots + \frac{B_n}{(x-b)^n}$$

ii. If there are quadratic factor such as $(Ax^2+Bx+C)^p$, then there are terms

$$\frac{a_1x+b_1}{Ax^2+Bx+C}+\frac{a_2x+b_2}{(Ax^2+Bx+C)^2}+\cdots+\frac{a_px+b_p}{(Ax^2+Bx+C)^p}$$

for each factor. To integrate, complete the square and make trigonometric substitution.

Remark 1.1.18. *Integration by Parts*

$$\int_b^a uv'\,dx = uv|_a^b - \int_a^b u'v\,dx$$

This is used when $u'v$ is simpler than uv' (often the case when u' is simpler than u).

Remark 1.1.19. *Arc-Length*
Given $ds=\sqrt{dx^2+dy^2}$, depending on whether to integrate with respect to x,t,y, this is

$$ds=\sqrt{1+\left(\frac{dy}{dx}\right)^2}\,dx;\quad ds=\sqrt{\left(\frac{dx}{dt}\right)^2+\left(\frac{dy}{dt}\right)^2}\,dt;\quad ds=\sqrt{\left(\frac{dx}{dy}\right)^2+1}\,dy$$

Remark 1.1.20. *Surface Area for Surface of Revolution*

i. Around x-axis: (requires formula for $y=y(x)$)

$$2\pi y\,ds = 2\pi y\sqrt{1+\left(\frac{dy}{dx}\right)^2}\,dx$$

ii. Around y-axis: (requires formula for $x=x(y)$)

$$2\pi x\,ds = 2\pi x\sqrt{\left(\frac{dx}{dy}\right)^2+1}\,dy$$

Remark 1.1.21. *Polar Coordinates*
Given $x=r\cos\theta$, $y=r\sin\theta$ (or more rarely $r=\sqrt{x^2+y^2}\theta$, $\theta=\tan^{-1}(y/x)$)

i. Find the polar equation for a curve from its equation in (x,y) variables by substitution.
ii. Sketch curves given in polar coordinates and find range of θ for integration.
iii. Integration area formula is then

$$\int_{\theta_1}^{\theta_2}\frac{1}{2}r^2\,d\theta$$

Theorem 1.1.22. *L'Hopital Rule*
For functions f and g defined on open interval I and differentiable on $I\backslash\{c\}$ for a accumulation point c of I, if

$$\lim_{x\to c}f(x)=\lim_{x\to c}g(x)=0\quad or\quad \pm\infty$$

and $g'(x)\neq 0$ for all $x\in I\backslash\{c\}$, and $\lim_{x\to c}\frac{f'(x)}{g'(x)}$ exists, then

$$\lim_{x\to c}\frac{f(x)}{g(x)}=\lim_{x\to c}\frac{f'(x)}{g'(x)}$$

1.2 Linear Algebra

Linear algebra is the branch of mathematics that studies vectors, matrices, and linear transformations. It provides the foundation for solving systems of equations and analysing multidimensional data. Its applications span across machine learning, computer graphics, physics, finance, and engineering.
References from Garcia and Horn (2017) and Axler (2024).

1.2.1 Vector Spaces

Definition 1.2.1. *Vector Space*
Let $\mathbb{F} = \mathbb{R}$ or $\mathbb{C}$. Then $\mathcal{V}$ is a vector space over a field $\mathbb{F}$ if the following are satisfied:

i. Identities: $\exists\ \mathbf{0} \in \mathcal{V}$ such that $\mathbf{0} + \mathbf{u} = \mathbf{u}\ \forall\ \mathbf{u} \in \mathcal{V}$, and $\exists\ \mathbf{1} \in \mathcal{V}$ such that $\mathbf{1u} = \mathbf{u}\ \forall\ \mathbf{u} \in \mathcal{V}$
ii. Commutativity: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}\ \forall\ \mathbf{u}, \mathbf{v} \in \mathcal{V}$
iii. Associativity: $\mathbf{u} + (\mathbf{v} + \mathbf{w}) = (\mathbf{u} + \mathbf{v}) + \mathbf{w}$ and $(ab)\mathbf{u} = a(b\mathbf{u})\ \forall\ a, b \in \mathbb{F}$ and $\forall\ \mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathcal{V}$
iv. Distributive: $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$ and $(a + b)\mathbf{u} = a\mathbf{u} + b\mathbf{u}\ \forall\ a, b \in \mathbb{F}$ and $\forall\ \mathbf{u}, \mathbf{v} \in \mathcal{V}$
v. Additive Inverse: $\forall\ \mathbf{u} \in \mathcal{V}\ \exists$ unique vector $-\mathbf{u} \in \mathcal{V}$ such that $\mathbf{u} + (-\mathbf{u}) = \mathbf{0}$
vi. Multiplicative Inverse: $\forall\ \mathbf{u} \in \mathcal{V}$ with $\mathbf{u} \neq \mathbf{0}\ \exists$ unique $\mathbf{w} \in \mathcal{V}$ such that $\mathbf{uw} = 1$

Remark 1.2.2. *Properties of Vector Spaces*
Let $\mathcal{V}$ be a vector space over a field $\mathbb{F}$.

i. A vector space has a unique additive identity
ii. Every element in a vector space has a unique additive inverse
iii. $\mathbf{0v} = \mathbf{v}$ for every $\mathbf{v} \in \mathcal{V}$
iv. $a\mathbf{0} = \mathbf{0}$ for every $a \in \mathbb{F}$
v. $(-1)\mathbf{v} = -\mathbf{v}$ for every $\mathbf{v} \in \mathcal{V}$

Definition 1.2.3. *Subspace*
A subspace of an $\mathbb{F}$-vector space $\mathcal{V}$ is a subset $\mathcal{U}$ of $\mathcal{V}$ that is an $\mathbb{F}$-vector space with the same vector addition and scalar multiplication operations as in $\mathcal{V}$. To check $\mathcal{U}$ satisfies the following:

i. Additive Identity: $\mathbf{0} \in \mathcal{U}$
ii. Closure Under Addition: $\mathbf{u}, \mathbf{v} \in \mathcal{U}$ implies $\mathbf{u} + \mathbf{v} \in \mathcal{U}$
iii. Closed Under Scalar Multiplication: $a \in \mathbb{F}$ and $\mathbf{u} \in \mathcal{U}$ implies $a\mathbf{u} \in \mathcal{U}$.

Definition 1.2.4. *Sum of Subspaces*
Suppose $\mathcal{U}_1, \ldots, \mathcal{U}_m$ are subspaces of $\mathcal{V}$. The sum of $\mathcal{U}_1, \ldots, \mathcal{U}_m$ as denoted by $\mathcal{U}_1 + \cdots + \mathcal{U}_m$ is defined to be set of all possible sums of elements of $\mathcal{U}_1, \ldots, \mathcal{U}_m$:

$$\mathcal{U}_1 + \cdots + \mathcal{U}_m \coloneqq \{\mathbf{u}_1 + \cdots + \mathbf{u}_m : \mathbf{u}_1 \in \mathcal{U}_1, \ldots, \mathbf{u}_m \in \mathcal{U}_m\}$$

Definition 1.2.5. *Direct Sum of Subspaces*
If $\mathcal{U}_1, \ldots, \mathcal{U}_m$ are subspaces of $\mathcal{V}$ such that $\mathcal{V} = \mathcal{U}_1 + \cdots + \mathcal{U}_m$, then every element of $\mathcal{V}$ may be written as

$$\mathbf{u}_1 + \cdots + \mathbf{u}_m$$

where each $\mathbf{u}_j \in \mathcal{U}_j$. Then $\mathcal{V}$ is the direct sum of subspaces $\mathcal{U}_1, \ldots, \mathcal{U}_m$, written as $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$

Definition 1.2.6. *Linear Combination*
Let $\mathcal{U}$ be non-empty subset of $\mathbb{F}$-vector space $\mathcal{V}$. A linear combination of elements of $\mathcal{U}$ is expressed as

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_r\mathbf{v}_r$$

where r is a positive integer, and $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_r \in \mathcal{U}$, $c_1, c_2, \ldots, c_r \in \mathbb{F}$.
A linear combination is *trivial* if $c_1 = c_2 = \cdots = c_r = 0$.

Definition 1.2.7. *Span*
Let $\mathcal{U}$ be subset of $\mathbb{F}$-vector space $\mathcal{V}$. If $\mathcal{U} \neq \emptyset$, then $\text{span}(\mathcal{U})$ is the set of linear combinations of elements of $\mathcal{U}$. Define $\text{span}(\emptyset) = \{\mathbf{0}\}$. If $\beta = \mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_r$ is a list of $r \geq 1$ vectors in $\mathcal{V}$, define $\text{span}(\beta) = \text{span}\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_r\}$.

Lemma 1.2.8. *Linear Dependence Lemma*
If $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is linearly dependent in $\mathcal{V}$ and $\mathbf{v}_1 \neq 0$, then $\exists\ j \in \{2, \ldots, m\}$ such that the following hold:

i. $\mathbf{v}_j \in span(\mathbf{v}_1, \ldots, \mathbf{v}_{j-1})$

ii. *if* $\mathbf{v}_j$ *is removed, the span of remaining list equals* $span(\mathbf{v}_1, \ldots, \mathbf{v}_m)$.

Proof. i. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ be linearly dependent with $\mathbf{v}_1 \neq \mathbf{0}$. Then $\exists$ scalars $a_1, \ldots, a_m$ not all zero, s.t.

$$a_1\mathbf{v}_1 + \cdots + a_m\mathbf{v}_m = \mathbf{0}$$

Since $\mathbf{v}_1 \neq \mathbf{0}$, not all of $a_2, \ldots, a_m$ can be zero. Otherwise $a_1\mathbf{v_1} = \mathbf{0}$, implying $a_1 = 0$, a contradiction. Hence $\exists$ some index $j \in \{2, \ldots, m\}$ with $a_j \neq 0$.
Choose j be largest index. Then $a_{j+1} = \cdots = a_m = 0$, so the dependence relation reduces to

$$a_1\mathbf{v}_1 + \cdots + a_j\mathbf{v}_j = \mathbf{0}$$

Solving for $\mathbf{v}_j$, we get

$$\mathbf{v}_j = -\frac{a_1}{a_j}\mathbf{v}_1 - \cdots - \frac{a_{j-1}}{a_j}\mathbf{v}_{j-1}$$

So $\mathbf{v}_j \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{j-1})$.

ii. Remove $\mathbf{v}_j$ from the list. Since $\mathbf{v}_j$ is already a linear combination of earlier vectors, any linear combination involving $\mathbf{v}_j$ can be rewritten with only remaining vectors. Hence removing $\mathbf{v}_j$ does not change the span. Hence the span of remaining list equals $\text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_m)$.

□

Theorem 1.2.9. *In a finite-dimensional vector space, length of every linearly independent list of vectors is less than or equal to length of every spanning list of vectors.*

Proof. Let $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ be linearly independent in $\mathcal{V}$, and let $(\mathbf{w}_1, \ldots, \mathbf{w}_n)$ span $\mathcal{V}$. To prove $m \leq n$.
Begin with spanning list $\mathcal{B}_0 = (\mathbf{w}_1, \ldots, \mathbf{w}_n)$. Construct for each $k = 0, 1, \ldots, m$ a spanning list $\mathcal{B}_k$ of length n which contains $\mathbf{u}_1, \ldots, \mathbf{u}_k$.
For $k = 0$, this is true by assumption. For $0 \leq k < m$ and suppose $\mathcal{B}_k$ is a spanning list of length n containing $\mathbf{u}_1, \ldots, \mathbf{u}_k$. Since $\mathcal{B}_k$ spans $\mathcal{V}$, the enlarged list $(\mathbf{u}_{k+1}, \mathcal{B}_k)$ is linearly dependent. Also $\mathbf{u}_{k+1} \neq \mathbf{0}$, because $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ is linearly independent. By Lemma 1.2.8, some vector in $\mathcal{B}_k$ can be removed from $(\mathbf{u}_{k+1}, \mathcal{B}_k)$ without changing the span.
Therefore the removed vector must be one of the original $\mathbf{w}$-vectors still present in $\mathcal{B}_k$. After removing it, obtain a new spanning list $\mathcal{B}_{k+1}$ of length n containing $\mathbf{u}_1, \ldots, \mathbf{u}_k, \mathbf{u}_{k+1}$.
By induction, after m steps there is a spanning list $\mathcal{B}_m$ of length n containing $\mathbf{u}_1, \ldots, \mathbf{u}_m$. Since $\mathcal{B}_m$ has only n vectors but contains m distinct linearly independent vectors, we must have $m \leq n$. Thus the length of every linearly independent list is less than or equal to the length of every spanning list. □

Proposition 1.2.10. *Every subspace of finite-dimensional vector space is finite dimensional.*

Proof. Let $\mathcal{V}$ be finite-dimensional and $\mathcal{U} \subset \mathcal{V}$ a subspace. If $\mathcal{U} = \{\mathbf{0}\}$, the result is trivial.
Assume $\mathcal{U} \neq \{\mathbf{0}\}$. Construct a linearly independent list in $\mathcal{U}$ as follows. Choose $\mathbf{v}_1 \in \mathcal{U}$, $\mathbf{v}_1 \neq \mathbf{0}$. Suppose chosen linearly independent vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k \in \mathcal{U}$.

i. If $\mathcal{U} = \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$, stop

ii. Otherwise, choose $\mathbf{v}_{k+1} \in \mathcal{U}$ such that $\mathbf{v}_{k+1} \notin \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$

Then $(\mathbf{v}_1, \ldots, \mathbf{v}_{k+1})$ remains linearly independent. Continuing, obtain a linearly linearly independent list in $\mathcal{U}$. Now since $\mathcal{V}$ is finite-dimensional, $\exists$ a spanning list of length n. By Theorem 1.2.9, every linearly independent list in $\mathcal{V}$ has length at most n, and construction cannot continue indefinitely. After finitely many steps (i.e., m), process stops. At termination, have $\mathcal{U} = \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_m)$, otherwise the list could be extended further.
Thus $\mathcal{U}$ is spanned by finitely many vectors, hence is finite-dimensional. □

Definition 1.2.11. A *basis* of $\mathcal{V}$ is a list of vectors in $\mathcal{V}$ that is linearly independent and spans $\mathcal{V}$.

Proposition 1.2.12. *A list* $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ *of vectors in* $\mathcal{V}$ *is a basis of* $\mathcal{V}$ *if and only if every* $\mathbf{v} \in \mathcal{V}$ *can be written uniquely in form* $\mathbf{v} = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n$, *where* $a_1, \ldots, a_n \in \mathbb{F}$.

Proof. ($\Rightarrow$) Suppose $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is basis of $\mathcal{V}$. Then it spans $\mathcal{V}$, so for every $\mathbf{v} \in \mathcal{V}$ $\exists$ scalars $a_1, \ldots, a_n$ such that $\mathbf{v} = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n$. To prove uniqueness, suppose also $\mathbf{v} = b_1\mathbf{v}_1 + \cdots + b_n\mathbf{v}_n$. Subtracting gives

$$0 = (a_1 - b_1)\mathbf{v}_1 + \cdots + (a_n - b_n)\mathbf{v}_n$$

Since the list is linearly independent, all coefficients must vanish, $a_j - b_j = 0$ for all j. Hence $a_j = b_j$ for all j, so the representation is unique.
($\Leftarrow$) Suppose every $\mathbf{v} \in \mathcal{V}$ can be written uniquely in the form $\mathbf{v} = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n$. First, the list spans $\mathcal{V}$, since by assumption every $\mathbf{v} \in \mathcal{V}$ admits such a representation..
Next, prove linearly independent. Suppose $0 = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n$. But also $0 = 0 \cdot \mathbf{v}_1 + \cdots + 0 \cdot \mathbf{v}_n$. By uniqueness of representation of zero vector, we must have $a_1 = \cdots = a_n = 0$.
Thus the list is linearly independent and therefore a basis of $\mathcal{V}$. □

Theorem 1.2.13. *Every spanning list in a vector space can be reduced to a basis of vector space.*

Proof. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be spanning list for $\mathcal{V}$. Construct a basis by removing redundant vectors.
Initialise $\mathcal{B} = (\mathbf{v}_1, \ldots, \mathbf{v}_n)$. Process the vectors in order. For each index $j = 1, \ldots, n$, do the following:

i. If current vector $\mathbf{v}_j$ lies in span of preceding vectors already retained in $\mathcal{B}$, remove $\mathbf{v}_j$ from $\mathcal{B}$.

ii. Otherwise, keep $\mathbf{v}_j$ in $\mathcal{B}$.

At each step, a vector is removed only if it lies in span of previously retained vectors. By Lemma 1.2.8, removing such a vector does not change the span. Since original list spans $\mathcal{V}$, final list $\mathcal{B}$ also spans $\mathcal{V}$. By construction, no vector in $\mathcal{B}$ lies in span of those preceding it, equivalent to linear independence (by Lemma 1.2.8).
Hence $\mathcal{B}$ is a basis of $\mathcal{V}$. □

Corollary 1.2.14. *Every finite-dimensional vector space has a basis.*

Proof. Let $\mathcal{V}$ be a finite-dimensional vector space. By definition, $\exists$ a finite list of vectors $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ that spans $\mathcal{V}$. By Theorem 1.2.13, any spanning list can be reduced to a basis by removing redundant vectors. Applying this result to $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ obtain a sublist that is both linearly independent and still spans $\mathcal{V}$. Hence this sublist is a basis of $\mathcal{V}$, therefore $\mathcal{V}$ has a basis. □

Theorem 1.2.15. *Every linearly independent list of vectors in a finite-dimensional vector space can be extended to a basis of the vector space.*

Proof. Let $\mathcal{V}$ be finite dimensional and $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ be linearly independent list in $\mathcal{V}$. Since $\mathcal{V}$ is finite dimensional, choose a finite spanning list $(\mathbf{w}_1, \ldots, \mathbf{w}_n)$ for $\mathcal{V}$.
Start with $\mathcal{B} = (\mathbf{v}_1, \ldots, \mathbf{v}_m)$. Now inspect the vectors $\mathbf{w}_1, \ldots, \mathbf{w}_n$ one at a time. For each $j = 1, \ldots, n$,

i. If $\mathbf{w}_j \in \text{span}(\mathcal{B})$, leave $\mathcal{B}$ unchanged.

ii. If $\mathbf{w}_j \notin \text{span}(\mathcal{B})$, append $\mathbf{w}_j$ to $\mathcal{B}$.

At every step, $\mathcal{B}$ remains linearly independent. Indeed, appending $\mathbf{w}_j$ only occurs when $\mathbf{w}_j \notin \text{span}(\mathcal{B})$, and adding a vector outside the span of a linearly independent list preserves linear independence.
After all $\mathbf{w}_j$'s have been inspected, the final list $\mathcal{B}$ contains $\mathbf{v}_1, \ldots, \mathbf{v}_m$. Moreover, every $\mathbf{w}_j$ lies in $\text{span}(\mathcal{B})$; either $\mathbf{w}_j$ was added to $\mathcal{B}$, or it was already in span of $\mathcal{B}$ when considered. Therefore,

$$\text{span}(\mathbf{w}_1, \ldots, \mathbf{w}_n) \subseteq \text{span}(\mathcal{B})$$

Since $(\mathbf{w}_1, \ldots, \mathbf{w}_n)$ spans $\mathcal{V}$, thus $\mathcal{B}$ spans $\mathcal{V}$. Thus $\mathcal{B}$ is both linearly independent and spanning, so $\mathcal{B}$ is a basis of $\mathcal{V}$. Hence every linearly independent list in a finite-dimensional vector space can be extended to a basis. □

This theorem can be applied to show that every subspace of a finite-dimensional vector space can be paired with another subspace to form a direct sum of the whole space.

Proposition 1.2.16. *Suppose $\mathcal{V}$ is finite dimensional and $\mathcal{U}$ is a subspace of $\mathcal{V}$. Then there is a subspace $\mathcal{W}$ of $\mathcal{V}$ such that $\mathcal{V} = \mathcal{U} \oplus \mathcal{W}$.*

Proof. Let $\mathcal{V}$ be finite dimensional, and let $\mathcal{U}$ be subspace of $\mathcal{V}$, By Proposition 1.2.10, $\mathcal{U}$ is finite dimensional. Choose a basis $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ of $\mathcal{U}$. Since this list is linearly independent in $\mathcal{V}$, Theorem 1.2.15 allows us to extend it to a basis of $\mathcal{V}$, $(\mathbf{u}_1, \ldots \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_m)$.
Define $\mathcal{W} = \text{span}(\mathbf{w}_1, \ldots, \mathbf{w}_n)$. Claim that $\mathcal{V} = \mathcal{U} \oplus \mathcal{W}$.
First, since $(\mathbf{u}_1, \ldots \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_m)$ spans $\mathcal{V}$, every $\mathbf{v} \in \mathcal{V}$ can be written as

$$\mathbf{v} = a_1\mathbf{u}_1 + \cdots + a_m\mathbf{u}_m + b_1\mathbf{w}_1 + \cdots + b_n\mathbf{w}_n = \mathbf{u} + \mathbf{w}$$

Let $\mathbf{u} = a_1\mathbf{u}_1 + \cdots + a_m\mathbf{u}_m \in \mathcal{U}$ and $\mathbf{w} = b_1\mathbf{w}_1 + \cdots + b_n\mathbf{w}_n \in \mathcal{W}$. Since $\mathbf{v} \in \mathcal{U}$, $\exists$ scalars $a_1, \ldots, a_m$ such that

$$\mathbf{v} = b_1\mathbf{w} + \cdots + b_n\mathbf{w}_n$$

Subtracting,

$$a_1\mathbf{u}_1 + \cdots + a_m\mathbf{u}_m - b_1\mathbf{w}_1 - \cdots - b_n\mathbf{w}_n = \mathbf{0}$$

Because $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{w}_1, \ldots, \mathbf{w}_n)$ is linearly independent, all coefficients are zero.
In particular, $a_1 = \cdots = a_m = 0$, so $\mathbf{v} = \mathbf{0}$. Therefore $\mathcal{U} \cap \mathcal{W} = \{\mathbf{0}\}$.
Thus $\mathcal{V} = \mathcal{U} + \mathcal{W}$ and $\mathcal{U} \cap \mathcal{W} = \{\mathbf{0}\}$, so $\mathcal{V} = \mathcal{U} \oplus \mathcal{W}$. □

Theorem 1.2.17. *Any two bases of a finite-dimensional vector space have the same length.*

Proof. Let $\mathcal{V}$ be finite dimensional, and $\mathcal{B}_1, \mathcal{B}_2$ be two bases of $\mathcal{V}$. Since $\mathcal{B}_1$ is a basis, it is linearly independent. Since $\mathcal{B}_2$ is a basis, it spans $\mathcal{V}$. Therefore, $|\mathcal{B}_1| \leq |\mathcal{B}_2|$.
By same reasoning, now using that $\mathcal{B}_2$ is linearly independent and $\mathcal{B}_1$ spans $\mathcal{V}$, $|\mathcal{B}_2| \leq |\mathcal{B}_1|$. Hence $|\mathcal{B}_1| = |\mathcal{B}_2|$. □

Definition 1.2.18. The *dimension* of a finite-dimensional vector space is defined to be the length of any basis of the vector space, denoted as $\dim(\mathcal{V})$.

Proposition 1.2.19. *If $\mathcal{V}$ is finite dimensional and $\mathcal{U}$ is a subspace of $\mathcal{V}$, then $\dim(\mathcal{U}) \leq \dim(\mathcal{V})$.*

Proof. Let $\mathcal{V}$ be finite-dimensional and $\mathcal{U} \subseteq \mathcal{V}$ a subspace. Let $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ be a basis of $\mathcal{U}$. Then this list is linearly independent in $\mathcal{V}$.
By Theorem 1.2.15, it can be extended to a basis of $\mathcal{V}$, say $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_k)$. Thus the dimension of $\mathcal{U}$ is m, and the dimension of $\mathcal{V}$ is $m + k$. Hence $\dim(\mathcal{U}) = m \leq m + k = \dim(\mathcal{V})$. □

Proposition 1.2.20. *If $\mathcal{V}$ is finite dimensional, then every spanning list of vectors in $\mathcal{V}$ with length $\dim(\mathcal{V})$ is a basis of $\mathcal{V}$.*

Proof. Let $\dim(\mathcal{V}) = n$ and let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be a spanning list of $\mathcal{V}$.
By Theorem 1.2.13, this list can be reduced to a basis by removing vectors that are linear combinations of preceding ones. Thus there exists a sublist of $(\mathbf{v}_1, \ldots \mathbf{v}_n)$ that forms a basis of $\mathcal{V}$. Every basis of $\mathcal{V}$ has exactly n vectors. Since the original list already has length n, no vector can be removed in this reduction process; otherwise would obtain a basis with fewer than n vectors, a contradiction.
Therefore original list $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is already linearly independent. Since it also spans $\mathcal{V}$, it is a basis of $\mathcal{V}$. □

Proposition 1.2.21. *If $\mathcal{V}$ is finite dimensional, then every linearly independent list of vectors in $\mathcal{V}$ with length $\dim(\mathcal{V})$ is a basis of $\mathcal{V}$.*

Proof. Let $\dim(\mathcal{V}) = n$ and $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be linearly independent in $\mathcal{V}$. By Theorem 1.2.15, this list can extended to a basis of $\mathcal{V}$. Thus $\exists$ vectors $\mathbf{w}_1, \ldots, \mathbf{w}_k$ such that $(\mathbf{v}_1, \ldots, \mathbf{v}_n, \mathbf{w}_1, \ldots, \mathbf{w}_k)$ is a basis of $\mathcal{V}$.
But every basis of $\mathcal{V}$ has exactly n vectors. Therefore $n + k = n$, which implies $k = 0$. Hence no additional vectors are needed, and the original list $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is already a basis of $\mathcal{V}$. □

The next theorem gives formula for dimension of sum of two subspaces of a finite-dimensional vector space.

Theorem 1.2.22. *If $\mathcal{U}_1$ and $\mathcal{U}_2$ are subspaces of a finite-dimensional vector space, then*

$$\dim(\mathcal{U}_1 + \mathcal{U}_2) = \dim(\mathcal{U}_1) + \dim(\mathcal{U}_2) - \dim(\mathcal{U}_1 \cap \mathcal{U}_2)$$

Proof. Let $\mathcal{U}_1, \mathcal{U}_2 \subset \mathcal{V}$, suppose $\mathcal{V}$ is finite dim. Let $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ be basis of $\mathcal{U}_1 \cap \mathcal{U}_2$, then $\dim(\mathcal{U}_1 \cap \mathcal{U}_2) = m$.
Extend this to a basis of $\mathcal{U}_1$, $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_j)$, $\dim(\mathcal{U}_1) = m + j$.
Similarly, extend it to a basis of $\mathcal{U}_2$, $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{w}_1, \ldots, \mathbf{w}_k)$, $\dim(\mathcal{U}_2) = m + k$.
We claim that $\mathcal{B} = (\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_j, \mathbf{w}_1, \ldots, \mathbf{w}_k)$ is a basis of $\mathcal{U}_1 + \mathcal{U}_2$.
Spanning: Every vector in $\mathcal{U}_1$ is a linear combination of $\mathbf{u}_i$'s and $\mathbf{w}_i$'s. Hence any vector in $\mathcal{U}_1 + \mathcal{U}_2$ is a linear combination of vectors in $\mathcal{B}$. There $\mathcal{B}$ spans $\mathcal{U}_1 + \mathcal{U}_2$.
Linear Independence. Suppose

$$a_1\mathbf{u}_1 + \cdots + a_m\mathbf{u}_m + b_1\mathbf{v}_1 + \cdots + b_j\mathbf{v}_j + c_1\mathbf{w}_1 + \cdots + c_k\mathbf{w}_k = \mathbf{0}$$

where $a_i, b_i, c_i \in \mathbb{F}$. Rearrange,

$$c_1\mathbf{w}_1 + \cdots + c_k\mathbf{w}_k = -a_1\mathbf{u}_1 - \cdots - a_m\mathbf{u}_m - b_1\mathbf{v}_1 - \cdots - b_j\mathbf{v}_j$$

The RHS is in $\mathcal{U}_1$, and LHS is in $\mathcal{U}_2$, hence this vector lies in $\mathcal{U}_1 \cap \mathcal{U}_2$. Therefore $\exists$ scalars $d_1, \ldots, d_m$ such that

$$c_1\mathbf{w}_1 + \cdots + c_k\mathbf{w}_k = d_1\mathbf{u}_1 + \cdots + d_m\mathbf{u}_m$$

Substituting back gives

$$\mathbf{0} = (a_1 + d_1)\mathbf{u}_1 + \cdots + (a_m + d_m)\mathbf{u}_m + b_1\mathbf{v}_1 + \cdots + b_j\mathbf{v}_j$$

Note that $c_1\mathbf{w}_1 + \cdots + c_k\mathbf{w}_k = d_1\mathbf{u}_1 + \cdots + d_m\mathbf{u}_m$, the LHS is in $\text{span}(\mathbf{u}_1, \ldots, \mathbf{u}_m)$. Since $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{w}_1, \ldots, \mathbf{w}_m))$ is a basis of $\mathcal{U}_2$, these spans intersect trivially, so both sides must be zero. Hence

$$c_1 = \cdots = c_k = 0, \quad d_1 = \cdots = d_m = 0$$

and therefore $a_1 = \cdots = a_m = 0$. Thus all coefficients vanish, proving learning independence.
Since $\mathcal{B}$ is a basis of $\mathcal{U}_1 + \mathcal{U}_2$, we obtain

$$\dim(\mathcal{U}_1 + \mathcal{U}_2) = m + j + k = (m + j) + (m + k) - m = \dim(\mathcal{U}_1) + \dim(\mathcal{U}_2) - \dim(\mathcal{U}_1 \cap \mathcal{U}_2)$$

□

Note that dimension meshes well with direct sums, which is analogous to disjoint union.

Proposition 1.2.23. *Suppose $\mathcal{V}$ is finite dimensional and $\mathcal{U}_1, \ldots, \mathcal{U}_m$ are subspaces of $\mathcal{V}$ such that*

$$\mathcal{V} = \mathcal{U}_1 + \cdots + \mathcal{U}_m, \qquad \dim(\mathcal{V}) = \dim(\mathcal{U}_1) + \cdots + \dim(\mathcal{U}_m)$$

Then $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$.

Proof. Let $\mathcal{V}$ be finite dimensional, and suppose $\mathcal{V} = \mathcal{U}_1 + \cdots + \mathcal{U}_m$, $\dim(\mathcal{V}) = \dim(\mathcal{U}_1) + \cdots + \dim(\mathcal{U}_m)$.
For each j, let $\mathcal{B}_j = (\mathbf{u}_{j1}, \ldots, \mathbf{u}_{d_j})$ be a basis of $\mathcal{U}_j$, where $d_j = \dim(\mathcal{U}_j)$.
Form the concentrated list $\mathcal{B} = \mathcal{B}_1 \cup \cdots \cup \mathcal{B}_m$.

i. $\mathcal{B}$ spans $\mathcal{V}$. Since $\mathcal{V} = \mathcal{U}_1 + \cdots + \mathcal{U}_m$, and $\mathbf{v} \in \mathcal{V}$ can be written as

$$\mathbf{v} = \mathbf{u}_1 + \cdots + \mathbf{u}_m, \quad \mathbf{u}_j \in \mathcal{U}_j$$

Each $\mathbf{u}_j$ is a linear combination of vectors in $\mathcal{B}_j$, hence $\mathbf{v}$ is a linear combination of vectors in $\mathcal{B}$.

ii. $\mathcal{B}$ is a basis. The length of $\mathcal{B}$ is $|\mathcal{B}| = d_1 + \cdots + d_m = \dim(\mathcal{V})$. Since $\mathcal{B}$ is a spanning list of length $\dim(\mathcal{V})$, Proposition 1.2.20 implies $\mathcal{B}$ is a basis of $\mathcal{V}$. In particular, it is linearly independent.

iii. Directness of the Sum. Suppose $\mathbf{u}_1 + \cdots + \mathbf{u}_m = \mathbf{0}$, $\mathbf{u}_j \in \mathcal{U}_j$, write each $\mathbf{u}_j$ as a linear combination of basis vectors in $\mathcal{B}_j$. Then we obtain a linear combination of vectors in $\mathcal{B}$ equal to $\mathbf{0}$.
Since $\mathcal{B}$ is linearly independent, all coefficients must equal zero. Hence each $\mathbf{u}_j = \mathbf{0}$. Thus

$$\mathcal{U}_1 \cap (\mathcal{U}_2 + \cdots + \mathcal{U}_m) = \{\mathbf{0}\}$$

and similarly for all components, so the sum is direct. Therefore $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$.

□

1.2.2 Linear Maps

Definition 1.2.24. A *Linear Map* from $\mathcal{V}$ to $\mathcal{W}$ is a function $T : \mathcal{V} \to \mathcal{W}$ with following properties:

i. *Additivity*: $T(\mathbf{u} + \mathbf{v}) = T\mathbf{u} + T\mathbf{v}$ for all $\mathbf{u}, \mathbf{v} \in \mathcal{V}$

ii. *Homogeneity*: $T(a\mathbf{v}) = a(T\mathbf{v})$ for all $a \in \mathbb{F}$ and all $\mathbf{v} \in \mathcal{V}$.

The set of all linear maps from $\mathcal{V}$ to $\mathcal{W}$ is denoted $\mathcal{L}(\mathcal{V}, \mathcal{W})$.
The linear map is *injective* if whenever $\mathbf{u}, \mathbf{v} \in \mathcal{V}$ and $T\mathbf{u} = T\mathbf{v}$, that $\mathbf{u} = \mathbf{v}$; and *surjective* if its range equals $\mathcal{W}$.

Example 1.2.25. Examples of linear maps include

i. Zero and Identitiy Map: $0 \in \mathcal{L}(\mathcal{V}, \mathcal{W})$ is defined by $0\mathbf{v} = 0$. $I \in \mathcal{L}(\mathcal{V}, \mathcal{V})$ is defined by $I\mathbf{v} = \mathbf{v}$

ii. Differentiation: $T \in \mathcal{L}(\mathcal{P}(\mathbf{R}), \mathcal{P}(\mathbf{R}))$ is defined by $T\mathbf{p} = \mathbf{p}'$

iii. Integration: $T \in \mathcal{L}(\mathcal{P}(\mathbf{R}), \mathbf{R})$ is defined by $T\mathbf{p} = \int_0^1 \mathbf{p}(\mathbf{x})\, d\mathbf{x}$

iv. Backward Shift: Let $\mathbb{F}^\infty$ be vector space of all sequences of elements of $\mathbb{F}$. Let $T \in \mathcal{L}(\mathbb{F}^\infty, \mathbb{F}^\infty)$. The backward shift map is then $T(x_1, x_2, x_3, \ldots) = (x_2, x_3, \ldots)$

v. From $\mathbb{F}^n$ to $\mathbb{F}^m$: $T \in \mathcal{L}(\mathbb{F}^n, \mathbb{F}^m)$ defined by $T(x_1, \ldots, x_n) = (a_{1,1}x_1 + \cdots + a_{1,n}x_n, \ldots, a_{m,1}x_1 + \cdots + a_{m,n}x_n)$, where $a_{j,k} \in \mathbb{F}$, $j = 1, \ldots, m$ and $k = 1, \ldots, n$.

Remark 1.2.26. *Properties of Product of Linear Maps*
Let $T \in \mathcal{L}(\mathcal{U}, \mathcal{V})$, and $S \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. The *product* of S and T is ST.

i. Associativity: $(T_1T_2)T_3 = T_1(T_2T_3)$
ii. Identity: $TI = T$ (identity map on $\mathcal{V}$) and $IT = T$ (identity map on $\mathcal{W}$)
iii. Distributive: $(S_1 + S_2)T = S_1T + S_2T$ and $S(T_1 + T_2) = ST_1 + ST_2$ whenever $T, T_1, T_2 \in \mathcal{L}(\mathcal{U}, \mathcal{V})$ and $S, S_1, S_2 \in \mathcal{L}(\mathcal{V}, \mathcal{W})$

Multiplication of linear maps is not commutative.

Definition 1.2.27. For $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, the *null space* of T, denoted $\text{null}(T)$, is the subset of $\mathcal{V}$ consisting of those vectors that maps T to 0, $\text{null}(T) = \{\mathbf{v} \in \mathcal{V} : T\mathbf{v} = \mathbf{0}\}$.

Proposition 1.2.28. *If $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, then null$(T)$ is a subspace of $\mathcal{V}$.*

Proof. Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. Because T is linear, $T(\mathbf{0}) = \mathbf{0}$, hence $\mathbf{0} \in \text{null}(T)$.
Now let $\mathbf{u}, \mathbf{v} \in \text{null}(T)$, and let $\alpha, \beta \in \mathbb{F}$. Then $T(\alpha\mathbf{u} + \beta\mathbf{v}) = \alpha T\mathbf{u} + \beta T\mathbf{v} = \mathbf{0} + \mathbf{0} = \mathbf{0}$, so $\alpha\mathbf{u} + \beta\mathbf{v} \in \text{null}(T)$.
Therefore $\text{null}(T)$ is closed under linear combinations and contains $\mathbf{0}$, so it is a subspace of $\mathcal{V}$. □

Proposition 1.2.29. *Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. Then T is injective if and only if null$(T) = \{\mathbf{0}\}$.*

Proof. Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$.
($\Rightarrow$) Suppose T is injective. If $\mathbf{v} \in \text{null}(T)$, then $T(\mathbf{v}) = \mathbf{0} = T(\mathbf{0})$. Since T is injective, $\mathbf{v} = \mathbf{0}$, so $\text{null}(T) = \{\mathbf{0}\}$.
($\Leftarrow$) Suppose $\text{null}(T) = \{\mathbf{0}\}$. If $T(\mathbf{u}) = T(\mathbf{v})$, then $T(\mathbf{u} - \mathbf{v}) = T(\mathbf{u}) - T(\mathbf{v}) = \mathbf{0}$, so $\mathbf{u} - \mathbf{v} \in \text{null}(T)$. Thus $\mathbf{u} - \mathbf{v} = 0$, hence $\mathbf{u} = \mathbf{v}$. Therefore T is injective. □

Infectivity is equivalent to saying the only vector mapped to 0 is the zero vector.

Definition 1.2.30. For $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, the *range* of T, denoted $\text{range}(T)$, is subset of $\mathcal{W}$ consisting of vectors that are of from $T\mathbf{v}$ for some $\mathbf{v} \in \mathcal{V}$, where $\text{range}(T) = \{T\mathbf{v} : \mathbf{v} \in \mathcal{V}\}$.

Proposition 1.2.31. *If $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, then range$(T)$ is a subspace of $\mathcal{W}$.*

Proof. Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. Because $T(\mathbf{0}) = \mathbf{0}$, we have $\mathbf{0} \in \text{range}(T)$. Now let $\mathbf{w}_1, \mathbf{w}_2 \in \text{range}(T)$ and $\alpha, \beta \in \mathbb{F}$.
Then $\exists\ \mathbf{v}_1, \mathbf{v}_2 \in \mathcal{V}$ s.t. $T(\mathbf{v}_1) = \mathbf{w}_1$ and $T(\mathbf{v}_2) = \mathbf{w}_2$. Hence $\alpha\mathbf{w}_1 + \beta\mathbf{w}_2 = \alpha T(\mathbf{v}_1) + \beta T(\mathbf{v}_2) = T(\alpha\mathbf{v}_1 + \beta\mathbf{v}_2)$, so $\alpha\mathbf{w}_1 + \beta\mathbf{w}_2 \in \text{range}(T)$. Therefore $\text{range}(T)$ is closed under addition and scalar multiplication and contains $\mathbf{0}$, hence is a subspace of $\mathcal{W}$. □

The range is the image of a linear map, and linearity ensures that images of linear combinations are linear combinations of images. This directly induces subspace structure, making separate closure checks unnecessary.

Theorem 1.2.32. *Rank Nullity Theorem If $\mathcal{V}$ is finite-dimensional and $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, then range$(T)$ is a finite dimensional subspace of $\mathcal{W}$, and dim$(\mathcal{V})$ = dim(null(T)) + dim(range(T)).*

Proof. Let $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ be a basis of $\text{null}(T)$. Extend this list to a basis $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{w}_1, \ldots, \mathbf{w}_n)$ of $\mathcal{V}$. Then $\dim(\mathcal{V}) = m + n$. Claim that $(T\mathbf{w}_1, \ldots, T\mathbf{w}_n)$ is a basis of $\text{range}(T)$.
Spanning: Let $\mathbf{v} \in \mathcal{V}$. Write $\mathbf{v} = a_1\mathbf{u}_1 + \cdots + a_m\mathbf{u}_m + b_1\mathbf{w}_1 + \cdots + b_n\mathbf{w}_n$. Applying T and using $T(\mathbf{u}_j) = \mathbf{0}$, obtain $T(\mathbf{v}) = b_1T\mathbf{w}_1 + \cdots + b_nT\mathbf{w}_n$. Thus every vector in $\text{range}(T)$ is a linear combination of $T\mathbf{w}_1, \ldots, T\mathbf{w}_n$, so these vectors span $\text{range}(T)$.
Linear Independence: Suppose $c_1T\mathbf{w}_1 + \cdots + c_nT\mathbf{w}_1 = \mathbf{0}$.
Then $T(c_1\mathbf{w}_1 + \cdots + c_n\mathbf{w}_n) = \mathbf{0}$, so $c_1\mathbf{w}_1 + \cdots + c_n\mathbf{w}_n \in \text{null}(T)$. Hence there exists scalars $d_1, \ldots, d_m$ such that $c_1\mathbf{w}_1 + \cdots + c_n\mathbf{w}_n = d_1\mathbf{u}_1 + \cdots + d_m\mathbf{u}_m$. Rewriting, $d_1\mathbf{u}_1 + \cdots + d_m\mathbf{u}_m - c_1\mathbf{w}_1 - \cdots - c_n\mathbf{w}_n = \mathbf{0}$.
Because $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{w}_1, \ldots, \mathbf{w}_n)$ is a basis of $\mathcal{V}$, all coefficients must be 0. Hence $c_1 = \cdots = c_n = 0$, so the vectors $T\mathbf{w}_1, \ldots, T\mathbf{w}_n$ are linearly independent. Thus $(T\mathbf{w}_1, \ldots, T\mathbf{w}_n)$ is a basis of $\text{range}(T)$, and so $\dim(\text{range}(T)) = n$. Therefore $\dim(\mathcal{V}) = m + n = \dim(\text{null}(T)) + \dim(\text{range}(T))$. □

This decomposes $\mathcal{V}$ into directions annihilated by T (nullspace), and those survive under T (by $\mathbf{w}_j$).

Corollary 1.2.33. *If $\mathcal{V}$ and $\mathcal{W}$ are finite-dimensional vector spaces such that dim$(\mathcal{V})$ > dim$(\mathcal{W})$, then no linear map from $\mathcal{V}$ to $\mathcal{W}$ is injective.*

Proof. Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. By Rank-Nullity Theorem 1.2.32, $\dim(\mathcal{V}) = \dim(\text{null}(T)) + \dim(\text{range}(T))$.
Because $\text{range}(T)$ is a subspace of $\mathcal{W}$, we have $\dim(\text{range}(T)) \leq \dim(\mathcal{W})$. Thus

$$\dim(\text{null}(T)) = \dim(\mathcal{V}) - \dim(\text{range}(T)) \geq \dim(\mathcal{V}) - \dim(\mathcal{W})$$

Since $\dim(\mathcal{V}) > \dim(\mathcal{W})$, it follows that $\dim(\text{null}(T)) > 0$. Hence $\text{null}(T)$ contains a nonzero vector, so T is not injective by Proposition 1.2.29. □

You cannot fit a higher-dimensional space injectively into a lower-dimensional one. Rank–nullity quantifies this: the deficit in dimensions must appear as a nontrivial nullspace.

Corollary 1.2.34. *If $\mathcal{V}$ and $\mathcal{W}$ are finite-dimensional vector spaces such that $\dim(\mathcal{V}) < \dim(\mathcal{W})$, then no linear map from $\mathcal{V}$ to $\mathcal{W}$ is surjective.*

Proof. Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. By Rank-Nullity Theorem 1.2.32, $\dim(\text{range}(T)) = \dim(\mathcal{V}) - \dim(\text{null}(T))$.
Because $\dim(\text{null}(T)) \geq 0$, it follows that $\dim(\text{range}(T)) \leq \dim(\mathcal{V}) < \dim(\mathcal{W})$.
Thus $\text{range}(T)$ is a proper subspace of $\mathcal{W}$, so $\text{range}(T) \neq \mathcal{W}$. Hence T is not surjective. □

A linear map cannot cover a higher-dimensional space if its domain has fewer degrees of freedom. Rank–nullity makes this precise by bounding the dimension of the range by $\dim(\mathcal{V})$.

Definition 1.2.35. A $m \times n$ *matrix* $\mathcal{M}$ has m rows and n columns,

$$\begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{m,1} & \cdots & a_{m,n} \end{bmatrix}$$

Proposition 1.2.36. *Suppose $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, and $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is a basis of $\mathcal{V}$, and $(\mathbf{w}_1, \ldots, \mathbf{w}_m)$ is a basis of $\mathcal{W}$. Then $\mathcal{M}(T\mathbf{v}) = \mathcal{M}(T)\mathcal{M}(\mathbf{v})$ for every $\mathbf{v} \in \mathcal{V}$.*

Proof. Let

$$\mathcal{M}(\mathbf{v}) = \begin{bmatrix} \mathbf{b}_1 \\ \vdots \mathbf{b}_n \end{bmatrix}$$

so that $\mathbf{v} = \mathbf{b}_1\mathbf{v}_1 + \cdots + \mathbf{b}_n\mathbf{v}_n$. By linearity of T, $T\mathbf{v} = \mathbf{b}_1 T\mathbf{v}_1 + \cdots + \mathbf{b}_n T\mathbf{v}_n$.
By definition of $\mathcal{M}(T)$, the k-th column of $\mathcal{M}(T)$ is $\mathcal{M}(T\mathbf{v}_k)$, therefore $\mathcal{M}(T\mathbf{v}) = \mathbf{b}_1\mathcal{M}(T\mathbf{v}_1) + \cdots + \mathbf{b}_n\mathcal{M}(T\mathbf{v}_n)$.
But multiplying $\mathcal{M}(T)$ by $\mathcal{M}(\mathbf{v})$ forms exactly this same linear combination of the columns of $\mathcal{M}(T)$.
Hence $\mathcal{M}(T\mathbf{v}) = \mathcal{M}(T)\mathcal{M}(\mathbf{v})$. □

Definition 1.2.37. A linear map $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$ is *invertible* if $\exists$ linear map $S \in \mathcal{L}(\mathcal{W}, \mathcal{V})$ such that ST equals identity map on $\mathcal{V}$ and TS equals identity map on $\mathcal{W}$.
A linear map $S \in \mathcal{L}(\mathcal{W}, \mathcal{V})$ satisfying $ST = I$ and $TS = I$ is the *inverse* of T.

Proposition 1.2.38. *A linear map is invertible if and only if it is injective and surjective.*

Proof. ($\Rightarrow$) Suppose T is invertible with inverse T^{-1}. If $T(\mathbf{u}) = T(\mathbf{v})$, then applying T^{-1} gives

$$\mathbf{u} = T^{-1}(T(\mathbf{u})) = T^{-1}(T(\mathbf{v})) = \mathbf{v}$$

so T is injective. For any $\mathbf{w} \in \mathcal{W}$, we have $\mathbf{w} = T(T^{-1}(\mathbf{w}))$, so $\mathbf{w} \in \text{range}(T)$, hence T is surjective.
($\Leftarrow$) Suppose T is injective and surjective. Define $S : \mathcal{W} \to \mathcal{V}$ as follows: for each $\mathbf{w} \in \mathcal{W}$, let $S(\mathbf{w})$ be the unique vector $\mathbf{v} \in \mathcal{V}$ such that $T(\mathbf{v}) = \mathbf{w}$. (Surjectivity ensures existence; injectivity ensures uniqueness.)
Then for all $\mathbf{w} \in \mathcal{W}$, $T(S(\mathbf{w})) = \mathbf{w}$, so $T \circ S = I_{\mathcal{W}}$.
For $\mathbf{v} \in \mathcal{V}$, let $\mathbf{w} = T(\mathbf{v})$, then $S(T(\mathbf{v})) = S(\mathbf{w}) = \mathbf{v}$, so $S \circ T = I_{\mathcal{V}}$.
To show that S is linear. Let $\mathbf{w}_1, \mathbf{w}_2 \in \mathcal{W}$, and $\alpha \in \mathbb{F}$, then $T(S(\mathbf{w}_1 + \mathbf{w}_2)) = \mathbf{w}_1 + \mathbf{w}_2 = T(S(\mathbf{w}_1) + S(\mathbf{w}_2))$.
By injectivity of T, this implies $S(\mathbf{w}_1 + \mathbf{w}_2) = S(\mathbf{w}_1) + S(\mathbf{w}_2)$.
Similarly, $T(S(\alpha\mathbf{w}_1)) = \alpha\mathbf{w}_1 = T(\alpha S(\mathbf{w}_1))$ so $S(\alpha\mathbf{w}_1) = \alpha S(\mathbf{w}_1)$. Thus S is linear, hence $S \in \mathcal{L}(\mathcal{W}, \mathcal{V})$, and S is the inverse of T. Therefore T is invertible. □

Invertibility is equivalent to having a well-defined undo operation. Surjectivity guarantees you can undo every output (existence), and injectivity guarantees the undoing is unique; together giving a bona fide inverse.

Definition 1.2.39. Two vector spaces are *isomorphic* if there is an invertible linear map from one vector space onto the other. Two isomorphic spaces have the same properties.

Theorem 1.2.40. *Two finite-dimensional vector spaces are isomorphic if and only if they have the same dim.*

Proof. ($\Rightarrow$) Suppose $\mathcal{V} \cong \mathcal{W}$. Then there exists an invertible linear map $T : \mathcal{V} \to \mathcal{W}$. Hence $\text{null}(T) = \{\mathbf{0}\}$ and $\text{range}(T) = \mathcal{W}$. By Rank-Nullity Theorem 1.2.32,

$$\dim(\mathcal{V}) = \dim(\text{null}(T)) + \dim(\text{range}(T)) = 0 + \dim(\mathcal{W}) = \dim(\mathcal{W})$$

($\Leftarrow$) Suppose $\dim(\mathcal{V}) = \dim(\mathcal{W}) = n$. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be a basis of $\mathcal{V}$ and $(\mathbf{w}_1, \ldots, \mathbf{w}_n)$ be basis of $\mathcal{W}$.
Define $T : \mathcal{V} \to \mathcal{W}$ by $T\left(\sum_{i=1}^{n} a_i \mathbf{v}_i\right) = \sum_{i=1}^{n} a_i \mathbf{w}_i$. This is well-defined because each vector in $\mathcal{V}$ has a unique representation in basis $(\mathbf{v}_i)$, and it is linear because it preserves linear combinations.
To show surjectivity, let $\mathbf{w} \in \mathcal{W}$. Then $\mathbf{w} = \sum_{i=1}^{n} a_i \mathbf{w}_i = T\left(\sum_{i=1}^{n} a_i \mathbf{v}_i\right)$, so $\mathbf{w} \in \text{range}(T)$.
To show injectivity, suppose $T\left(\sum_{i=1}^{n} a_i \mathbf{v}_i\right) = 0$. Then $\sum_{i=1}^{n} a_i \mathbf{w}_i = \mathbf{0}$. Because $(\mathbf{w}_i, \ldots, \mathbf{w}_n)$ is linear independent, all $a_i = 0$. Hence the original vector is $\mathbf{0}$, so $\text{null}(T) = \{\mathbf{0}\}$, and T is injective.
Thus T is bijective and linear, hence an isomorphism. Therefore $\mathcal{V} \cong \mathcal{W}$. □

Finite-dimensional vector spaces are completely classified by dimension: once dimensions match, you can construct an isomorphism by mapping one basis to another and extending linearly.

Theorem 1.2.41. *Suppose $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is a basis of $\mathcal{V}$ and $(\mathbf{w}_1, \cdots, \mathbf{w}_m)$ is a basis of $\mathcal{W}$. Then $\mathcal{M}$ is an invertible linear map between $\mathcal{L}(\mathcal{V}, \mathcal{W})$ and Mat$(m, n, \mathbb{F})$,*

Proof. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be a basis of $\mathcal{V}$, and $(\mathbf{w}_1, \ldots, \mathbf{w}_m)$ be a basis of $\mathcal{W}$. Define $\mathcal{M} : \mathcal{L}(\mathcal{V}, \mathcal{W}) \to \text{Mat}(m, n, \mathbb{F})$ by sending T to its matrix relative to these basis.
First, $\mathcal{M}$ is linear as the matrix entries depend linearly on T.
To show $\mathcal{M}$ is invertible, it suffices to show it is bijective.
Injectivity: Suppose $\mathcal{M}(T) = 0$. Then every column of $\mathcal{M}(T)$ is $\mathbf{0}$, so for each $k = 1, \ldots, n$, $T\mathbf{v}_k = \mathbf{0}$.
Because $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is a basis of $\mathcal{V}$, this implies $T = 0$. Hence $\mathcal{M}$ is injective.
Surjectivity: Let $A = [a_{j,k}] \in \text{Mat}(m, n, \mathbb{F})$. Define $T : \mathcal{V} \to \mathcal{W}$ by specifying its action on basis $T\mathbf{v}_k = \sum_{j=1}^{m} a_{j,k} \mathbf{w}_j$ for $k = 1, \ldots, n$, and extend linearly to all of $\mathcal{V}$. Then T is linear, and by construction the k-th column of $\mathcal{M}(T)$ is the coordinate vector of $T\mathbf{v}_k$, which is exactly the k-t column of A. Hence $\mathcal{M}(T) = A$.
Thus $\mathcal{M}$ is surjective. Therefore $\mathcal{M}$ is bijective and hence an isomorphism. □

A linear map is completely determined by its action on a basis. The matrix $\mathcal{M}(T)$ records exactly this data, and every matrix corresponds to a unique such assignment. This establishes a one-to-one correspondence between $\mathcal{L}(\mathcal{V}, \mathcal{W})$ and Mat$(m, n, \mathbb{F})$.

Proposition 1.2.42. *If $\mathcal{V}$ and $\mathcal{W}$ are finite-dimensional, then $\mathcal{L}(\mathcal{V}, \mathcal{W})$ is finite-dimensional, and*

$$dim\mathcal{L}(\mathcal{V}, \mathcal{W}) = (dim(\mathcal{V}))(dim(\mathcal{W}))$$

Proof. Let $\dim(\mathcal{V}) = n$ and $\dim(\mathcal{W}) = m$. By Theorem 1.2.41, $\mathcal{L}(\mathcal{V}, \mathcal{W})$ is isomorphic to Mat$(m, n, \mathbb{F})$.
The space Mat$(m, n, \mathbb{F})$ has dimension mn, because a basis is given by the matrices $E_{j,k}$ that have a 1 in the (i, j)-entry and 0 elsewhere for $1 \leq j \leq m, 1 \leq k \leq n$.
Because isomorphic vector spaces have the same dimension, hence $\dim\mathcal{L}(\mathcal{V}, \mathcal{W}) = mn = (\dim(\mathcal{V}))(\dim(\mathcal{W}))$. □

The result is a direct consequence of identifying linear maps with matrices. Once this identification is made, counting degrees of freedom becomes counting matrix entries, giving mn.

Definition 1.2.43. A linear map from a vector space to itself is an *operator*, denoted $\mathcal{L}(\mathcal{V})$.

Theorem 1.2.44. *Suppose $\mathcal{V}$ is finite-dimensional. If $T \in \mathcal{L}(\mathcal{V})$, then the following are equivalent:*

i. T is invertible.

ii. T is injective.

iii. T is surjective.

Proof. Suppose $\mathcal{V}$ is finite-dimensional and $T \in \mathcal{L}(\mathcal{V})$.

$(i) \Rightarrow (ii)$ If T is invertible, then it has an inverse, so it is injective.

$(ii) \Rightarrow (iii)$ Suppose T is injective, then $\text{null}(T) = \{\mathbf{0}\}$, so $\dim(\text{null}(T)) = 0$. By Rank-Nullity Theorem 1.2.32, $\dim(\text{range}(T)) = \dim(\mathcal{V}) - \dim(\text{null}(T)) = \dim(\mathcal{V})$. Because range$(T)$ is a subspace of $\mathcal{V}$ with same dimension as $\mathcal{V}$, it follows that $\text{range}(T) = \mathcal{V}$, so T is surjective.

$(iii) \Rightarrow (i)$ Suppose T is surjective, then $\text{range}(T) = \mathcal{V}$, so $\dim(\text{range}(T)) = \dim(\mathcal{V})$.
By Rank-Nullity Theorem 1.2.32, $\dim(\mathcal{V}) = \dim(\text{null}(T)) + \dim(\text{range}(T)) = \dim(\text{null}(T)) + \dim(\mathcal{V})$, which implies $\dim(\text{null}(T)) = 0$. Thus $\text{null}(T) = \{\mathbf{0}\}$, so T is injective.
Therefore T is both injective and surjective, hence invertible.

□

In finite dimensions, injectivity and surjectivity are not independent: rank–nullity forces a trade-off between null and range dimensions. If one is maximal, the other must be as well, collapsing the distinction between injective, surjective, and invertible.

Definition 1.2.45. *Addition and Multiplication Modulo n*
Let $1 < n \in \mathbb{Z}$. For $a, b \in \mathbb{Z}$ with $0 \leq a, b < n$, choose arbitrary $A, B \in \mathbb{Z}$ s.t. $A \equiv a \pmod{n}$, $B \equiv b \pmod{n}$.

i. *Addition Modulo n*: $a +_n b := (a + b) \pmod{n}$
ii. *Multiplication Modulo n*: $a \cdot_n b := (a \cdot b) \pmod{n}$

The set $\{0, 1 \ldots, n-1\}$ equipped with these operations is denoted $\mathbb{Z}/n\mathbb{Z}$.

Theorem 1.2.46. *The Prime Fields Theorem for Finite Fields*
Let $1 < n \in \mathbb{Z}$, then $\mathbb{Z}/n\mathbb{Z}$ is a field if and only if n is prime.

Proof. Let $n > 1$.

($\Rightarrow$) If n is not prime, then $\mathbb{Z}/n\mathbb{Z}$ is not a field.
If n is composite, write $n = rs$ with $1 < r, s < n$. Then $\mathbb{Z}/n\mathbb{Z}$, $[r][s] = [rs] = [0]$ while $[r] \neq 0$ and $[s] \neq 0$. Thus $\exists$ nonzero elements whose product is 0, i.e., nonzero elements in nullspace of multiplication by $[r]$. Equivalently, consider the linear map $\mathcal{M}_{[r]} : \mathbb{Z}/n\mathbb{Z} \to \mathbb{Z}/n\mathbb{Z}$, $\mathcal{M}_{[r]}([x]) = [r][x]$. Then $[s] \neq 0$ lies in $\text{null}(\mathcal{M}_{[r]})$, so the nullspace is nontrivial. Hence $\mathcal{M}_{[r]}$ is not injective, so $[r]$ cannot have a multiplicative inverse. Therefore the ring is not a field.

($\Leftarrow$) If $n = p$ is prime, then $\mathbb{Z}/p\mathbb{Z}$ is a field.
Fix a nonzero element $[x] \in \mathbb{Z}/p\mathbb{Z}$. Consider the linear map $\mathcal{M}_{[x]} : \mathbb{Z}/p\mathbb{Z} \to \mathbb{Z}/p\mathbb{Z}$, $\mathcal{M}_{[x]}([z]) = [x][z]$.
To show $\text{null}(\mathcal{M}_{[x]}) = \{0\}$. Indeed, if $[x][z] = [0]$, then $p \mid xz$. Since p is prime and $x \not\equiv 0 \pmod{p}$, it follows that $p \mid z$, hence $[z] = 0$. Thus the nullspace is trivial, so $\mathcal{M}_{[x]}$ is injective. Because $\mathbb{Z}/p\mathbb{Z}$ is finite, injectivity implies surjectivity. Hence $\exists$ $[z]$ such that $[x][z] = [1]$, so $[x]$ has a multiplicative inverse.
Since every nonzero element is invertible, $\mathbb{Z}/p\mathbb{Z}$ is a field.

□

The entire theorem reduces to:

i. Composite n: multiplication map has nontrivial nullspace $\to$ zero-divisors $\to$ not invertible.
ii. Prime p: multiplication by nonzero element has trivial nullspace $\to$ injective $\to$ bijective $\to$ invertible.

The argument is parallel to standard results about linear operators:

$$\text{null}(T) = \{0\} \Leftrightarrow T \text{ injective } \Leftrightarrow T \text{ invertible (finite-dimensional case)}$$

Definition 1.2.47. Let $T \in \mathcal{L}(\mathcal{V})$, and $\exists$ direct sum decomposition $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$ where each $\mathcal{U}_j$ is proper subspace of $\mathcal{V}$. The *restriction* of T to smaller domain $\mathcal{U}_j$ is denoted $T|_{\mathcal{U}_j}$

Definition 1.2.48. For $T \in \mathcal{L}(\mathcal{V})$ and $\mathcal{U}$ a subspace of $\mathcal{V}$, $\mathcal{U}$ is invariant under T if $\mathbf{u} \in \mathcal{U}$ implies $T\mathbf{u} \in \mathcal{U}$ ($\mathcal{U}$ is invariant under T if $T|_{\mathcal{U}}$ is an operator under $\mathcal{U}$.

Definition 1.2.49. A scalar $\lambda \in \mathbb{F}$ is an *eigenvalue* of $T \in \mathcal{L}(\mathcal{V})$ if $\exists$ a nonzero vector $\mathbf{u} \in \mathcal{V}$ such that $T\mathbf{u} = \lambda\mathbf{u}$. A vector $\mathbf{u} \in \mathcal{V}$ is an *eigenvector* of T if $T\mathbf{u} = \lambda\mathbf{u}$.

Note that the set of eigenvectors of T corresponding to λ equals $\text{null}(T - \lambda I)$.

Theorem 1.2.50. *Let $T \in \mathcal{L}(\mathcal{V})$, and $\lambda_1, \ldots, \lambda_m$ distinct eigenvalues of T, $\mathbf{v}_1, \ldots, \mathbf{v}_m$ are corresponding nonzero eigenvectors, then $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is linearly independent.*

Proof. Let $T \in \mathcal{L}(\mathcal{V})$, let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues with corresponding nonzero eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_m$. Assume for contradiction that $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is linearly dependent. By linear dependence lemma 1.2.8, $\exists$ smallest index $k \geq 2$ such that $\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{k-1})$. Thus $\exists$ scalars $a_1, \ldots, a_{k-1}$ such that $\mathbf{v}_k = a_1\mathbf{v}_1 + \cdots + a_{k-1}\mathbf{v}_{k-1}$. By minimality of k, the list $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is linearly independent.
Applying T to both sides, $T(\mathbf{v}_k) = a_1T(\mathbf{v}_1) + \cdots + a_{k-1}T(\mathbf{v}_{k-1})$. Using the eigenvector relations $T(\mathbf{v}_i) = \lambda_i\mathbf{v}_i$,

$$\lambda_k\mathbf{v}_k = a_1\lambda_1\mathbf{v}_1 + \cdots + a_{k-1}\lambda_{k-1}\mathbf{v}_{k-1}$$

Multiply the original equation by λ_k,

$$\lambda_k \mathbf{v}_k = a_1 \lambda_k \mathbf{v}_1 + \cdots + a_{k-1} \lambda_k \mathbf{v}_{k-1}$$

Subtracting the two equations gives

$$0 = \sum_{i=1}^{k-1} a_i (\lambda_k - \lambda_i) \mathbf{v}_i$$

Because $(\mathbf{v}_1, \ldots, \mathbf{v}_{k-1})$ is linearly independent, each coefficient must be zero, $a_i(\lambda_k - \lambda_i) = 0$ for all $i < k$.
Since the eigenvalues are distinct $(\lambda_k - \lambda_i \neq 0)$, hence $a_i = 0$ for all i.
Thus $\mathbf{v}_k = \mathbf{0}$, contradicting that $\mathbf{v}_k$ is an eigenvector. Therefore, $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is linearly independent. □

Corollary 1.2.51. *Each operator on $\mathcal{V}$ has at most $\dim(\mathcal{V})$ distinct eigenvalues.*

Proof. Let $T \in \mathcal{L}(\mathcal{V})$, and $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T. For each i, choose a corresponding nonzero eigenvector $\mathbf{v}_i \in \mathcal{V}$. By Theorem 1.2.50, the list $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is linearly independent.
Let $(\mathbf{w}_1, \ldots, \mathbf{w}_n)$ be a basis of $\mathcal{V}$, then this list spans $\mathcal{V}$ and has length $n = \dim(\mathcal{V})$. Applying Theorem 1.2.9 to the linearly independent list $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ and the spanning list $(\mathbf{w}_1, \ldots, \mathbf{w}_n)$, we obtain $m \leq n = \dim(\mathcal{V})$.
Hence T has at most $\dim(\mathcal{V})$ distinct eigenvalues. □

Theorem 1.2.52. *Every operator on a finite-dimensional, nonzero complex vector space has an eigenvalue.*

Proof. Let $\mathcal{V}$ be a complex vector space with $\dim(\mathcal{V}) = n > 0$, and let $T \in \mathcal{L}(\mathcal{V})$. Choose $\mathbf{v} \in \mathcal{V}$ with $\mathbf{v} \neq \mathbf{0}$. Consider the list $(\mathbf{v}, T\mathbf{v}, T^2\mathbf{v}, \ldots, T^n\mathbf{v})$, a list of $n+1$ vectors in a n-dimensional space, hence it is linearly dependent. Therefore $\exists$ scalars $a_0, \ldots, a_m \in \mathbb{C}$, not all zero, such that

$$0 = a_0 \mathbf{v} + a_1 T\mathbf{v} + \cdots + a_n T^n \mathbf{v}$$

Let m be largest index such that $a_m \neq 0$. Then $m \geq 1$, and defining the polynomial

$$p(z) = a_0 + a_1 z + \cdots + a_m z^m$$

we have $p(T)\mathbf{v} = 0$. As $\mathbb{C}$ is algebraically closed, the polynomial p factors as $p(z) = c(z - \lambda_1) \cdots (z - \lambda_m)$, where $c \neq 0$ and $\lambda_1, \ldots, \lambda_m \in \mathbb{C}$. Substituting T for $\mathbf{z}$, we obtain $p(T) = c(T - \lambda_1 I) \cdots (T - \lambda_m I)$ and hence

$$c(T - \lambda_1 I) \cdots (T - \lambda_m I)\mathbf{v} = \mathbf{0}$$

Because $\mathbf{v} \neq \mathbf{0}$, this implies the operator $(T - \lambda_1 I) \cdots (T - \lambda_m I)$ is not injective. Therefore at least one factor $T - \lambda_j I$ is not injective. Hence $\text{null}(T - \lambda_i I) \neq \{0\}$, so there exists a nonzero vector $\mathbf{u}$ such that $T(\mathbf{u}) = \lambda_j \mathbf{u}$.
Thus λ_j is an eigenvalue of T. □

Definition 1.2.53. *Upper triangular matrices* are of the form

$$\begin{bmatrix} \lambda_1 & & * \\ & \ddots & \\ 0 & & \lambda_n \end{bmatrix}$$

Next proposition shows useful connection between upper-triangular matrices and invariant subspaces.

Proposition 1.2.54. *Suppose $T \in \mathcal{L}(\mathcal{V})$, and $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is a basis of $\mathcal{V}$. Following are equivalent:*

i. The matrix of T with respect to $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is upper triangular.

ii. $T\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ for each $k = 1, \ldots, n$

iii. $\text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is invariant under T for each $k = 1, \ldots, n$

Proof.

$(i) \Rightarrow (ii)$ If the matrix T is upper triangular, then for each k, the coordinate representation of $T\mathbf{v}_k$ has nonzero entries below the k-th position. Hence $T\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$.

$(ii) \Rightarrow (iii)$ Fix k, and let $\mathbf{u} \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$. Then $\mathbf{u} = a_1\mathbf{v}_1 + \cdots + a_k\mathbf{v}_k$.
By linearity, $T\mathbf{u} = a_1 T\mathbf{v}_1 + \cdots + a_k T\mathbf{v}_k$. By (ii), each $T\mathbf{v}_j \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$, hence $T\mathbf{u}$ lies in the same span. Therefore, $T(\text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)) \subseteq \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$, so the subspace is invariant.

$(iii) \Rightarrow (ii)$ Fix k. Since $\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$ and this subspace is invariant under T, $T\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$.

(*ii*) $\Rightarrow$ (*i*) For each k, $T\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$. Then the coordinate vector of $T\mathbf{v}_k$ relative to the basis $(\mathbf{v}_1, \ldots \mathbf{v}_n)$ has zero coefficients on $\mathbf{v}_{k+1}, \ldots, \mathbf{v}_n$. Therefore, the k-th column of the matrix of T has zeroes below the diagonal. Since this holds for every k, the matrix of T is upper triangular.

□

An operator is upper triangular if and only if it preserves a nested sequence of subspaces (a flag):

$$\{0\} \subset \text{span}((v)_1) \subset \text{span}((v)_1, (v)_2) \subset \cdots \subset \text{span}((v)_1, \ldots, (v)_n) = \mathcal{V}$$

Theorem 1.2.55. *Schur Triangularisation Theorem Suppose $\mathcal{V}$ is a complex vector space and $T \in \mathcal{L}(\mathcal{V})$, then T has an upper-triangular matrix with respect to some basis of $\mathcal{V}$.*

Proof. Proceed by induction on $\dim(\mathcal{V})$. The result is trivial if $\dim(\mathcal{V}) = 1$.
Assume $\dim(\mathcal{V}) > 1$ and the result holds for all smaller dimensions. Because $\mathcal{V}$ is a complex vector space, and T has eigenvalue λ, thus $\text{null}(T - \lambda I) \neq \{\mathbf{0}\}$. Let $\mathcal{U} = \text{null}(T - \lambda I)$. Then $\mathcal{U}$ is a nonzero subspace of $\mathcal{V}$ and is invariant under T: if $\mathbf{u} \in \mathcal{U}$, then

$$(T - \lambda I)\mathbf{u} = \mathbf{0} \;\Rightarrow T\mathbf{u} = \lambda\mathbf{u} \;\in \mathcal{U}$$

Hence $T|_{\mathcal{U}}$ is a linear operator on $\mathcal{U}$. Since $\dim(\mathcal{U}) < \dim(\mathcal{V})$, by induction hypothesis $\exists$ a basis $(\mathbf{u}_1, \ldots, \mathbf{u}_m)$ of $\mathcal{U}$ such that $T|_{\mathcal{U}}$ has an upper-triangular matrix. Equivalently,

$$T\mathbf{u}_j \in \text{span}(\mathbf{u}_1, \ldots, \mathbf{u}_j) \quad \text{for each } j$$

Extend this to a basis of $\mathcal{V}$, $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_k)$.
For each $\mathbf{v}_k$, $(T - \lambda I)\mathbf{v}_k \in \text{range}(T - \lambda I)$. But more importantly, decompose into $T\mathbf{v}_k = (T - \lambda I)\mathbf{v}_k + \lambda\mathbf{v}_k$.
Since $(T - \lambda I)\mathbf{v}_k \in \mathcal{U} = \text{span}(\mathbf{u}_1, \ldots, \mathbf{u}_m)$, we obtain $T\mathbf{v}_k \in \text{span}(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_k)$.
Thus with respect to the basis $(\mathbf{u}_1, \ldots, \mathbf{u}_m, \mathbf{v}_1, \ldots, \mathbf{v}_k)$, the matrix of T is upper triangular. □

Over $\mathbb{C}$, every operator has an eigenvector, which gives a nontrivial invariant subspace (via the nullspace of $T - \lambda I$); recursively building such invariant subspaces yields an upper-triangular representation.

Proposition 1.2.56. *Suppose $T \in \mathcal{L}(\mathcal{V})$ has an upper-triangular matrix with respect to some basis of $\mathcal{V}$. Then T is invertible if and only if all entries of the diagonal of upper-triangular matrix are non-zero.*

Proof. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be basis w.r.t. which T has an upper-triangular matrix with diagonal entries $\lambda_1, \ldots, \lambda_n$.
($\Rightarrow$) Suppose T is invertible. Then $\text{null}(T) = \{\mathbf{0}\}$.
Assume for contradiction that $\lambda_k = 0$ for some k. Consider $\mathbf{v}_k$. By upper-triangularity, $T\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_k)$, and the coefficient of $\mathbf{v}_k$ in $T\mathbf{v}_k$ is exactly $\lambda_k = 0$. Hence $T\mathbf{v}_k \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{k-1})$.
Now consider vectors of the form

$$\mathbf{v} = \mathbf{v}_k + \sum_{j<k} a_j \mathbf{v}_j$$

By choosing coefficients a_j recursively, we can cancel all components of $T\mathbf{v}_k$, yielding a nonzero vector $\mathbf{v}$ such that $T\mathbf{v} = 0$. Thus $\text{null}(T) \neq \{\mathbf{0}\}$, contradicting invertibility.
($\Leftarrow$) Suppose all $\lambda_k \neq 0$. To show $\text{null}(T) = \{\mathbf{0}\}$. Let $\mathbf{v} = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n \in \text{null}(T)$ so $T\mathbf{v} = 0$. Let k be largest index such that $a_K \neq 0$. Then $0 = T\mathbf{v} = a_1 T\mathbf{v}_1 + \cdots + a_k T\mathbf{v}_k$.
By upper-triangularity, $T\mathbf{v}_j \in \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_j)$, and the coefficient of $\mathbf{v}_k$ appears only in $a_k T\mathbf{v}_k$ and equals $a_k\lambda_k$. Thus the coefficient of $\mathbf{v}_k$ in $T\mathbf{v}$ is $a_k\lambda_k$. Since $\lambda_k \neq 0$ and $a_k \neq 0$, this is non-zero, a contradiction.
Hence no such k exists, so all $a_k = 0$, therefore $\mathbf{v} = \mathbf{0}$. Thus $\text{null}(T) = \{\mathbf{0}\}$, so T is invertible. □

Upper-triangular structure isolates each coordinate sequentially. Invertibility reduces to checking that no coordinate gets killed, i.e., all diagonal entries are nonzero, ensuring a trivial nullspace.

Proposition 1.2.57. *Suppose $T \in \mathcal{L}(\mathcal{V})$ has upper-triangular matrix with respect to some basis of $\mathcal{V}$, then eigenvalues of T consists of precisely the entries of the diagonal of the upper-triangular matrix.*

Proof. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be a basis of $\mathcal{V}$ s.t. T has an upper-triangular matrix with diagonal entries $\lambda_1, \ldots, \lambda_n$.
Fix $\lambda \in \mathbb{F}$. Then the matrix of $T - \lambda I$ w.r.t. this basis is also upper triangular with diagonal $\lambda_1 - \lambda, \ldots, \lambda_n - \lambda$.
By Proposition 1.2.56, $T - \lambda I$ is invertible if and only if all diagonal entries are nonzero, $\lambda_j - \lambda \neq 0$ for all j.
Equivalently, $T - \lambda I$ is not invertible $\Leftrightarrow \lambda = \lambda_j$ for some j.
Using nullspace characterisation, λ is an eigenvalue of $T \Leftrightarrow \text{null}(T - \lambda I) \neq \{\mathbf{0}\} \Leftrightarrow T - \lambda I$ is not invertible.
Therefore, λ is an eigenvalue of $T \Leftrightarrow \lambda \in \{\lambda_1, \ldots \lambda_n\}$. □

Upper-triangular structure reduces the eigenvalue problem to diagonal entries: solving $(T - \lambda I)\mathbf{v} = \mathbf{0}$ depends only on whether a diagonal term vanishes, making eigenvalues exactly the diagonal entries.

Definition 1.2.58. A *diagonal matrix* is a square matrix that is 0 everywhere except possibly along diagonal.

Operator $T \in \mathcal{L}(\mathcal{V})$ has diagonal matrix $\text{diag}(\lambda_1, \ldots, \lambda_n)$ w.r.t. basis $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ of $\mathcal{V}$ if and only if

$$\begin{aligned} T\mathbf{v}_1 &= \lambda_1 \mathbf{v}_1 \\ &\vdots \\ T\mathbf{v}_n &= \lambda_n \mathbf{v}_n \end{aligned}$$

This follows from definition of matrix of operator with respect to a basis,

If an operator has a diagonal matrix w.r.t. some basis, the entires along diagonal are precisely eigenvalues.

Proposition 1.2.59. *If $T \in \mathcal{L}(\mathcal{V})$ has $\dim(\mathcal{V})$ distinct eigenvalues, then T has a diagonal matrix with respect to some basis of $\mathcal{V}$.*

Proof. Suppose $T \in \mathcal{L}(\mathcal{V})$ has $\dim(\mathcal{V})$ distinct eigenvalues $\lambda_1, \ldots, \lambda_n$, where $n = \dim(\mathcal{V})$.
For each j, choose a nonzero eigenvector $\mathbf{v}_j \in \mathcal{V}$ such that $T\mathbf{v}_j = \lambda_j \mathbf{v}_j$. Because the eigenvalues are distinct, the list $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is linearly dependent (by Theorem 1.2.50). Since this list has length $n = \dim(\mathcal{V})$, it forms a basis of $\mathcal{V}$ (by Proposition 1.2.21).
With respect to this basis, the action of T is given by $T\mathbf{v}_j = \lambda_j \mathbf{v}_j$ for each j. Thus the matrix of T has λ_j on the diagonal and zero elsewhere, i.e., it is diagonal □

Distinct eigenvalues force eigenvectors to be independent; having enough of them (equal to the dimension) produces a basis, and in that basis the operator acts by simple scaling—hence diagonalisation.

Proposition 1.2.60. *Suppose $T \in \mathcal{L}(\mathcal{V})$. Let $\lambda_1, \ldots, \lambda_m$ denote distinct eigenvalues of T. Then the following are equivalent:*

i. T has a diagonal matrix with respect to some basis of $\mathcal{V}$;

ii. $\mathcal{V}$ has a basis consisting of eigenvectors of T;

iii. $\exists$ one-dimensional subspaces $\mathcal{U}_1, \ldots, \mathcal{U}_n$ of $\mathcal{V}$, each invariant under T, such that

$$\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_n$$

iv. $\mathcal{V} = \text{null}(T - \lambda_1 I) \oplus \cdots \oplus \text{null}(T - \lambda_m I)$

v. $\dim(\mathcal{V}) = \dim(\text{null}(T - \lambda_1 I)) + \cdots + \dim(\text{null}(T - \lambda_m I))$

Proof.

(*i*) $\Rightarrow$ (*ii*) Suppose T has a diagonal matrix w.r.t. some basis $(\mathbf{v}_1, \ldots, \mathbf{v_n})$ of $\mathcal{V}$. Then for each j, $T\mathbf{v}_j = \alpha_j \mathbf{v}_j$ for some scalar α_j. Thus each $\mathbf{v}_j$ is an eigenvector of T, so $\mathcal{V}$ has a basis consisting of eigenvectors.

(*i*) $\Leftarrow$ (*ii*) Suppose $\mathcal{V}$ has a basis $(\mathbf{v}_1, \ldots, \mathbf{v_n})$ consisting of eigenvectors of T. Then for each j, $\exists$ a scalar α_j such that $T\mathbf{v}_j = \alpha_j \mathbf{v}_j$. Therefore the matrix of T w.r.t. this basis has only diagonal entries possibly nonzero. Hence the matrix is diagonal.

(*ii*) $\Rightarrow$ (*iii*) Suppose $\mathcal{V}$ has a basis $(\mathbf{v}_1, \ldots, \mathbf{v_n})$ consisting of eigenvectors of T. Define $\mathcal{U}_j = \text{span}(\mathbf{v}_j)$. Each $\mathcal{U}_j$ is one-dimensional. Also, if $\mathbf{u} \in \mathcal{U}_j$, then $\mathbf{u} = a\mathbf{v}_j$ for some scalar a, and $T\mathbf{u} = (a\mathbf{v}_j) = aT\mathbf{v}_j = a\alpha_j \mathbf{v}_j \in \mathcal{U}_j$, which means $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_n$.

(*ii*) $\Leftarrow$ (*iii*) Suppose $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_n$, where $\mathcal{U}_j$ is one-dimensional and invariant under T. Choose a nonzero vector $\mathbf{v}_j \in \mathcal{U}_j$. Since $\mathcal{U}_j$ is one-dimensional, $\mathcal{U}_j = \text{span}(\mathbf{v}_j)$. Because $\mathcal{U}_j$ is invariant under T, we have $T\mathbf{v}_j \in \mathcal{U}_j = \text{span}(\mathbf{v}_j)$. Hence $\exists$ a scalar α_j such that $T\mathbf{v}_j = \alpha_j \mathbf{v}_j$. Thus each $\mathbf{v}_j$ is an eigenvector of T. Since $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_n$, the list $\mathbf{v}_1, \ldots, \mathbf{v_n})$ is linearly independent and spans $\mathcal{V}$. Therefore, it is a basis of $\mathcal{V}$ consisting of eigenvectors.

(*ii*) $\Rightarrow$ (*iv*) Suppose $\mathcal{V}$ has a basis consisting of eigenvectors of T. Every eigenvector belongs to one of eigenspaces $\text{null}(T - \lambda_j I)$. Therefore every vector in $\mathcal{V}$, being a linear combination of basis eigenvectors, belongs to $\text{null}(T - \lambda_1 I) + \cdots + \text{null}(T - \lambda_m I)$. Hence, $\mathcal{V} = \text{null}(T - \lambda_1 I) + \cdots + \text{null}(T - \lambda_m I)$.
It remains to show that the sum is direct. Suppose $\mathbf{u}_1 + \cdots + \mathbf{u}_m = \mathbf{0}$, where $\mathbf{u}_j \in \text{null}(T - \lambda_j I)$. If some $\mathbf{u}_j \neq \mathbf{0}$, then $\mathbf{u}_j$ is an eigenvector of T with eigenvalue λ_j. Since eigenvectors corresponding to distinct eigenvalues are linearly independent, we must have $\mathbf{u}_1 = \cdots = \mathbf{u}_m = \mathbf{0}$. Thus the sum is direct.

$(iv) \Rightarrow (v)$ Suppose $\mathcal{V} = \text{null}(T-\lambda_1 I) \oplus \cdots \oplus \text{null}(T-\lambda_m I)$. For a direct sum of finite-dimensional subspaces, dimensions add. Therefore, $\dim(\mathcal{V}) = \dim(\text{null}(T-\lambda_1 I)) + \cdots + \dim(\text{null}(T-\lambda_m I))$.

$(v) \Rightarrow (ii)$ Suppose $\dim(\mathcal{V}) = \sum_{j=1}^{m} \dim(\text{null}(T-\lambda_j I))$. For each j, choose a basis $\mathcal{B}_j$ of $\text{null}(T-\lambda_j I)$. Every nonzero vector in $\mathcal{B}_j$ is an eigenvector of T with eigenvalue λ_j.

Let $\mathcal{B} = \mathcal{B}_1 \cup \cdots \cup \mathcal{B}_m$. Then $|\mathcal{B}| = \sum_{j=1}^{m} \dim(\text{null}(T-\lambda_j I)) = \dim(\mathcal{V})$.

Claim that $\mathcal{B}$ is linearly independent. Suppose a linear combination of vectors in $\mathcal{B}$ equals zero. Group the terms according to eigenspaces: $\mathbf{u}_1 + \cdots + \mathbf{u}_m = \mathbf{0}$ where $\mathbf{u}_j \in \text{null}(T-\lambda_j I)$. Each nonzero $\mathbf{u}_j$ is an eigenvector of T with eigenvalue λ_j. Since eigenvalues $\lambda_1, \ldots, \lambda_m$ are distinct, eigenvectors corresponding to distinct eigenvalue are linearly independent. Therefore every $\mathbf{u}_j = \mathbf{0}$.
Since each $\mathcal{B}_j$ is itself linearly independent, all coefficients in the original linear combination must be zero. Hence $\mathcal{B}$ is linearly independent. Thus $\mathcal{B}$ is linearly independent of list of length $\dim(\mathcal{V})$, so $\mathcal{B}$ is a basis of $\mathcal{V}$. Since every vector in $\mathcal{B}$ is an eigenvector of T, $\mathcal{V}$ has a basis consisting of eigenvectors of T.

□

Diagonalisability means that $\mathcal{V}$ can be decomposed into eigenspaces of T. Equivalently, the eigenspaces together contain enough independent directions to form a basis of $\mathcal{V}$.

Theorem 1.2.61. *Every operator on a finite-dimensional, nonzero, real vector space has an invariant subspace of dimension* 1 *or* 2.

Proof. Let $\mathcal{V}$ be a real vector space with $\dim(\mathcal{V}) = n > 0$, and let $T \in \mathcal{L}(\mathcal{V})$. Choose $\mathbf{v} \in \mathcal{V}$ with $\mathbf{v} \neq \mathbf{0}$, then the list $(\mathbf{v}, T\mathbf{v}, T^2\mathbf{v}, \ldots, T^n\mathbf{v})$ contains $n+1$ vectors in a n-dimensional space, hence is linearly dependent. Therefore $\exists$ scalars $a_0, \ldots, a_n \in \mathbb{R}$, not all zero, such that $a_0\mathbf{v} + a_1 T\mathbf{v} + \cdots + a_n T^n\mathbf{v} = \mathbf{0}$.
Define a polynomial $p(x) = a_0 + a_1 x + \cdots + a_n x^n$. Then $p(T)\mathbf{v} = 0$.
Over $\mathbb{R}$, the polynomial p factors as $p(x) = c(x-\lambda_1)\cdots(x-\lambda_m)(x^2+\alpha_1 x+\beta_1)\cdots(x^2+\alpha_M x+\beta_M)$ where each quadratic factor has no real roots.
Thus, $0 = p(T)\mathbf{v} = c(T-\lambda_1 I)\cdots(T-\lambda_m I)(T^2+\alpha_1 T+\beta_1 I)\cdots(T^2+\alpha_M T+\beta_M I)\mathbf{v}$.
If all factors were injective, then their composition will be injective, implying $p(T)$ is injective, contradicting $p(T)\mathbf{v} = 0$ with $\mathbf{v} \neq \mathbf{0}$. Hence at least one factor is not injective.

i. Case 1: A linear factor is not injective.
Suppose $T - \lambda I$ is not injective. Then $\text{null}(T-\lambda I) \neq \{\mathbf{0}\}$, so there exists nonzero $\mathbf{u}$ such that $T\mathbf{u} = \lambda\mathbf{u}$. Thus $\text{span}(\mathbf{u})$ is a one-dimensional invariant subspace.

ii. Case 2: A quadratic factor is not injective.
Suppose for some j, $T^2 + \alpha T + \beta I$ is not injective. Then $\exists$ nonzero $\mathbf{u}$ such that $T^2\mathbf{u} + \alpha T\mathbf{u} + \beta\mathbf{u} = 0$.
Consider the subspace $\mathcal{U} = \text{span}(\mathbf{u}, T\mathbf{u})$. To show $\mathcal{U}$ is invariant under T. Let $\mathbf{w} \in \mathcal{U}$, so $\mathbf{w} = a\mathbf{u} + bT\mathbf{u}$. Then $T\mathbf{w} = aT\mathbf{u} + bT^2\mathbf{u}$. Using the relation $T^2\mathbf{u} = -\alpha T\mathbf{u} - \beta\mathbf{u}$, we get

$$T\mathbf{w} = aT\mathbf{u} + b(-\alpha T\mathbf{u} - \beta\mathbf{u}) = (-b\beta)\mathbf{u} + (a - b\alpha)T\mathbf{u} \ \in \mathcal{U}$$

Thus $\mathcal{U}$ is invariant under T, and $\dim(\mathcal{U}) \leq 2$. Since $\mathbf{u} \neq \mathbf{0}$, we have $\dim(\mathcal{U}) \geq 1$, so dim = 1 or 2.

In both cases, there exists an invariant subspace of dimension 1 or 2 □

Over $\mathbb{R}$, operators may lack real eigenvalues, but every polynomial splits into linear and quadratic factors; failure of injectivity in one factor produces either a 1D eigenspace or a 2D invariant subspace.

Definition 1.2.62. Let $\mathcal{U}$, $\mathcal{W}$ be subspaces of $\mathcal{V}$, where $\mathcal{V} = \mathcal{U} \oplus \mathcal{W}$.
Let $\mathbf{u} \in \mathcal{U}, \mathbf{w} \in \mathcal{W}, \mathbf{v} \in \mathcal{V}$, and $\mathbf{v} = \mathbf{u} + \mathbf{w}$. Define *projection* $P_{\mathcal{U},\mathcal{W}} \in \mathcal{L}(\mathcal{V})$ by $P_{\mathcal{U},\mathcal{W}}\mathbf{v} = \mathbf{u}$. Note that

$$P_{\mathcal{W},\mathcal{U}}\mathbf{v} = \mathbf{w}, \quad \mathbf{v} = P_{\mathcal{U},\mathcal{W}}\mathbf{v} + P_{\mathcal{W},\mathcal{U}}\mathbf{v}$$
$$\text{range}(P_{\mathcal{U},\mathcal{W}}) = \mathcal{U}, \quad \text{null}(P_{\mathcal{U},\mathcal{W}}) = \mathcal{W}$$

Theorem 1.2.63. *Every operator on an odd-dimensional real vector space has an eigenvalue.*

Proof. Proceed by induction on $\dim(\mathcal{V})$. If $\dim(\mathcal{V}) = 1$, the result is trivial.
Now suppose $\dim(\mathcal{V}) = n > 1$ is odd, and assume result holds for all smaller dimensions.
Let $T \in \mathcal{L}(\mathcal{V})$. By Theorem 1.2.61, there exists a nonzero invariant subspace $\mathcal{U} \subseteq \mathcal{V}$ with $\dim(\mathcal{U}) = 1$ or 2.
If $\dim(\mathcal{U}) = 1$, then $\mathcal{U} = \text{span}(\mathbf{u})$ for some nonzero $\mathbf{u}$, and invariance gives $T\mathbf{u} \in \mathcal{U} \Rightarrow T\mathbf{u} = \lambda\mathbf{u}$ for some $\lambda \in \mathbb{R}$.
Hence $\mathbf{u}$ is an eigenvector, and we are done.
Now suppose $\dim(\mathcal{V}) = 2$. Choose complementary subspace $\mathcal{W}$ s.t. $\mathcal{V} = \mathcal{U} \oplus \mathcal{W}$. Then $\dim(\mathcal{W}) = n-2$, which is

still odd. Although $\mathcal{W}$ need not be invariant under T, define linear operator $S : \mathcal{W} \to \mathcal{W}$ by $S(\mathbf{w}) = P_{\mathcal{W}}(T\mathbf{w})$, where $P_{\mathcal{W}}$ is the projection onto $\mathcal{W}$ along $\mathcal{U}$. Then $S \in \mathcal{L}(\mathcal{W})$. By induction hypothesis (since $\dim(\mathcal{W})$ is odd), S has an eigenvalue λ with eigenvector $\mathbf{w} \neq \mathbf{0}$, $S\mathbf{w} = \lambda\mathbf{w}$. Thus $T\mathbf{w} = P_{\mathcal{U}}(T\mathbf{w}) + P_{\mathcal{W}}(T\mathbf{w}) = P_{\mathcal{U}}(T\mathbf{w}) + \lambda\mathbf{w}$.
Now consider the subspace $\mathcal{U} + \text{span}(\mathbf{w})$. For any vector $\mathbf{u} + a\mathbf{w}$, $(T - \lambda I)(\mathbf{u} + a\mathbf{w}) = T\mathbf{u} - \lambda\mathbf{u} + a(T\mathbf{w} - \lambda\mathbf{w})$.
Using decomposition above, $T\mathbf{w} - \lambda\mathbf{w} = P_{\mathcal{U}}(T\mathbf{w})$, so $(T - \lambda I)(\mathbf{u} + a\mathbf{w}) = (T\mathbf{u} - \lambda\mathbf{u}) + aP_{\mathcal{U}}(T\mathbf{w})$.
Since $\mathcal{U}$ is invariant, $T\mathbf{u} \in \mathcal{U}$, and clearly $\lambda\mathbf{u} \in \mathcal{U}$, and $P_{\mathcal{U}}(T\mathbf{w}) \in \mathcal{U}$, thus $(T - \lambda I)(\mathbf{u} + a\mathbf{w}) \in \mathcal{U}$.
Thus $(T - \lambda I)(\mathcal{U} + \text{span}(\mathbf{w})) \subseteq \mathcal{U}$. But $\dim(\mathcal{U} + \text{span}(\mathbf{w}) > \dim(\mathcal{U})$, so $T - \lambda I$ cannot be injective on this subspace. Hence $\exists$ a nonzero vector $\mathbf{v} \in \mathcal{U} + \text{span}(\mathbf{w})$ such that $(T - \lambda I)\mathbf{v} = \mathbf{0}$.
Therefore $\mathbf{v}$ is an eigenvector of T, and T has an eigenvalue. □

Odd dimension prevents full decomposition into 2D invariant blocks. The extra dimension forces a real eigenvalue to exist.

1.2.3 Inner-Product Spaces

Definition 1.2.64. The *norm* is defined as $\|\mathbf{z}\| = \sqrt{|z_1|^2 + \cdots + |z_n|^2} = \sqrt{\langle \mathbf{z}, \mathbf{z} \rangle}$ for $\mathbf{z} = (z_1, \ldots, z_n) \in \mathbb{C}^n$.
The norm should be think of as the *inner product* of $\mathbf{z}$ with itself, $\|z\|^2 = z_1\overline{z_1} + \cdots + z_n\overline{z_n}$.
If $\mathbf{p} \in \mathcal{P}_m(\mathbb{F})$ (polynomial space), then $\|\mathbf{p}\| = \sqrt{\int_0^1 |\mathbf{p}(x)|^2 \, dx}$

Remark 1.2.65. An inner product on $\mathcal{V}$ that is a function that takes each ordered pair $(\mathbf{u}, \mathbf{v})$ of elements in $\mathcal{V}$ to a number $\langle \mathbf{u}, \mathbf{v} \rangle \in \mathbb{F}$ has following properties:

i. Positivity: $\langle \mathbf{v}, \mathbf{v} \rangle \geq 0 \; \forall \; \mathbf{v} \in \mathcal{V}$
ii. Definiteness: $\langle \mathbf{v}, \mathbf{v} \rangle = 0$ if and only if $\mathbf{v} = \mathbf{0}$
iii. Additivity in First Slot: $\langle \mathbf{u} + \mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{w} \rangle + \langle \mathbf{v}, \mathbf{w} \rangle$ for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathcal{V}$
iv. Homogeneity in First Slot: $\langle a\mathbf{v}, \mathbf{w} \rangle = a \langle \mathbf{v}, \mathbf{w} \rangle$ for all $a \in \mathbb{F}$ and $\mathbf{v}, \mathbf{w} \in \mathcal{V}$
v. Conjugate Symmetry: $\langle \mathbf{v}, \mathbf{w} \rangle = \overline{\langle \mathbf{w}, \mathbf{v} \rangle}$ for all $\mathbf{v}, \mathbf{w} \in \mathcal{V}$

Definition 1.2.66. An *inner-product space* is a vector space $\mathcal{V}$ with an inner product on $\mathcal{V}$.

Definition 1.2.67. Two vectors $\mathbf{u}$, $\mathbf{v}$ are *orthogonal* if $\langle \mathbf{u}, \mathbf{v} \rangle = 0$.

Theorem 1.2.68. *Pythagorean Theorem*
If $\mathbf{u}, \mathbf{v}$ are orthogonal vectors in $\mathcal{V}$, then $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathcal{V}$ be orthogonal, i.e., $\langle \mathbf{u}, \mathbf{v} \rangle = 0$. Then

$$\begin{aligned} \|\mathbf{u} + \mathbf{v}\|^2 &= \langle \mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{v} \rangle \\ &= \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + \langle \mathbf{u}, \mathbf{v} \rangle + \langle \mathbf{v}, \mathbf{u} \rangle \quad \text{by linearity in first slot and conjugate symmetry} \\ &= \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 \end{aligned}$$

□

Orthogonality eliminates the cross terms in the inner product expansion, so the squared norm behaves additively, exactly like perpendicular vectors in Euclidean geometry.

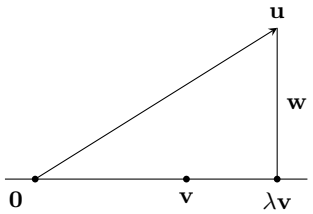


Figure 1: Orthogonal decomposition of $\mathbf{u}$ as a scalar multiple of $\mathbf{v}$ plus vector $\mathbf{w}$ orthogonal to $\mathbf{v}$

Definition 1.2.69. To compute the *orthogonal decomposition* of vector $\mathbf{u}$, let $a \in \mathbb{F}$, then

$$\mathbf{u} = a\mathbf{v} + (\mathbf{u} - a\mathbf{v})$$

To choose a so that $\mathbf{v}$ is orthogonal to $(\mathbf{u} - a\mathbf{v})$, which is

$$0 = \langle \mathbf{u} - a\mathbf{v}, \mathbf{v} \rangle = \langle \mathbf{u}, \mathbf{v} \rangle - a \|\mathbf{v}\|^2$$

Need to choose a to be $\langle \mathbf{u}, \mathbf{v} \rangle / \|\mathbf{v}\|^2$, where $\mathbf{v} \neq \mathbf{0}$, then

$$\mathbf{u} = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{v}\|^2}\mathbf{v} + \left(\mathbf{u} - \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2}\mathbf{v} \right)$$

Theorem 1.2.70. *Cauchy-Schwarz Inequality*
If $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, then $|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\| \, \|\mathbf{u}\|$.
This inequality is an equality if and only if one of $\mathbf{u}, \mathbf{v}$ is a scalar multiple of the other.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathcal{V}$. If $\mathbf{v} = \mathbf{0}$, the inequality is trivial. Assume $\mathbf{u} \neq \mathbf{0}$. Define

$$\mathbf{u} := \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{v}\|^2}\mathbf{v} + \mathbf{w}$$

where $\mathbf{w} \perp \mathbf{v}$. By Pythagorean Theorem 1.2.68,

$$\|\mathbf{u}\|^2 = \left\| \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{v}\|^2}\mathbf{v} \right\|^2 + \|\mathbf{w}\|^2 = \frac{|\langle \mathbf{u}, \mathbf{v} \rangle|^2}{\|\mathbf{v}\|^2} + \|\mathbf{w}\|^2 \geq \frac{|\langle \mathbf{u}, \mathbf{v} \rangle|^2}{\|\mathbf{v}\|^2}$$

Multiplying both sides by $\|\mathbf{v}\|^2$ then taking square roots give the desired inequality.
Equality holds if and only if $\mathbf{w} = \mathbf{0}$, which is equivalent to $\mathbf{u}$ being a scalar multiple of $\mathbf{v}$. □

Project $\mathbf{u}$ onto the line spanned by $\mathbf{v}$. The inner product $\langle \mathbf{u}, \mathbf{v} \rangle$ only sees this projected component, while $\|\mathbf{u}\|$ may also include an orthogonal component, making the inequality inevitable.

Theorem 1.2.71. *Triangle Inequality*
If $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, then $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$.
This inequality is an equality if and only if one of $\mathbf{u}, \mathbf{v}$ is a nonnegative multiple of the other.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, then

$$\|\mathbf{u} + \mathbf{v}\|^2 = \langle \mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{v} \rangle = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + \langle \mathbf{u}, \mathbf{v} \rangle + \langle \mathbf{v}, \mathbf{u} \rangle = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + 2\mathrm{Re}\, \langle \mathbf{v}, \mathbf{u} \rangle$$

Since $\mathrm{Re} z \leq |z|$ for all $z \in \mathbb{C}$, $\|(\|\, \mathbf{u} + \mathbf{v})^2 \leq \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + 2\, |\langle \mathbf{u}, \mathbf{v} \rangle|$.
By Cauchy-Swards Inequality (Theorem 1.2.70), $\langle \mathbf{u}, \mathbf{v} \rangle \leq \|\mathbf{u}\| \, \|\mathbf{v}\|$, hence

$$\|\mathbf{u} + \mathbf{v}\|^2 \leq \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + 2\, \|\mathbf{u}\| \, \|\mathbf{v}\| = (\|\mathbf{u}\| + \|\mathbf{v}\|)^2$$

Taking square roots on both sides gives the triangle inequality.
Note this is an equality if and only if $\langle \mathbf{u}, \mathbf{v} \rangle = |\langle \mathbf{u}, \mathbf{v} \rangle|$, which together imply $\mathbf{u} = \lambda \mathbf{v}$ with $\lambda \geq 0$. □

Expand the squared norm and control the cross term using Cauchy–Schwarz; the inequality arises because the inner product cannot exceed the product of norms.

Theorem 1.2.72. *Parallelogram Inequality*
If $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, then $\|\mathbf{u} + \mathbf{v}\|^2 + \|\mathbf{u} - \mathbf{v}\|^2 = 2(\|\mathbf{u}\|^2 + \|\mathbf{v}\|^2)$.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, then

$$\begin{aligned} \|\mathbf{u} + \mathbf{v}\|^2 + \|\mathbf{u} - \mathbf{v}\|^2 &= \langle \mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{v} \rangle + \langle \mathbf{u} - \mathbf{v}, \mathbf{u} - \mathbf{v} \rangle \\ &= 2\, \|\mathbf{u}\|^2 + 2\, \|\mathbf{v}\|^2 + \langle \mathbf{u}, \mathbf{v} \rangle + \langle \mathbf{v}, \mathbf{u} \rangle - \langle \mathbf{u}, \mathbf{v} \rangle - \langle \mathbf{v}, \mathbf{u} \rangle \\ &= 2(\|\mathbf{u}\|^2 + \|\mathbf{v}\|^2) \end{aligned}$$

□

The cross terms cancel when adding the expansions of $\|\mathbf{u} + \mathbf{v}\|^2$ and $\|\mathbf{u} - \mathbf{v}\|^2$, leaving only the pure norm terms. This reflects a fundamental symmetry of inner-product geometry.

Definition 1.2.73. A list of vectors $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ in $\mathcal{V}$ is *orthonormal* if

$$\langle \mathbf{e}_j, \mathbf{e}_k \rangle = \begin{cases} \langle \mathbf{e}_j, \mathbf{e}_k \rangle = 0 & \text{if } j \neq k \\ \langle \mathbf{e}_j, \mathbf{e}_k \rangle = 1 & \text{if } j = k \end{cases}$$

Proposition 1.2.74. *If $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ is an orthonormal list of vectors in $\mathcal{V}$, then*

$$\|a_1\mathbf{e}_1 + \cdots + a_m\mathbf{e}_m\|^2 = |a_1|^2 + \cdots + |a_m|^2$$

for all $a_1, \ldots, a_m \in \mathbb{F}$.

Proof. Because $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ is orthonormal, $\langle \mathbf{e}_i, \mathbf{e}_j \rangle = 1$ for $i = j$ and 0 for $i \neq j$. Thus the vectors $a_1\mathbf{e}_1, \ldots, a_m\mathbf{e}_m$ are pairwise orthogonal. By repeated application of Pythagorean Theorem 1.2.68,

$$\left\| \sum_{j=1}^{m} a_j \mathbf{e}_j \right\|^2 = \sum_{j=1}^{m} \|a_j \mathbf{e}_j\|^2$$

Since $\|a_j\mathbf{e}_j\| = |a_j| \, \|\mathbf{e}_j\| = |a_j|$, we obtain $\|a_1\mathbf{e}_1 + \cdots + a_m\mathbf{e}_m\|^2 = |a_1|^2 + \cdots + |a_m|^2$. □

Orthonormality eliminates all cross terms. Each component lies in a mutually orthogonal direction, so the squared norm is just the sum of squared coefficients.

Corollary 1.2.75. *Every orthonormal list of vectors is linearly independent.*

Proof. Suppose $a_1\mathbf{e}_1 + \cdots + a_m\mathbf{e}_m = \mathbf{0}$. Taking norms and using Proposition 1.2.74,

$$0 = \|0\|^2 = \left\| \sum_{j=1}^{m} a_j \mathbf{e}_j \right\| = \sum_{j=1}^{m} |a_j|^2$$

Hence $|a_j|^2 = 0$ for all j, so $a_j = 0$ for all j. Thus the list is linearly independent. □

In an orthonormal system, the squared norm of a linear combination is the sum of squared coefficients, so the only way to get zero is for all coefficients to vanish.

Definition 1.2.76. An *orthonormal basis* of $\mathcal{V}$ is an orthonormal list of vectors in $\mathcal{V}$ that is also a basis.

Theorem 1.2.77. *Suppose $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ is an orthonormal basis of $\mathcal{V}$, then*

$$\mathbf{v} = \langle \mathbf{v}, \mathbf{e}_1 \rangle \, \mathbf{e}_1 + \cdots + \langle \mathbf{v}, \mathbf{e}_n \rangle \, \mathbf{e}_n$$
$$\|\mathbf{v}\|^2 = |\langle \mathbf{v}, \mathbf{e}_1 \rangle|^2 + \cdots + |\langle \mathbf{v}, \mathbf{e}_n \rangle|^2$$

for every $\mathbf{e} \in \mathcal{V}$.

Proof. Let $\mathbf{v} \in \mathcal{V}$. Since $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ is a basis, write $\mathbf{v} = \sum_{j=1}^{n} a_j \mathbf{e}_j$. Taking inner product with $\mathbf{e}_k$,

$$\langle \mathbf{v}, \mathbf{e}_j \rangle = \sum_{j=1}^{n} a_j \langle \mathbf{e}_j, \mathbf{e}_k \rangle = a_k$$

using orthonormality. Hence $\mathbf{v} = \sum_{j=1}^{n} \langle \mathbf{v}, \mathbf{e}_j \rangle \, \mathbf{e}_j$. Applying Proposition 1.2.74, $\|v\|^2 = \sum_{j=1}^{n} \left| \langle \mathbf{v}, \mathbf{e}_j \rangle \right|^2$ □

An orthonormal basis isolates coordinates via inner products. Orthogonality eliminates cross terms, turning geometry into coefficient-wise algebra.

Theorem 1.2.78. *If $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is a linearly independent list of vectors in $\mathcal{V}$, then there exists an orthonormal list $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ of vectors in $\mathcal{V}$ such that*

$$\mathit{span}(\mathbf{v}_1, \ldots, \mathbf{v}_j) = \mathit{span}(\mathbf{e}_1, \ldots, \mathbf{e}_j)$$

for $j = 1, \ldots, m$.

Proof. Construct $\mathbf{e}_1, \ldots, \mathbf{e}_m$ inductively. Set $\mathbf{e}_1 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|}$.
Assume $\mathbf{e}_1, \ldots, \mathbf{e}_{j-1}$ are orthonormal and satisfy $\operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{j-1}) = \operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_{j-1})$. Define

$$\mathbf{u}_j = \mathbf{v}_j - \sum_{k=1}^{j-1} \langle \mathbf{v}_j, \mathbf{e}_k \rangle \, \mathbf{e}_k, \quad \mathbf{e}_j = \frac{\mathbf{u}_j}{\|\mathbf{u}_j\|}$$

Since $\mathbf{v}_j \notin \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{j-1}) = \operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_{j-1})$, we have $\mathbf{u}_j \neq \mathbf{0}$, so $\mathbf{e}_j$ is well-defined.
For $k < j$,

$$\langle \mathbf{u}_j, \mathbf{e}_k \rangle = \langle \mathbf{v}_j, \mathbf{e}_k \rangle - \sum_{k=1}^{j-1} \langle \mathbf{v}_j, \mathbf{e}_i \rangle \, \langle \mathbf{e}_i, \mathbf{e}_k \rangle = \langle \mathbf{v}_j, \mathbf{e}_k \rangle - \langle \mathbf{v}_j, \mathbf{e}_k \rangle = \mathbf{0}$$

so $\mathbf{e}_j \perp \mathbf{e}_k$. Also $\|\mathbf{e}_j\| = 1$, hence $(\mathbf{e}_1, \ldots, \mathbf{e}_j)$ is orthonormal. By construction, $\mathbf{u}_j \in \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_j)$, hence $\mathbf{e}_j \in \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_j)$, so $\operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_j) \subset \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_j)$. The reverse inclusion follows since

$$\mathbf{v}_j = \mathbf{u}_j + \sum_{k=1}^{j-1} \langle \mathbf{v}_j, \mathbf{e}_k \rangle \, \mathbf{e}_k \in \operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_j)$$

and the induction hypothesis gives the rest. Thus the spans are equal. □

At each step, subtract the projection of $\mathbf{v}_j$ onto the previously constructed subspace, leaving a component orthogonal to it. Normalising yields an orthonormal basis without changing the span.

Corollary 1.2.79. *Every finite-dimensional inner-product space has an orthonormal basis.*

Proof. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ be a basis of $\mathcal{V}$. Apply Gram-Schmidt (Theorem 1.2.78), to obtain an orthonormal list $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ s.t. $\operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_m) = \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_m) = \mathcal{V}$. Thus $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ is an orthonormal basis of $\mathcal{V}$ □

Start from any basis and orthonormalise it without changing its span. This converts an arbitrary basis into an orthonormal one.

Corollary 1.2.80. *Every orthonormal list of vectors in $\mathcal{V}$ can be extended to an orthonormal basis of $\mathcal{V}$.*

Proof. Let $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ be an orthonormal list in $\mathcal{V}$. Extend it to a basis $(\mathbf{e}_1, \ldots, \mathbf{e}_m, \mathbf{v}_{m+1}, \ldots, \mathbf{v}_n)$ of $\mathcal{V}$.
Apply Gram-Schmidt (Theorem 1.2.78) to this list to obtain an orthonormal list $(\mathbf{e}_1, \ldots, \mathbf{e}_m, \mathbf{f}_{m+1}, \ldots, \mathbf{f}_n)$.
Because $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ is already orthonormal, Gram-Schmidt leaves these vectors unchanged.
Hence $(\mathbf{e}_1, \ldots, \mathbf{e}_m, \mathbf{f}_{m+1}, \ldots, \mathbf{f}_n)$ is an orthonormal basis of $\mathcal{V}$, extending the original list. □

Extend the orthonormal list to a basis, then orthonormalise. The existing orthonormal part is unaffected, so the extension remains intact.

Corollary 1.2.81. *Suppose $T \in \mathcal{L}(\mathcal{V})$. If T has an upper-triangular matrix with respect to some basis of $\mathcal{V}$, then T has an upper-triangular matrix with respect to some orthonormal basis of $\mathcal{V}$.*

Proof. Let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be a basis with respect to which T is upper triangular.
Then for each j, $T(\operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_n)) \subset \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_n)$. Apply Gram-Schmidt (Theorem 1.2.78) to obtain an orthonormal basis $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ such that $\operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_n) = \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ for all j. Hence each $\operatorname{span}(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ is T-invariant. By Proposition 1.2.54, T has an upper-triangular matrix w.r.t. $\mathbf{e}_1, \ldots, \mathbf{e}_n$. □

Upper-triangular structure is equivalent to preservation of a nested chain of subspaces (a flag). Gram–Schmidt preserves this flag while making the basis orthonormal.

Corollary 1.2.82. *Suppose $\mathcal{V}$ is a complex vector space and $T \in \mathcal{L}(\mathcal{V})$, then T has an upper-triangular matrix with respect to some orthonormal basis of $\mathcal{V}$.*

Proof. By Theorem 1.2.55, T has an upper-triangular matrix with respect to some basis $\mathcal{V}$.
By Corollary 1.2.81, there exists an orthonormal basis of $\mathcal{V}$ with respect to which T is also upper-triangular. □

Over $\mathbb{C}$, every operator admits a triangular form. Gram–Schmidt then converts the underlying basis into an orthonormal one without destroying that structure.

Definition 1.2.83. If $\mathcal{U}$ is a subset of $\mathcal{V}$, then the *orthogonal complement* of $\mathcal{U}$, denoted $\mathcal{U}^\perp$, is the set of all vectors in $\mathcal{V}$ that are orthogonal to every vector in $\mathcal{U}$:

$$\mathcal{U}^\perp = \{\mathbf{v} \in \mathcal{V} : \langle \mathbf{v}, \mathbf{u} \rangle = 0 \ \ \forall\, \mathbf{u} \in \mathcal{U}\}$$

Note that $\mathcal{U}^\perp \subset \mathcal{V}$, $\mathcal{V}^\perp = \{\mathbf{0}\}$, $\{\mathbf{0}\}^\perp = \mathcal{V}$. Also note that if $\mathcal{U}_1 \subset \mathcal{U}_2$, then $\mathcal{U}_1^\perp \subset \mathcal{U}_2^\perp$.

Theorem 1.2.84. *If $\mathcal{U}$ is a subspace of $\mathcal{V}$, then $\mathcal{V} = \mathcal{U} \oplus \mathcal{U}^\perp$.*

Proof. Let $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ be an orthonormal basis of $\mathcal{U}$. For every $\mathbf{v} \in \mathcal{V}$, define $\mathbf{u} = \sum_{j=1}^{m} \langle \mathbf{v}, \mathbf{e}_j \rangle \, \mathbf{e}_j$, $\mathbf{w} = \mathbf{v} - \mathbf{u}$. Then $\mathbf{u} \in \mathcal{U}$. For each j, $\langle \mathbf{w}, \mathbf{e}_j \rangle = \langle \mathbf{v}, \mathbf{e}_j \rangle - \langle \mathbf{v}, \mathbf{e}_j \rangle = \mathbf{0}$, so $\mathbf{w} \perp \mathcal{U}$, i.e., $\mathbf{w} \in \mathcal{U}^\perp$. Thus $\mathbf{v} = \mathbf{u} + \mathbf{w}$, proving $\mathcal{V} = \mathcal{U} + \mathcal{U}^\perp$. If $\mathbf{v} \in \mathcal{U} \cap \mathcal{U}^\perp$, then $\langle \mathbf{v}, \mathbf{v} \rangle = \mathbf{0}$, hence $\mathbf{v} = \mathbf{0}$. Thus $\mathcal{U} \cap \mathcal{U}^\perp = \{\mathbf{0}\}$. Therefore $\mathcal{V} = \mathcal{U} \oplus \mathcal{U}^\perp$. □

Decompose any vector into its projection onto $\mathcal{U}$ and the residual orthogonal component. Orthogonality forces uniqueness, giving a direct sum decomposition.

Corollary 1.2.85. *If $\mathcal{U} \subset \mathcal{V}$, then $\mathcal{U} = (\mathcal{U}^\perp)^\perp$.*

Proof. First, if $\mathbf{u} \in \mathcal{U}$, then for every $\mathbf{v} \in \mathcal{U}^\perp$, $\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{0}$, so $\mathbf{u} \in (\mathcal{U}^\perp)^\perp$. Hence $\mathcal{U} \subset (\mathcal{U}^\perp)^\perp$.
Conversely, let $\mathbf{v} \in (\mathcal{U}^\perp)^\perp$. By Theorem 1.2.84, write $\mathbf{v} = \mathbf{u} + \mathbf{w}$, $\mathbf{u} \in \mathcal{U}$, $\mathbf{w} \in \mathcal{U}^\perp$. Then $\mathbf{w} = \mathbf{v} - \mathbf{u} \in (\mathcal{U}^\perp)^\perp$, so $\mathbf{w} \in \mathcal{U}^\perp \cap (\mathcal{U}^\perp)^\perp$. Thus $\langle \mathbf{w}, \mathbf{w} \rangle = \mathbf{0} \Rightarrow \mathbf{w} = \mathbf{0}$, hence $\mathbf{v} = \mathbf{u} \in \mathcal{U}$. Therefore $(\mathcal{U}^\perp)^\perp \subset \mathcal{U}$. Thus $\mathcal{U} = (\mathcal{U}^\perp)^\perp$. □

The orthogonal decomposition $\mathcal{V} = \mathcal{U} \oplus \mathcal{U}^\perp$ is unique. A vector orthogonal to $\mathcal{U}^\perp$ cannot have any component in $\mathcal{U}^\perp$, so it must lie entirely in $\mathcal{U}$.

Definition 1.2.86. Let $\mathcal{U} \subset \mathcal{V}$. Decomposition $\mathcal{V} = \mathcal{U} \oplus \mathcal{U}^\perp$ means each vector $\mathbf{v} \in \mathbf{V}$ can be written as $\mathbf{v} = \mathbf{u} + \mathbf{w}$, where $\mathbf{u} \in \mathcal{U}$ and $\mathbf{w} \in \mathcal{U}^\perp$.
The *orthogonal projection* of $\mathcal{V}$ onto $\mathcal{U}$ is an operator on $\mathcal{V}$, denoted $P_\mathcal{U} = P_{\mathcal{U},\mathcal{U}^\perp}$.
Note that for $\mathbf{v} \in \mathcal{V}$, denote $P_\mathcal{U}\mathbf{v}$ to be vector $\mathbf{u}$ in decomposition $\mathbf{v} = \mathbf{u} + \mathbf{w}$.

Remark 1.2.87. *Properties of Orthogonal Projection* of $P_\mathcal{U} \in \mathcal{L}(\mathcal{V})$:

i. range$P_\mathcal{U} = \mathcal{U}$
ii. null$P_\mathcal{U} = \mathcal{U}^\perp$
iii. $\mathbf{v} - P_\mathcal{U}\mathbf{v} \in \mathcal{U}^\perp$ for every $\mathbf{v} \in \mathcal{V}$
iv. $P_\mathcal{U}^2 = P_\mathcal{U}$
v. $\|P_\mathcal{U}\mathbf{v}\| \leq \|\mathbf{v}\|$ for every $\mathbf{v} \in \mathcal{V}$
vi. If $(\mathbf{e}_1, \ldots, \mathbf{e}_m)$ is an orthonormal basis of $\mathcal{U}$, then $P_\mathcal{U}\mathbf{v} = \langle \mathbf{v}, \mathbf{e}_1 \rangle \, \mathbf{e}_1 + \cdots + \langle \mathbf{v}, \mathbf{e}_m \rangle \, \mathbf{v}_m$ for every $\mathbf{v} \in \mathcal{V}$

Proposition 1.2.88. *Suppose $\mathcal{U} \subset \mathcal{V}$ and $\mathbf{v} \in \mathcal{V}$. Then*

$$\|\mathbf{v} - P_\mathcal{U}\mathbf{v}\| \leq \|\mathbf{v} - \mathbf{u}\|$$

for every $\mathbf{v} \in \mathcal{U}$. Furthermore, if $\mathbf{v} \in \mathcal{U}$ and the inequality above is an equality, then $\mathbf{v} = P_\mathcal{U}\mathbf{v}$.

Proof. Let $\mathbf{u} \in \mathcal{U}$. Then $\mathbf{v} - \mathbf{u} = (\mathbf{v} - P_\mathcal{U}\mathbf{v}) + (P_\mathcal{U}\mathbf{v} - \mathbf{u})$, where $\mathbf{v} - P_\mathcal{U}\mathbf{v} \in \mathcal{U}^\perp$ and $P_\mathcal{U}\mathbf{v} - \mathbf{u} \in \mathcal{U}$. Hence these two vectors are orthogonal. By Pythagorean Theorem 1.2.68,

$$\|\mathbf{v} - \mathbf{u}\|^2 = \|\mathbf{v} - P_\mathcal{U}\mathbf{v}\|^2 + \|P_\mathcal{U}\mathbf{v} - \mathbf{u}\|^2 \geq \|\mathbf{v} - P_\mathcal{U}\mathbf{v}\|^2$$

Taking square roots, $\|\mathbf{v} - P_\mathcal{U}\mathbf{v}\| \leq \|\mathbf{v} - \mathbf{u}\|$.
If equality holds for some $\mathbf{u} \in \mathcal{U}$, then $\|P_\mathcal{U}\mathbf{v} - \mathbf{u}\| = \mathbf{0}$, hence $\mathbf{u} = P_\mathcal{U}\mathbf{v}$. □

Decompose $\mathbf{v} - \mathbf{u}$ into orthogonal components: one fixed $(\mathbf{v} - P_\mathcal{U}\mathbf{v})$ and one variable inside $\mathcal{U}$. The projection minimises distance because any deviation adds an orthogonal error term.

Example 1.2.89. Approximate $\sin(x)$ on $[-\pi, \pi]$ with basis $(1, x, x^2, x^3, x^4, x^5)$ of $\mathcal{U}$.

Proof. Let $\langle f, g \rangle = \int_{-\pi}^{\pi} f(x)g(x)\,dx$. Since $\sin x$ is odd, it suffices to project onto $U_{\text{odd}} = \text{span}\{x, x^3, x^5\}$.
Define orthogonal polynomials $p_1 = x, p_2 = x^3 - \frac{3\pi^2}{5}x, p_3 = x^5 - \frac{10\pi^2}{9}x^3 + \frac{5\pi^4}{21}x$.
The norms are then $\|p_1\|^2 = \frac{2\pi^3}{3}, \|p_2\|^2 = \frac{8\pi^7}{175}, \|p_3\|^2 = \frac{128\pi^{11}}{43659}$.
Using symmetry and integration, $\langle \sin x, p_1 \rangle = 2\pi, \langle \sin x, p_2 \rangle = \frac{4\pi}{5}(\pi^2 - 15), \langle \sin x, p_3 \rangle = \frac{16\pi}{63}(\pi^4 - 105\pi^2 + 945)$.
The best approximation is the projection $u = \sum_{k=1}^{3} \frac{\langle \sin x, p_k \rangle}{\|p_k\|^2} p_k = Ax + Bx^3 + Cx^5$,
where $A = \frac{105(\pi^4 - 153\pi^2 + 1485)}{8\pi^6}, B = \frac{315(125\pi^2 - \pi^4 - 1155)}{4\pi^8}, C = \frac{693(\pi^4 - 105\pi^2 + 945)}{8\pi^{10}}$ □

Project $\sin x$ onto the odd polynomial subspace using an orthogonal basis. Orthogonality diagonalises the least-squares problem, reducing it to independent scalar projections.

Definition 1.2.90. A *linear functional* on $\mathcal{V}$ is a linear map from $\mathcal{V}$ to the scalars $\mathbb{F}$.

Theorem 1.2.91. *Suppose φ is a linear functional on $\mathcal{V}$. Then there is a unique vector $\mathbf{v} \in \mathcal{V}$ such that*

$$\varphi(\mathbf{u}) = \langle \mathbf{u}, \mathbf{v} \rangle$$

for every $\mathbf{u} \in \mathcal{V}$.

Proof. Let $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ be an orthonormal basis of $\mathcal{V}$. For any $\mathbf{u} \in \mathcal{V}$, $\mathbf{u} = \sum_{j=1}^{n} \langle \mathbf{u}, \mathbf{e}_j \rangle \, \mathbf{e}_j$. Applying φ,

$$\varphi(\mathbf{u}) = \sum_{j=1}^{n} \langle \mathbf{u}, \mathbf{e}_j \rangle \, \varphi(\mathbf{e}_j) = \left\langle \mathbf{u}, \sum_{j=1}^{n} \overline{\varphi(\mathbf{e}_j)} \mathbf{e}_j \right\rangle$$

Set $\mathbf{v} = \sum_{j=1}^{n} \overline{\varphi(\mathbf{e}_j)} \mathbf{e}_j$, then $\varphi(\mathbf{u}) = \langle \mathbf{u}, \mathbf{v} \rangle$ for all $\mathbf{u} \in \mathcal{V}$.
For uniqueness, if $\langle \mathbf{u}, \mathbf{v}_1 \rangle = \langle \mathbf{u}, \mathbf{v}_2 \rangle$ for all $\mathbf{u}$, then $\langle \mathbf{u}, \mathbf{v}_1 - \mathbf{v}_2 \rangle = \mathbf{0}$ for all $\mathbf{u}$.
Taking $\mathbf{u} = \mathbf{v}_1 - \mathbf{v}_2$ gives $\mathbf{v}_1 - \mathbf{v}_2 = \mathbf{0}$, hence $\mathbf{v}_1 = \mathbf{v}_2$. □

An orthonormal basis converts any linear functional into coordinates. The inner product reconstructs it by pairing those coordinates with basis vectors, yielding a representing vector.

Definition 1.2.92. Let $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, where $\mathcal{W}, \mathcal{V}$ are finite-dimensional nonzero, inner-product space over $\mathbb{F}$. Fix $\mathbf{w} \in \mathcal{W}$, and consider the linear functional $f_{\mathbf{w}} : \mathcal{V} \to \mathbb{F}$ given by $f_{\mathbf{w}}(\mathbf{v}) = \langle T\mathbf{v}, \mathbf{w} \rangle$. There is a unique vector $\mathbf{y} \in \mathcal{V}$ such that $f_{\mathbf{w}}(\mathbf{v}) = \langle \mathbf{v}, \mathbf{y} \rangle \ \forall \ \mathbf{v} \in \mathcal{V}$, and let this unique vector $\mathbf{y}$ be $T^*\mathbf{w}$. Hence

$$\langle T\mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{v}, T^*\mathbf{w} \rangle$$

for all $\mathbf{v} \in \mathcal{V}$. Doing this for every $\mathbf{w} \in \mathcal{W}$ gives a map $T^* : \mathcal{W} \to \mathcal{V}$, which is the *adjoint* of T.

Remark 1.2.93. *Property of Adjoints*

i. Additivity: $(S+T)^* = S^* + T^*$ for all $S, T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$
ii. Conjugate Homogeneity: $(aT)^* = \overline{a}T^*$ for all $a \in \mathbb{F}$ and $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$
iii. Adjoint of Adjoint: $(T^*)^* = T$ for all $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$
iv. Identity: $I^* = I$, where I is the identity operator on $\mathcal{V}$
v. Products: $(ST^*) = T^*S^*$ for all $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$ and $S \in \mathcal{L}(\mathcal{W}, \mathcal{U})$ ($\mathcal{U}$ is inner-product space over $\mathbb{F}$)

Proposition 1.2.94. *Suppose $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, then*

i. $null(T^*) = (range(T))^\perp$
ii. $range(T^*) = (null())T)^\perp$
iii. $null(T) = (range(T^*))^\perp$
iv. $range(T) = (null(T^*))^\perp$

Proof.

(*i*) For $\mathbf{w} \in \mathcal{W}$,

$$\mathbf{w} \in \text{null}(T^*) \Leftrightarrow T^*\mathbf{w} = \mathbf{0} \Leftrightarrow \langle \mathbf{v}, T^*\mathbf{w} \rangle = \mathbf{0} \ \forall \ \mathbf{v} \in \mathcal{V} \Leftrightarrow \langle T\mathbf{v}, \mathbf{w} \rangle = \mathbf{0} \ \forall \ \mathbf{v} \in \mathcal{V} \Leftrightarrow \mathbf{w} \in \text{range}(T)$$

Thus $\text{null}(T^*) = (\text{range}(T))^\perp$.

(*iv*) Taking orthogonal complements and using $(\mathcal{U}^\perp)^\perp$, $\text{range}(T) = (\text{null}(T^*))^\perp$.

(*iii*) Replace T by T^* in (*i*), $\text{null}(T^*) = (\text{range}(T^*))^\perp$.

(*ii*) Taking orthogonal complements, $\text{range}(T^*) = (\text{null}(T))^\perp$.

□

The adjoint transfers action from one slot of the inner product to the other, converting algebraic conditions (null/range) into geometric ones (orthogonality).

Definition 1.2.95. The *conjugate transpose* of an $m \times n$ matrix is obtained by interchanging rows and columns, then taking the complex conjugate of each entry.

Proposition 1.2.96. *Suppose $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$. If $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ is an orthonormal basis of $\mathcal{V}$ and $(\mathbf{f}_1, \ldots, \mathbf{f}_m)$ is an orthonormal basis of $\mathcal{W}$, then*

$$\mathcal{M}(T^*, (\mathbf{f}_1, \ldots, \mathbf{f}_m), (\mathbf{e}_1, \ldots, \mathbf{e}_n))$$

is the conjugate transpose of

$$\mathcal{M}(T, (\mathbf{e}_1, \ldots, \mathbf{e}_n), (\mathbf{f}_1, \ldots, \mathbf{f}_m))$$

Proof. Let $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ and $(\mathbf{f}_1, \ldots, \mathbf{f}_m)$ be orthonormal bases of $\mathcal{V}$ and $\mathcal{W}$.
The (j,k)-entry of $\mathcal{M}(T)$ is $\mathcal{M}(T)_{j,k} = \langle T\mathbf{e}_k, \mathbf{f}_j \rangle$.
The (k,j)-entry of $\mathcal{M}(T^*)$ is $\mathcal{M}(T^*)_{j,k} = \langle T^*\mathbf{f}_j, \mathbf{e}_k \rangle = \langle \mathbf{f}_j, T\mathbf{e}_k \rangle$.
By conjugate symmetry, $\langle \mathbf{f}_j, T\mathbf{e}_k \rangle = \overline{\langle T\mathbf{e}_k, \mathbf{f}_j \rangle}$.
Thus $\mathcal{M}(T^*)_{k,j} = \overline{\mathcal{M}(T)_{j,k}}$, so $\mathcal{M}(T^*)$ is the conjugate transpose of $\mathcal{M}(T)$. □

Matrix entries of an operator are inner products; the adjoint swaps the operator across the inner product, which corresponds exactly to taking the conjugate transpose.

1.2.4 Operators on Inner-Product Spaces

Definition 1.2.97. An operator $T \in \mathcal{L}(\mathcal{V})$ is *self-adjoint* or *Hermitian* if $T = T^*$.

A self-adjoint operator is analogous to a real number. It plays similar role to complex conjugation on $\mathbb{C}$.

Proposition 1.2.98. *Every eigenvalue of a self-adjoint operator is real.*

Proof. Let $T \in \mathcal{L}(\mathcal{V})$ be self-adjoint, and suppose $T\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$. Then $\langle T\mathbf{v}, \mathbf{v} \rangle = \langle \mathbf{v}, T\mathbf{v} \rangle$.
Substituting $T\mathbf{v} = \lambda\mathbf{v}$, $\langle \lambda\mathbf{v}, \mathbf{v} \rangle = \langle \mathbf{v}, \lambda\mathbf{v} \rangle$.
Using linearity in first slot and conjugate linearity in the second, $\lambda \|\mathbf{v}\|^2 = \overline{\lambda} \|\mathbf{v}\|^2$. Because $\|\mathbf{v}\|^2 = 0$, it follows that $\lambda = \overline{\lambda}$, hence $\lambda \in \mathbb{R}$. This can be interpreted as $\langle (T - \lambda I)\mathbf{v}, \mathbf{v} \rangle = 0$, and self-adjointness forces symmetry of the scalar, which constrains λ to be real. □

Self-adjointness forces inner products $\langle T\mathbf{v}, \mathbf{v} \rangle$ to be real. Applying this to an eigenvector converts the eigenvalue into its own complex conjugate, hence it must be real.

Proposition 1.2.99. *If $\mathcal{V}$ is a complex inner-product space and T is an operator on $\mathcal{V}$ such that $\langle T\mathbf{v}, \mathbf{v} \rangle = 0$ for all $\mathbf{v} \in \mathcal{V}$, then $T = 0$.*

Proof. Let $\mathcal{V}$ be a complex inner-product space and $T \in \mathcal{L}(\mathcal{V})$ satisfy $\langle T\mathbf{v}, \mathbf{v} \rangle = \mathbf{0}$ for all $\mathbf{v} \in \mathcal{V}$.
Use polarisation identity: for all $\mathbf{u}, \mathbf{w} \in \mathcal{V}$,

$$\langle T\mathbf{u}, \mathbf{w} \rangle = \frac{\langle T(\mathbf{u}+\mathbf{w}), \mathbf{u}+\mathbf{w} \rangle - \langle T(\mathbf{u}-\mathbf{w}), \mathbf{u}-\mathbf{w} \rangle}{4} + \frac{\langle T(\mathbf{u}+i\mathbf{w}), \mathbf{u}+i\mathbf{w} \rangle - \langle T(\mathbf{u}-i\mathbf{w}), \mathbf{u}-i\mathbf{w} \rangle}{4} i$$

Each term on the right is of form $\langle T\mathbf{u}, \mathbf{w} \rangle$, hence equals $\mathbf{0}$ by assumption. Therefore, $\langle T\mathbf{u}, \mathbf{w} \rangle = \mathbf{0}$ for all $\mathbf{u}, \mathbf{w} \in \mathcal{V}$. Fix $\mathbf{u}$. Taking $\mathbf{w} = T\mathbf{u}$, we obtain $\langle T\mathbf{u}, T\mathbf{u} \rangle = \mathbf{0}$, which implies $T\mathbf{u} = \mathbf{0}$. Thus $T = 0$.
Structural Remark: The condition implies $\text{range}(T) \subseteq (\text{range}(T))^{\perp}$, so every vector in the range is orthogonal to itself, forcing the range to be $\{\mathbf{0}\}$. Hence $T = 0$. □

The quadratic form $\mathbf{v} \mapsto \langle T\mathbf{v}, \mathbf{v} \rangle$ determines the full sesquilinear form via polarisation. If it vanishes everywhere, then $\langle T\mathbf{v}, \mathbf{v} \rangle = \mathbf{0}$ for all $\mathbf{u}, \mathbf{w}$, forcing $T = 0$.

Corollary 1.2.100. *Let $\mathcal{V}$ be complex inner-product space and let $T \in \mathcal{L}(\mathcal{V})$. Then T is self-adjoint if and only if $\langle T\mathbf{v}, \mathbf{v} \rangle \in \mathbb{R}$ for every $\mathbf{v} \in \mathcal{V}$.*

Proof. Let $\mathcal{V}$ be a complex inner-product space and $T \in \mathcal{L}(\mathcal{V})$.
($\Rightarrow$) Suppose T is self-adjoint, so $T = T^*$. Then for any $\mathbf{v} \in \mathcal{V}$, $\langle T\mathbf{v}, \mathbf{v} \rangle = \langle \mathbf{v}, T\mathbf{v} \rangle = \overline{\langle T\mathbf{v}, \mathbf{v} \rangle}$. Hence $\langle T\mathbf{v}, \mathbf{v} \rangle \in \mathbb{R}$.
($\Leftrightarrow$) Suppose T is self-adjoint, so $T = T^*$. Then for any $\mathbf{v} \in \mathcal{V}$, $\langle T\mathbf{v}, \mathbf{v} \rangle = \overline{\langle T\mathbf{v}, \mathbf{v} \rangle} = \langle \mathbf{v}, T\mathbf{v} \rangle = \langle T^*\mathbf{v}, \mathbf{v} \rangle$.
Thus $\langle (T - T^*)\mathbf{v}, \mathbf{v} \rangle = \mathbf{0}$ for all $\mathbf{v} \in \mathcal{V}$. By Proposition 1.2.99, $T - T^* = 0$, hence $T = T^*$, so T is self-adjoint.
Structural Remark: The condition says the quadratic form of $T - T^*$ vanishes everywhere. By the previous proposition, this forces $\text{range}(T - T^*) = \{\mathbf{0}\}$, so $T = T^*$. □

Self-adjointness is exactly the condition that the quadratic form $\mathbf{v} \mapsto \langle T\mathbf{v}, \mathbf{v}\rangle$ is real-valued. The polarisation mechanism ensures this quadratic form fully determines the operator.

Proposition 1.2.101. *If T is a self-adjoint operator on $\mathcal{V}$ such that $\langle T\mathbf{v}, \mathbf{v}\rangle = 0$ for all $\mathbf{v} \in \mathcal{V}$, then $T = 0$.*

Proof. Let $\mathcal{V}$ be a real inner-product space and $T \in \mathcal{L}(\mathcal{V})$ be self-adjoint. Assume $\langle T\mathbf{v}, \mathbf{v}\rangle = \mathbf{0}$ for all $\mathbf{v} \in \mathcal{V}$. For any $\mathbf{u}, \mathbf{w} \in \mathcal{V}$, the real polarisation identity gives

$$\langle T\mathbf{u}, \mathbf{v}\rangle = \frac{\langle T(\mathbf{u}+\mathbf{w}), \mathbf{u}+\mathbf{w}\rangle - \langle T(\mathbf{u}-\mathbf{w}), \mathbf{u}-\mathbf{w}\rangle}{4}$$

Because T is self-adjoint, $\langle T\mathbf{w}, \mathbf{u}\rangle = \langle \mathbf{w}, T\mathbf{u}\rangle = \langle T\mathbf{u}, \mathbf{w}\rangle$, so the cross terms combine correctly in the expansion. Each term on the right is of form $\langle T\mathbf{v}, \mathbf{v}\rangle$, hence equals $\mathbf{0}$ by assumption. Therefore, $\langle T\mathbf{u}, \mathbf{w}\rangle = \mathbf{0}\ \forall\ \mathbf{u}, \mathbf{w} \in \mathcal{V}$. Now take $\mathbf{w} = T\mathbf{u}$, then $\langle T\mathbf{u}, T\mathbf{u}\rangle = \mathbf{0}$ which implies $T\mathbf{u} = 0$. Thus $T = 0$.
Structural Remark: Over a real inner-product space, skew-adjoint operators satisfy $\langle T\mathbf{v}, \mathbf{v}\rangle = \mathbf{0}\ \forall\ \mathbf{v}$ without being zero (i.e. $90°$ rotation in $\mathbb{R}$). Unlike the complex case, the condition alone does not force $T = 0$; self-adjointness removes the skew-symmetric part. □

In a real inner-product space, the quadratic form $\mathbf{v} \mapsto \langle T\mathbf{v}, \mathbf{v}\rangle$ determines only the symmetric/self-adjoint part of T. Since T is self-adjoint and the quadratic form vanishes everywhere, the operator itself must vanish.

Definition 1.2.102. An operator on an inner-product space is *normal* if it commutes with its adjoint, i.e., $T \in \mathcal{L}(\mathcal{V})$ is normal if $TT^* = T^*T$. Every self-adjoint operator is normal.

Proposition 1.2.103. *An operator $T \in \mathcal{L}(\mathcal{V})$ is normal if and only if $\|T\mathbf{v}\| = \|T^*\mathbf{v}\|$ for all $\mathbf{v} \in \mathcal{V}$.*

Proof. Let $T \in \mathcal{L}(\mathcal{V})$. Recall that T is normal if $T^*T = TT^*$. Because both T^*T and TT^* are self-adjoint, $T^*T - TT^*$ is also self-adjoint. Hence by Proposition 1.2.101, $T^*T - TT^* = 0$ if and only if

$$\langle (T^*T - TT^*)\mathbf{v}, \mathbf{v}\rangle = \mathbf{0} \quad \text{for all } \mathbf{v} \in \mathcal{V}$$

Thus, T is normal if and only if $\langle T^*T\mathbf{v}, \mathbf{v}\rangle = \langle TT^*\mathbf{v}, \mathbf{v}\rangle$ for all $\mathbf{v} \in \mathcal{V}$.
Using definition of adjoint, $\langle T^*T\mathbf{v}, \mathbf{v}\rangle = \langle Tv, T\mathbf{v}\rangle = \|T\mathbf{v}\|^2$, and similarly, $\langle TT^*\mathbf{v}, \mathbf{v}\rangle = \langle T^*\mathbf{v}, T^*\mathbf{v}\rangle = \|T^*\mathbf{v}\|^2$.
Therefore, T is normal $\Leftrightarrow \|T\mathbf{v}\| = \|T^*\mathbf{v}\|$ for all $\mathbf{v} \in \mathcal{V}$.
Structural Interpretation: This states that normality is exactly the condition that T and T^* distort vector lengths identically. Self-adjoint operators, unitary operators, and orthogonal projections all fit into this framework. □

Normality measures the compatibility between T and its adjoint. The condition $T^*T = TT^*$ is equivalent to saying T and T^* produce the same squared norm on every vector.

Corollary 1.2.104. *Suppose $T \in \mathcal{L}(\mathcal{V})$ is normal. If $\mathbf{v} \in \mathcal{V}$ is an eigenvector of T with eigenvalue $\lambda \in \mathbb{F}$, then $\mathbf{v}$ is also an eigenvector of T^* with eigenvalue $\overline{\lambda}$.*

Proof. Suppose $T \in \mathcal{L}(\mathcal{V})$ is normal and $\mathbf{v} \neq 0$ satisfies $T\mathbf{v} = \lambda\mathbf{v}$. Then $(T - \lambda I)\mathbf{v} = \mathbf{0}$, so $\mathbf{v} \in \text{null}(T - \lambda I)$.
Because T is normal, $T - \lambda I$ is also normal. By Proposition 1.2.103, $\|(T - \lambda I)\mathbf{v}\| = \|(T - \lambda I)^*\mathbf{v}\|\ \forall\ \mathbf{v} \in \mathcal{V}$.
Since $(T - \lambda I)\mathbf{v} = \mathbf{0}$, $0 = \|(T - \lambda I)\mathbf{v}\| = \|(T - \lambda I)^*\mathbf{v}\|$.
A vector has norm 0 if and only if it is $\mathbf{0}$, hence $(T - \lambda I)^*\mathbf{v} = \mathbf{0}$.
Using $(T - \lambda I)^* = T^* - \overline{\lambda} I$, we obtain $(T^* - \overline{\lambda} I)\mathbf{v} = \mathbf{0}$, that is, $T^*\mathbf{v} = \overline{\lambda}\mathbf{v}$.
Therefore, $\mathbf{v}$ is an eigenvalue of T^* with eigenvalue $\overline{\lambda}$.
Structural Interpretation: For normal operators, $\text{null}(T - \lambda I) = \text{null}((T - \lambda I)^*)$. This is the real mechanism behind the theorem. Eigenvectors of T automatically lie in the corresponding adjoint nullspace. □

Normal operators preserve the nullspace structure between an operator and its adjoint. Thus if $(T - \lambda I)\mathbf{v} = \mathbf{0}$, normality forces $(T^* - \overline{\lambda} I)\mathbf{v} = \mathbf{0}$ as well.

Corollary 1.2.105. *If $T \in \mathcal{L}(\mathcal{V})$ is normal, eigenvectors of T with unique eigenvalues are orthogonal.*

Proof. Suppose $T \in \mathcal{L}(\mathcal{V})$ is normal. Let $T\mathbf{u} = \alpha\mathbf{u}$, $T\mathbf{v} = \beta\mathbf{v}$ where $\alpha \neq \beta$.
By Corollary 1.2.104, $T^*\mathbf{v} = \overline{\beta}\mathbf{v}$. Now compute $\alpha \langle \mathbf{u}, \mathbf{v}\rangle = \langle \alpha\mathbf{u}, \mathbf{v}\rangle = \langle T\mathbf{u}, \mathbf{v}\rangle$. Using definition of adjoint,

$$\langle T\mathbf{u}, \mathbf{v}\rangle = \langle \mathbf{u}, T^*\mathbf{v}\rangle = \langle \mathbf{u}, \overline{\beta}\mathbf{v}\rangle = \beta \langle \mathbf{u}, \mathbf{v}\rangle$$

Therefore, $\alpha \langle \mathbf{u}, \mathbf{v}\rangle = \beta \langle \mathbf{u}, \mathbf{v}\rangle$, so $(\alpha - \beta) \langle \mathbf{u}, \mathbf{v}\rangle = \mathbf{0}$. Because $\alpha \neq \beta$, $\langle \mathbf{u}, \mathbf{v}\rangle = \mathbf{0}$. Hence $\mathbf{u}, \mathbf{v}$ are orthogonal.
Structural Interpretation: Distinct eigenspaces of a normal operator are orthogonal subspaces, $E_\alpha(T) \perp E_\beta(T)$, $\alpha \neq \beta$. This orthogonality allows normal operators to admit orthonormal eigenbases and diagonalisation. □

Normality allows eigenvectors of T to also behave compatibly with T^*. Distinct eigenvalues force incompatible scalar actions on the same inner product, leaving orthogonality as the only possibility.

Theorem 1.2.106. *Complex Spectral Theorem*
Suppose $\mathcal{V}$ is a complex inner-product space and $T \in \mathcal{L}(\mathcal{V})$. Then $\mathcal{V}$ has an orthonormal basis consisting of eigenvectors of T if and only if T is normal.

Proof. ($\Rightarrow$) Suppose $\mathcal{V}$ has an orthonormal basis consisting of eigenvectors of T.
With respect to this basis, the matrix of T is diagonal, $\mathcal{M}(T) = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. The matrix of T^* is the conjugate transpose, $\mathcal{M}(T^*) = \operatorname{diag}(\overline{\lambda_1}, \dots, \overline{\lambda_n})$.
Diagonal matrices commute, then $TT^* = T^*T$, therefore T is normal.
($\Leftarrow$) Now suppose T is normal. By Schur Triangularisation Theorem 1.2.55, $\exists$ an orthonormal basis $(\mathbf{e}_1, \dots, \mathbf{e}_n)$ such that the matrix of T is upper triangular,

$$\mathcal{M}(T, (\mathbf{e}_1, \dots, \mathbf{e}_n)) = \begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & \ddots & \vdots \\ 0 & \cdots & a_{n,n} \end{bmatrix}$$

To show all entries above the diagonal are 0.
Because $T\mathbf{e}_1 = a_{1,1}\mathbf{e}_1$, hence $\|T\mathbf{e}_1\|^2 = |a_{1,1}|^2$. On the other hand, $T^*\mathbf{e}_1 = \overline{a_{1,1}}\mathbf{e}_1 + \overline{a_{1,2}}\mathbf{e}_2 + \cdots + \overline{a_{1,n}}\mathbf{e}_n$, so $\|T^*\mathbf{e}_1\|^2 = |a_{1,1}|^2 + |a_{1,2}|^2 + \cdots + |a_{1,n}|^2$. Since T is normal, $\|T\mathbf{e}_1\| = \|T^*\mathbf{e}_1\|$. Therefore, $|a_{1,1}|^2 + \cdots + |a_{1,n}|^2 = 0$, hence $a_{1,2} = \cdots = a_{1,n} = 0$. Thus the first row is diagonal.
Consider $\mathbf{e}_2$. Since $a_{1,2} = 0$, $T\mathbf{e}_2 = a_{2,2}\mathbf{e}_2$, so $\|T\mathbf{e}_2\|^2 = |a_{2,2}|^2$. Similarly, $\|T^*\mathbf{e}_2\|^2 = |a_{2,2}|^2 + |a_{2,3}|^2 + \cdots + |a_{2,n}|^2$.
Again, normality gives $\|T\mathbf{e}_2\| = \|T^*\mathbf{e}_2\|$, so $a_{2,3} = \cdots = a_{2,n} = 0$.
Continuing inductively, every entry above the diagonal is 0, hence the matrix of T is diagonal.
Therefore, the orthonormal basis $(\mathbf{e}_1, \dots, \mathbf{e}_n)$ consists of eigenvectors of T.
Structural Interpretation: Schur states every complex operator is almost diagonal (upper triangular). Normality is the extra rigidity that collapses the triangular part, forcing genuine diagonalisation by an orthonormal basis. □

Every complex operator can be orthogonally triangularized. Normality forces the strictly upper-triangular part to vanish because T and T^* must act with identical norm behaviour on basis vectors.

Lemma 1.2.107. *Suppose $T \in \mathcal{L}(\mathcal{V})$ is self-adjoint. If $\alpha, \beta \in \mathbb{R}$ are such that $\alpha^2 < 4\beta$, then*

$$T^2 + \alpha T + \beta I$$

is invertible.

Proof. Suppose $T \in \mathcal{L}(\mathcal{V})$ is self-adjoint and $\alpha^2 < 4\beta$, where $\alpha, \beta \in \mathbb{R}$. To show $T^2 + \alpha T + \beta I$ is invertible.
Let $S = T^2 + \alpha T + \beta I$. To prove invertibility, it suffices to show $\operatorname{null}(S) = \{\mathbf{0}\}$. Let $\mathbf{v} \neq \mathbf{0}$. Then

$$\langle S\mathbf{v}, \mathbf{v} \rangle = \langle T^2\mathbf{v}, \mathbf{v} \rangle + \alpha \langle T\mathbf{v}, \mathbf{v} \rangle + \beta \langle \mathbf{v}, \mathbf{v} \rangle$$

Because T is self-adjoint, $\langle T^2\mathbf{v}, \mathbf{v} \rangle = \langle T\mathbf{v}, T\mathbf{v} \rangle = \|T\mathbf{v}\|^2$. Hence $\langle S\mathbf{v}, \mathbf{v} \rangle = \|T\mathbf{v}\|^2 + \alpha \langle T\mathbf{v}, \mathbf{v} \rangle + \beta \|\mathbf{v}\|^2$.
By Cauchy-Schwarz Inequality (Theorem 1.2.70), $|\langle T\mathbf{v}, \mathbf{v} \rangle| \leq \|T\mathbf{v}\| \, \|\mathbf{v}\|$, so

$$\langle S\mathbf{v}, \mathbf{v} \rangle \geq \|T\mathbf{v}\|^2 - |\alpha| \, \|T\mathbf{v}\| \, \|\mathbf{v}\| + \beta \|\mathbf{v}\|^2$$

Completing the square,

$$\langle S\mathbf{v}, \mathbf{v} \rangle \geq \left(\|T\mathbf{v}\| - \frac{|\alpha|}{2} \|\mathbf{v}\| \right)^2 + \left(\beta - \frac{\alpha^2}{4} \right) \|\mathbf{v}\|^2$$

Since $\beta - \frac{\alpha^2}{4} > 0$ and $\mathbf{v} \neq \mathbf{0}$, the right-hand-side is strictly positive, hence $\langle S\mathbf{v}, \mathbf{v} \rangle > 0$.
In particular, $S\mathbf{v} \neq 0$. Thus $\mathbf{N}(S) = \{\mathbf{0}\}$, so S is injective. Since $\mathcal{V}$ is finite-dimensional injective implies invertible. Therefore $T^2 + \alpha T + \beta I$ is invertible.
Structural Interpretation: A self-adjoint operator behaves spectrally like a real number. A quadratic polynomial with no real roots stays strictly positive on the spectrum, so the operator cannot acquire a nontrivial nullspace. □

A self-adjoint operator behaves like a real scalar inside inner products; since the quadratic is positive on $\mathbb{R}$, the operator polynomial cannot have a nontrivial nullspace.

Lemma 1.2.108. *Suppose $T \in \mathcal{L}(\mathcal{V})$ is self-adjoint, then T has an eigenvalue.*

Proof. Suppose $T \in \mathcal{L}(\mathcal{V})$ is self-adjoint, $\mathcal{V}$ is finite-dim real inner-product space. To show T has an eigenvalue. Let $n = \dim(\mathcal{V})$, choose $\mathbf{v} \neq \mathbf{0}$. The list $(\mathbf{v}, T\mathbf{v}, T^2\mathbf{v}, \ldots, T^n\mathbf{v})$ contains $n+1$ vectors in n-dim space, so it is linearly dependent. Hence $\exists$ scalars $a_0, \ldots, a_n \in \mathbb{R}$ not all zero, s.t. $a_0\mathbf{v} + a_1T\mathbf{v} + \cdots + a_nT^n\mathbf{v} = \mathbf{0}$.
Define polynomial $p(x) = a_0 + a_1x + \cdots + a_nx^n$. Then $p(T)\mathbf{v} = \mathbf{0}$. As p is a real polynomial, it factors over R,

$$p(x) = c(x^2 + \alpha_1 x + \beta_1) \cdots (x^2 + \alpha_M x + \beta_M)(x - \lambda_1) \cdots (x - \lambda_m)$$

where $c \neq 0$, each $\lambda_j \in \mathbb{R}$, each quadratic factor satisfies $\alpha_j^2 < 4\beta_j$. Substituting T for x,

$$0 = p(T)\mathbf{v} = c(T^2 + \alpha_1 T + \beta_1 I) \cdots (T^2 + \alpha_M T + \beta_M I)(T - \lambda_1 I) \cdots (T - \lambda_m I)\mathbf{v}$$

By Lemma 1.2.107, each operator $T^2 + \alpha_j T + \beta_j I$ is invertible because T is self-adjoint and $\alpha_j^2 < 4\beta_j$.
Also $c \neq 0$. Multiplying by inverses of quadratic factors gives $0 = (T - \lambda_1 I) \cdots (T - \lambda_m I)\mathbf{v}$. Thus the product operator $(T - \lambda_1 I) \cdots (T - \lambda_m I)$ is not injective.
A product of injective operators is injective, so at least one factor $T - \lambda_j I$ is not injective. Hence $\text{null}(T - \lambda_j I) \neq \{\mathbf{0}\}$, so there exists a nonzero vector $\mathbf{u}$ such that $T\mathbf{u} = \lambda_j\mathbf{u}$. Therefore T has an eigenvalue.
Structural Interpretation: The proof separates the polynomial annihilating $\mathbf{v}$ into invertible pieces and potentially singular pieces. Self-adjointness forces every irreducible quadratic factor to stay invertible, leaving only linear factors capable of creating a nullspace. □

A polynomial in a self-adjoint operator annihilates some vector. Since irreducible quadratic factors remain invertible, the nullspace must come from a linear factor $T - \lambda I$, producing an eigenvalue.

Theorem 1.2.109. *Real Spectral Theorem*
Suppose $\mathcal{V}$ is a real inner-product space and $T \in \mathcal{L}(\mathcal{V})$, then $\mathcal{V}$ has an orthonormal basis consisting of eigenvectors of T if and only if T is self-adjoint.

Proof. ($\Rightarrow$) Suppose $\mathcal{V}$ has an orthonormal basis $\mathbf{e}_1, \ldots, \mathbf{e}_n$ consisting of eigenvectors of T, with $T\mathbf{e}_j = \lambda_j\mathbf{e}_j$. Because the basis is orthonormal, to show T is self-adjoint, verify $\langle T\mathbf{v}, \mathbf{w}\rangle = \langle \mathbf{v}, T\mathbf{w}\rangle$ on basis vectors.
For i, j, $\langle T\mathbf{e}_i, \mathbf{e}_j\rangle = \langle \lambda_i\mathbf{e}_i, \mathbf{e}_j\rangle = \lambda_j\delta_{i,j}$, while $\langle \mathbf{e}_i, T\mathbf{e}_j\rangle = \langle \mathbf{e}_i, \lambda_j\mathbf{e}_j\rangle = \lambda_j\delta_{i,j}$. These are equal because when $i \neq j$, both are 0, and when $i = j$, both equal λ_i. Hence $\langle T\mathbf{v}, \mathbf{w}\rangle = \langle \mathbf{v}, T\mathbf{w}\rangle \ \forall\ \mathbf{v}, \mathbf{w} \in \mathcal{V}$, so T is self-adjoint.
($\Leftarrow$) Suppose T is self-adjoint. Proceed by induction on $\dim(\mathcal{V})$. Result is clear if $\dim(\mathcal{V}) = 1$.
Assume $\dim(\mathcal{V}) > 1$, and this holds for all smaller dimensions. As T is self-adjoint, T has real eigenvalue λ.
Let $\mathcal{U} = \text{null}(T - \lambda I)$ be the eigenspace corresponding to λ. Choose unit vector $\mathbf{u} \in \mathcal{U}$. Claim that $\mathcal{U}^\perp$ is invariant under T. Indeed, let $\mathbf{v} \in \mathcal{U}^\perp$. For any $\mathbf{x} \in \mathcal{U}$, $T\mathbf{x} = \lambda\mathbf{x}$. Using self-adjointness,

$$\langle T\mathbf{v}, \mathbf{x}\rangle = \langle \mathbf{v}, T\mathbf{x}\rangle = \langle \mathbf{v}, \lambda\mathbf{x}\rangle = \lambda\langle \mathbf{v}, \mathbf{x}\rangle = 0$$

Thus $T\mathbf{v} \in \mathcal{U}^\perp$. Hence the restriction $S = T|_{\mathcal{U}^\perp}$ is a well-defined operator on $\mathcal{U}^\perp$. Moreover, for $\mathbf{v}, \mathbf{w} \in \mathcal{U}^\perp$,

$$\langle S\mathbf{v}, \mathbf{w}\rangle = \langle T\mathbf{v}, \mathbf{w}\rangle = \langle \mathbf{v}, T\mathbf{w}\rangle = \langle \mathbf{v}, S\mathbf{w}\rangle$$

So S is self-adjoint. Because $\dim(\mathcal{U}^\perp) < \dim(\mathcal{V})$, the induction hypothesis implies $\mathcal{U}^\perp$ has an orthonormal basis consisting of eigenvectors of S. Every eigenvector of S is also an eigenvector of T, since $S\mathbf{v} = T\mathbf{v}$ for $\mathbf{v} \in \mathcal{U}^\perp$.
Adjoin the unit vector $\mathbf{u}$ (or an orthonormal basis of $\mathcal{U}$ if $\dim(\mathcal{U}) > 1$) to this basis of $\mathcal{U}^\perp$. Since $\mathcal{V} = \mathcal{U} \oplus \mathcal{U}^\perp$, obtain an orthonormal basis of $\mathcal{V}$ consisting of all eigenvectors of T. Hence T has an orthonormal eigenbasis. □

A self-adjoint operator preserves orthogonality relative to its eigenspaces. Once an eigenspace $\mathcal{U} = \text{null}(T - \lambda I)$ is found, the invariant subspace $\mathcal{U}^\perp$ allows the problem to be reduced inductively to lower dimension.

Corollary 1.2.110. *Suppose $T \in \mathcal{L}(\mathcal{V})$ is self-adjoint, let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T, then*

$$\mathcal{V} = \textit{null}(T - \lambda_1 I) \oplus \cdots \oplus \textit{null}(T - \lambda_m I)$$

Each vector in each null$(T - \lambda_j I)$ is orthogonal to all vectors in other subspaces of this decomposition.

Proof. By Spectral Theorems 1.2.106 and 1.2.109, $\mathcal{V}$ has an orthonormal basis consisting of eigenvectors of T. Each basis vector belongs to some eigenspace $\text{null}(T - \lambda_j I)$. Hence the sum $\text{null}(T - \lambda_1 I) + \cdots + \text{null}(T - \lambda_m I)$ contains an orthonormal basis of $\mathcal{V}$, therefore equals $\mathcal{V}$.
To show the sum is direct, suppose $\mathbf{v}_j \in \text{null}(T - \lambda_j I)$ and $\mathbf{v}_1 + \cdots + \mathbf{v}_m = \mathbf{0}$. Because eigenvectors corresponding to distinct eigenvalues are linearly independent, each $\mathbf{v}_j = \mathbf{0}$, thus the sum is direct.
Finally, if $\mathbf{u} \in \text{null}(T - \lambda_i I)$, $\mathbf{v} \in \text{null}(T - \lambda_j I)$, $i \neq j$, then $\mathbf{u}, \mathbf{v}$ are eigenvectors corresponding to distinct eigenvalues of self-adjoint operator T. Hence $\langle \mathbf{u}, \mathbf{u}\rangle = 0$. Therefore the decomposition is orthogonal. □

A self-adjoint operator admits an orthonormal eigenbasis, so the whole space splits into eigenspaces. Distinct eigenspaces are automatically orthogonal, giving an orthogonal direct-sum decomposition of $\mathcal{V}$.

Lemma 1.2.111. *Suppose $\mathcal{V}$ is a real inner-product space, dim($\mathcal{V}$) and $T \in \mathcal{L}(\mathcal{V})$, these are equivalent:*

i. T is normal but not self-adjoint

ii. the matrix of T with respect to every orthonormal basis of $\mathcal{V}$ has the form

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

with $b \neq 0$.

iii. the matrix of T with respect to some orthonormal basis of $\mathcal{V}$ has the form

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

with $b > 0$.

Proof.

$(i) \Rightarrow (ii)$ Let $(\mathbf{e}_1, \mathbf{e}_2)$ be any orthonormal basis, and write

$$\mathcal{M}(T, (\mathbf{e}_1, \mathbf{e}_2)) = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$$

As the basis is orthonormal, $\mathcal{M}(T^*) = \mathcal{M}(T)^T$ (transpose). Since T is normal, $TT^* = T^*T$. Comparing the $(1,1)$-entry gives $a^2 + c^2 = a^2 + b^2$, so $c^2 = b^2$, hence $c = b$ or $c = -b$.
If $c = b$, then $\mathcal{T} = \mathcal{M}(T)^T$, so T is self-adjoint, contradicting (i). Hence $c = -b$. Compare off-diagonal entries of TT^* and T^*T, $ab - bd = ac + cd$. Substitute $c = -b$, so $ab - bd = -ab - bd$, hence $a = d$. Thus

$$\mathcal{M}(T) = \begin{bmatrix} a & -b \\ b & d \end{bmatrix}$$

Because T is not self-adjoint, $b \neq 0$.

$(ii) \Rightarrow (iii)$ If $b < 0$, replace $\mathbf{e}_2$ by $-\mathbf{e}_2$, then matrix becomes

$$\mathcal{M}(T) = \begin{bmatrix} a & -(-b) \\ (-b) & d \end{bmatrix}$$

and now $-b > 0$.

$(iii) \Rightarrow (i)$ Suppose

$$\mathcal{M}(T) = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}, \quad b > 0$$

The transpose will be different as $b \neq 0$, hence T is not self-adjoint. Direct computation shows

$$TT^* = T^*T = \begin{bmatrix} a^2 + b^2 & 0 \\ 0 & a^2 + b^2 \end{bmatrix}$$

Hence T is normal

□

In dimension 2, a real normal operator is either symmetric or acts like multiplication by a complex number $a + bi$. The matrix below is precisely the real form of complex multiplication.

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

Proposition 1.2.112. *Suppose $T \in \mathcal{L}(\mathcal{V})$ is normal and $\mathcal{U}$ is a subspace of $\mathcal{V}$ that is invariant under T. Then*

i. $\mathcal{U}^\perp$ is invariant under T

ii. $\mathcal{U}$ is invariant under T^*

iii. $(T|_{\mathcal{U}})^* = (T^*)_{\mathcal{U}}$

iv. $T|_{\mathcal{U}}$ *is a normal operator on* $\mathcal{U}$

v. $T_{\mathcal{U}^\perp}$ *is a normal operator on* $\mathcal{U}^\perp$

Proof.

(*i*) Let $\mathbf{v} \in \mathcal{U}^\perp$. To show $T\mathbf{v} \in \mathcal{U}^\perp$.
Take any $\mathbf{u} \in \mathcal{U}$. Since $\mathcal{U}$ is invariant under T, $T\mathbf{u} \in \mathcal{U}$.
Because T is normal, $\text{null}(T - \lambda I) = \text{null}((T - \lambda I)^*)$ for every λ, and more generally normal operators admit orthogonal decompositions compatible with invariant subspaces.
Equivalently, in an orthonormal basis adapted to $\mathcal{V} = \mathcal{U} \oplus \mathcal{U}^\perp$, the matrix of T has block form

$$\begin{bmatrix} A & B \\ 0 & C \end{bmatrix}$$

Normality forces $B = 0$, thus $T(\mathcal{U}^\perp) \subseteq \mathcal{U}^\perp$, so $\mathcal{U}^\perp$ is invariant under T.

(*ii*) Since $\mathcal{U}^\perp$ is invariant under T for $\mathbf{u} \in \mathcal{U}$ and $\mathbf{v} \in \mathcal{U}^\perp$, $\langle T^*\mathbf{u}, \mathbf{v}\rangle = \langle \mathbf{u}, T\mathbf{v}\rangle$.
Because $T\mathbf{v} \in \mathcal{U}^\perp$ and $\mathbf{u} \in \mathcal{U}$, $\langle \mathbf{u}, T\mathbf{v}\rangle = 0$. Hence $\langle T^*\mathbf{u}, \mathbf{v}\rangle = 0$ for every $\mathbf{v} \in \mathcal{U}^\perp$.
Therefore, $T^*\mathbf{u} \in (\mathcal{U}^\perp)^\perp = \mathcal{U}$, so $\mathcal{U}$ is invariant under T^*.

(*iii*) Let $S = T|_{\mathcal{U}}$. For $\mathbf{u}, \mathbf{v} \in \mathcal{U}$, $\langle S\mathbf{u}, \mathbf{v}\rangle = \langle T\mathbf{u}, \mathbf{v}\rangle = \langle \mathbf{u}, T^*\mathbf{v}\rangle$.
By (*ii*), $T^*\mathbf{v} \in \mathcal{U}$, so $S^*\mathbf{v} = T^*\mathbf{v}$, hence $(T|_{\mathcal{U}})^* = (T^*)|_{\mathcal{U}}$.

(*iv*) Using (*iii*), $(T|_{\mathcal{U}})(T|_{\mathcal{U}})^* = (T|_{\mathcal{U}})(T^*|_{\mathcal{U}}) = (TT^*)|_{\mathcal{U}}$.
Similarly, $(T|_{\mathcal{U}})^*(T|_{\mathcal{U}}) = (TT^*)|_{\mathcal{U}}$. Because T is normal, $TT^* = T^*T$.
Therefore, $(T|_{\mathcal{U}})(T|_{\mathcal{U}})^* = (T|_{\mathcal{U}})^*(T|_{\mathcal{U}})$, so $T|_{\mathcal{U}}$ is normal.

(*v*) By (*i*), $\mathcal{U}^\perp$ is invariant under T. Applying (*iv*) to $\mathcal{U}^\perp$, hence $T|_{\mathcal{U}^\perp}$ is normal.

□

For normal operators, invariant subspaces automatically come with invariant orthogonal complements. Thus the operator splits orthogonally into independent pieces, and normality is preserved on each piece separately.

Definition 1.2.113. A *block diagonal matrix* is a square matrix of form

$$\begin{bmatrix} A_1 & & 0 \\ & \ddots & \\ 0 & & A_m \end{bmatrix}$$

where $A_1, \ldots, A_m$ are square matrices along the diagonal, and all other entries of matrix equal 0.

Theorem 1.2.114. *Suppose* $\mathcal{V}$ *is a real inner-product space and* $T \in \mathcal{L}(\mathcal{V})$. *Then* T *is normal if and only if there is an orthonormal basis of* $\mathcal{V}$ *with respect to which* T *has a block diagonal matrix where each block is a* 1×1 *or* 2×2 *matrix of form*

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

where $b > 0$.

Proof.

(⇐) A 1×1 block is trivially normal. For 2×2 block,

$$A = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}, \quad A^T = \begin{bmatrix} a & b \\ -b & a \end{bmatrix}$$

and direct computation gives $AA^T = A^TA = (a^2 + b^2)I$, hence each block is normal.
A block diagonal operator is normal exactly when each block is normal, so T is normal.

(⇒) Assume T is normal, proceed by induction on $\dim(\mathcal{V})$. Result is trivial if $\dim(\mathcal{V}) = 1$.
Assume $\dim(\mathcal{V}) > 2$, and theorem holds for smaller dimensions.
First, construct a nonzero invariant subspace $\mathcal{U} \subseteq \mathcal{V}$ for dimension 1 or 2. If T has a real eigenvalue, let $\mathcal{U} = \text{null}(T - \lambda I)$. Choose a unit vector $\mathbf{u} \in \mathcal{U}$, then $\text{span}(\mathbf{u})$ is a 1-dim invariant subspace.
Otherwise, T has no eigenvalues. In this case, $\exists$ a 2-dim invariant subspace $\mathcal{U} \subseteq \mathcal{V}$. Since $T|_{\mathcal{U}}$ has no real eigenvalues, it cannot be self-adjoint. Because restrictions of normal operators to invariant subspaces remain norma, $T|_{\mathcal{U}}$ is normal by not self-adjoint.
By Lemma 1.2.111, $\exists$ an orthonormal basis of $\mathcal{U}$ such that matrix of $T|_{\mathcal{U}}$ is A defined in (⇐), with $b > 0$.

Thus in either case, $\mathcal{U}$ has an orthonormal basis for which the matrix $T|_{\mathcal{U}}$ has the required form.
Because T is normal and $\mathcal{U}$ is invariant under T, Proposition 1.2.112 implies $\mathcal{U}^{\perp}$ is invariant under T, and $T|_{\mathcal{U}^{\perp}}$ is normal. Since $\dim(\mathcal{U}^{\perp}) < \dim(\mathcal{V})$, the induction hypothesis applies to $T|_{\mathcal{U}^{\perp}}$.
Hence $\exists$ an orthonormal basis of $\mathcal{U}^{\perp}$ such that the matrix of $T|_{\mathcal{U}^{\perp}}$ has the desired block-diagonal form.
Combining this basis with orthonormal basis of $\mathcal{U}$ gives an orthonormal basis of $\mathcal{V}$ for which the matrix of T has the required block-diagonal form.

□

A real normal operator decomposes orthogonally into invariant pieces of dimension 1 or 2. The 1-dimensional pieces correspond to real eigenvalues, while the 2×2 blocks encode conjugate complex eigenvalue pairs as real rotation-scaling operators.

Definition 1.2.115. An operator $T \in \mathcal{L}(\mathcal{V})$ is *positive semi-definite* if T is self-adjoint and $\langle T\mathbf{v}, \mathbf{v}\rangle \geq 0 \; \forall \mathbf{v} \in \mathcal{V}$.

Definition 1.2.116. An operator S is a *square root* of an operator T if $S^2 = T$.

Theorem 1.2.117. *Let $T \in \mathcal{L}(\mathcal{V})$, then the following are equivalent:*

i. T *is positive*

ii. T *is self-adjoint and all eigenvalues of* T *are nonnegative*

iii. T *has a positive square root*

iv. T *has a self-adjoint square root*

v. there exists an operator $S \in \mathcal{L}(\mathcal{V})$ *such that* $T = S^*S$

Proof.

$(i) \Rightarrow (ii)$ Suppose T is positive. By definition, T is self-adjoint, and $\langle T\mathbf{v}, \mathbf{v}\rangle > 0 \; \forall \; \mathbf{v} \in \mathcal{V}$.
Let $\mathbf{v} \neq \mathbf{0}$ be an eigenvector of T with eigenvalue λ, then $0 \leq \langle T\mathbf{v}, \mathbf{v}\rangle = \langle \lambda\mathbf{v}, \mathbf{v}\rangle = \lambda \langle \mathbf{v}, \mathbf{v}\rangle$.
Since $\langle \mathbf{v}, \mathbf{v}\rangle > 0$, hence $\lambda \geq 0$, and every eigenvalue of T is non-negative.

$(ii) \Rightarrow (iii)$ Suppose T is self-adjoint and all eigenvalues are non-negative. By Spectral Theorem 1.2.109 and 1.2.106, $\exists$ an orthonormal basis $\mathbf{e}_1, \ldots, \mathbf{e}_n$ of eigenvectors of T, with $T\mathbf{e}_j = \lambda_j \mathbf{e}_j$, $\lambda_j \geq 0$. Define $S \in \mathcal{L}(\mathcal{V})$ by $S\mathbf{e}_j = \sqrt{\lambda_j}\mathbf{e}_j$. Then S is positive, and $S^2\mathbf{e}_j = \lambda_j \mathbf{e}_j = T\mathbf{e}_j$ for every j.
Hence $S^2 = T$, so T has a positive square root.

$(iii) \Rightarrow (iv)$ Every positive operator is self-adjoint by definition. Hence a positive square root is self-adjoint.

$(iv) \Rightarrow (v)$ Suppose R is a self-adjoint square root of T, then $R^2 = T$. Because $R = R^*$, thus $T = R^2 = R^*R$.
Hence $T = S^*S$ with $S = R$.

$(v) \Rightarrow (i)$ Suppose $T = S^*S$. Then $T^* = (S^*S)^* = S^*(S^*)^* = S^*S = T$, so T is self-adjoint.
Also, for every $\mathbf{v} \in \mathcal{V}$, $\langle T\mathbf{v}, \mathbf{v}\rangle = \langle S^*S\mathbf{v}, \mathbf{v}\rangle = \langle S\mathbf{v}, S\mathbf{v}\rangle = \|S\mathbf{v}\|^2 \geq 0$. Hence T is positive.

□

Positive operators are precisely the self-adjoint operators with nonnegative spectrum, and equivalently can be written as a square S^*S. The spectral theorem allows positivity to be understood entirely through eigenvalues.

Proposition 1.2.118. *Every positive operator on $\mathcal{V}$ has a unique positive square root.*

Proof. Let $T \in \mathcal{L}(\mathcal{V})$ be positive. By Theorem 1.2.117, T is self-adjoint, all eigenvalues of T are non-negative. Let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T. By Spectral Theorem 1.2.109 and 1.2.106,

$$\mathcal{V} = \operatorname{null}(T - \lambda_1 I) \oplus \cdots \oplus \operatorname{null}(T - \lambda_m I)$$

Define $S \in \mathcal{L}(\mathcal{V})$ by declaring $S\mathbf{v} = \sqrt{\lambda_j}\mathbf{v}$ for $\mathbf{v} \in \operatorname{null}(T - \lambda_j I)$. As the decomposition is direct, S is well-defined. Also, $S^2\mathbf{v} = \lambda_j \mathbf{v} = T\mathbf{v}$ for every $\mathbf{v} \in \operatorname{null}(T - \lambda_j I)$. Hence $S^2 = T$. Since each $\sqrt{\lambda_j} \geq 0$, the operator S is self-adjoint with non-negative eigenvalues, so S is positive. Thus T has a positive square root.
Now suppose R is another positive square root of T. Let $\mathbf{v} \in \operatorname{null}(T - \lambda_j I)$. Since $R^2 = T$, then $R^2\mathbf{v} = \lambda_j \mathbf{v}$. As R is positive, its eigenvalues are non-negative, hence the only positive eigenvalue on this eigenspace is $\sqrt{\lambda_j}$. Hence $R\mathbf{v} = \sqrt{\lambda_j}\mathbf{v}$. So R agrees with S on every eigenspace of T. Since these eigenspaces span $\mathcal{V}$, $R = S$. Therefore the positive square root is unique. □

A positive operator acts by nonnegative scalars on its orthogonal eigenspace decomposition. Its positive square root is obtained by replacing each eigenvalue λ with $\sqrt{\lambda}$ which uniquely determines the operator.

Definition 1.2.119. An operator $S \in \mathcal{L}(\mathcal{V})$ is an *isometry* if $\|S\mathbf{v}\| = \|\mathbf{v}\|$ for all $\mathbf{v} \in \mathcal{V}$.
An isometry on a real inner-product space is an *orthogonal operator*.
An isometry on a complex inner-product space is an *unitary operator*.
If $S \in \mathcal{L}(\mathcal{V})$ is an isometry, then S is injective, hence every isometry is invertible.

Theorem 1.2.120. *Suppose $S \in \mathcal{L}(\mathcal{V})$, then the following are equivalent:*

i. S *is an isometry*

ii. $\langle S\mathbf{u}, S\mathbf{v}\rangle = \langle \mathbf{u}, \mathbf{v}\rangle$ *for all* $\mathbf{u}, \mathbf{v} \in \mathcal{V}$ *(inner product is preserved)*

iii. $S^*S = I$

iv. $(S\mathbf{e}_1, \ldots, S\mathbf{e}_n)$ *is orthonormal whenever* $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ *is an orthonormal list of vectors in* $\mathcal{V}$

v. $\exists$ *an orthonormal basis* $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ *of* $\mathcal{V}$ *such that* $(S\mathbf{e}_1, \ldots, S\mathbf{e}_n)$ *is orthonormal*

vi. S^* *is an isometry*

vii. $\langle S^*\mathbf{u}, S^*\mathbf{v}\rangle = \langle \mathbf{u}, \mathbf{v}\rangle$ *for all* $\mathbf{u}, \mathbf{v} \in \mathcal{V}$

viii. $SS^* = I$

ix. $(S^*\mathbf{e}_1, \ldots, S^*\mathbf{e}_n)$ *is orthonormal whenever* $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ *is an orthonormal list of vectors in* $\mathcal{V}$

x. $\exists$ *an orthonormal basis* $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ *of* $\mathcal{V}$ *such that* $(S^*\mathbf{e}_1, \ldots, S^*\mathbf{e}_n)$ *is orthonormal*

Proof.

$(i) \Rightarrow (ii)$ Suppose S is an isometry, $\|S\mathbf{v}\| = \|\mathbf{v}\| \ \forall\ \mathbf{v} \in \mathcal{V}$. Using polarisation identity (real case),

$$\begin{aligned}\langle S\mathbf{u}, S\mathbf{v}\rangle &= \frac{\|S(\mathbf{u}+\mathbf{v})\|^2 - \|S(\mathbf{u}-\mathbf{v})\|^2}{4} \\ &= \frac{\|\mathbf{u}+\mathbf{v}\|^2 - \|\mathbf{u}-\mathbf{v}\|^2}{4} = \langle \mathbf{u}, \mathbf{v}\rangle\end{aligned}$$

as S preserves norms. Hence S also preserves inner products.

$(ii) \Rightarrow (iii)$ For all $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, $\langle (S^*S - I)\mathbf{u}, \mathbf{v}\rangle = \langle S\mathbf{u}, S\mathbf{v}\rangle - \langle \mathbf{u}, \mathbf{v}\rangle = 0$.
Hene $(S^*S - I)\mathbf{u} = 0$ for all $\mathbf{u}$, so $S^*S = I$.

$(iii) \Rightarrow (iv)$ Let $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ be an orthonormal list. Then $\langle S\mathbf{e}_j, S\mathbf{e}_k\rangle = \langle S^*S\mathbf{e}_j, \mathbf{e}_k\rangle = \langle \mathbf{e}_j, \mathbf{e}_k\rangle$.
Hence $(S\mathbf{e}_1, \ldots, S\mathbf{e}_n)$ is orthonormal.

$(iv) \Rightarrow (v)$ Immediate result from proof of $(iii) \Rightarrow (iv)$

$(v) \Rightarrow (i)$ Suppose $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ is an orthonormal basis such that $(S\mathbf{e}_1, \ldots, S\mathbf{e}_n)$ is orthonormal.
Let $\mathbf{v} = \sum_{j=1}^{n} \langle \mathbf{v}, \mathbf{e}_j\rangle \mathbf{e}_j$. Then $S\mathbf{v} = \sum_{j=1}^{n} \langle \mathbf{v}, \mathbf{e}_j\rangle S\mathbf{e}_j$. As the vectors $S\mathbf{e}_j$ are orthonormal,

$$\|S\mathbf{v}\|^2 = \sum_{j=1}^{n} \left|\langle \mathbf{v}, \mathbf{e}_j\rangle\right|^2 = \|\mathbf{v}\|^2$$

Thus S is an isometry.

Replace S with S^* to give the equivalence of $(vi) \Leftrightarrow (vii) \Leftrightarrow (viii) \Leftrightarrow (ix) \Leftrightarrow (x)$.
Finally, because $\mathcal{V}$ is finite-dimensional, $S^*S = I$ implies S is invertible with inverse S^*. Hence $SS^* = I$.
Similarly, $SS^* = I \Rightarrow S^*S = I$. Therefore all ten statements are equivalent. □

An operator is an isometry precisely when it preserves the inner product, equivalently when $S^*S = I$. Thus orthogonal/unitary operators are exactly the linear maps preserving the full geometry of inner-product space.

Theorem 1.2.121. *Suppose $\mathcal{V}$ is a complex inner-product space and $S \in \mathcal{L}(\mathcal{V})$, then S is an isometry if and only if there is an orthonormal basis of $\mathcal{V}$ consisting of eigenvectors of S all of whose corresponding eigenvalues have absolute value 1.*

Proof. Let $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ be an orthonormal basis of $\mathcal{V}$.

$(\Leftarrow)$ Suppose $S\mathbf{e}_j = \lambda_j \mathbf{e}_j$, $|\lambda_j| = 1$ for all j. For any $\mathbf{v} \in \mathcal{V}$, $\mathbf{v} = \sum_{j=1}^{n} \langle \mathbf{v}, \mathbf{e}_j\rangle \mathbf{e}_j$, hence $S\mathbf{v} = \sum_{j=1}^{n} \lambda_j \langle \mathbf{v}, \mathbf{e}_j\rangle \mathbf{e}_j$.

As basis is orthonormal, $\|S\mathbf{v}\|^2 = \sum_{j=1}^{n} |\lambda_j|^2 \left|\langle \mathbf{v}, \mathbf{e}_j\rangle\right|^2 = \sum_{j=1}^{n} \left|\langle \mathbf{v}, \mathbf{e}_j\rangle\right|^2 = \|\mathbf{v}\|^2$. Therefore S is an isometry.

($\Rightarrow$) Suppose S is an isometry.
Since S preserves norms, $\|S\mathbf{v}\| = \|\mathbf{v}\| \ \forall \ \mathbf{v} \in \mathcal{V}$, so S is injective, hence invertible (finite-dimensionality). By Complex Spectral Theorem 1.2.106, S has an orthonormal basis of eigenvectors $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ with $S\mathbf{e}_j = \lambda_j \mathbf{e}_j$. Since S is an isometry, $1 = \|\mathbf{e}_j\| = \|S\mathbf{e}_j\| = \|\lambda_j \mathbf{e}_j\| = |\lambda_j| \, \|\mathbf{e}_j\| = |\lambda_j|$.
Thus every eigenvalue satisfies $|\lambda_j| = 1$.

□

An isometry preserves lengths, so in an orthonormal eigenbasis it can only rotate phases, not rescale magnitudes. Thus the action of S is diagonal with diagonal entries lying on the unit circle.

Theorem 1.2.122. *Suppose $\mathcal{V}$ is a real inner-product space with $S \in \mathcal{L}(\mathcal{V})$. Then S is an isometry if and only if there is an orthonormal basis of $\mathcal{V}$ with respect to which S has a block diagonal matrix where each block on the diagonal is a 1×1 matrix containing 1 or -1, or a 2×2 matrix of form*

$$\begin{bmatrix} \sin\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

with $\theta \in (0, \pi)$.

Proof.

($\Rightarrow$) Suppose S is an isometry with real inner-product space $\mathcal{V}$, then $\langle S\mathbf{v}, S\mathbf{w}\rangle = \langle \mathbf{v}, \mathbf{w}\rangle$, so $S^*S = I$ Hence S is orthogonal, and therefore normal. By Real Spectral Theorem 1.2.109 for normal operators, $\exists$ an orthonormal basis with respect to which S has block diagonal form whose blocks are either 1×1 real eigenvalue blocks (λ), or 2×2 blocks of form

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}, \quad b > 0$$

For a 1×1 block, since S preserves norms, $|\lambda| = 1$, and because $\lambda \in \mathbb{R}$, $\lambda = \pm 1$.
For a 2×2 block, apply S to the first basis vector $\mathbf{e}_j$, hence $S\mathbf{e}_j = a\mathbf{e}_j + b\mathbf{e}_{j+1}$.
As S is an isometry, $1 = \|S\mathbf{e}_j\|^2 = a^2 + b^2$, thus $\exists \ \theta \in (0, \pi)$ such that $a = \cos\theta$, $b = \sin\theta$.

($\Leftarrow$) Suppose S has a block diagonal with respect to some orthonormal basis.
Each 1×1 block (± 1) preserves norms, and each rotation block is orthogonal, hence preserves Euclidean norm on its 2-dim invariant subspace. Because the blocks act on mutually orthogonal subspaces, the entire operator preserves norms on $\mathcal{V}$, therefore S is an isometry.

□

Over $\mathbb{R}$, an isometry cannot always be diagonalised because complex eigenvalues may occur. Every real isometry decomposes orthogonally into independent actions consisting of reflections (± 1) and planar rotations.

Theorem 1.2.123. *Polar Decomposition*
If $T \in \mathcal{L}(\mathcal{V})$, then there exists an isometry $S \in \mathcal{L}(\mathcal{V})$ such that $T = S\sqrt{T^*T}$.

Proof. Let $P = \sqrt{T^*T}$, then P is positive and self-adjoint.
$\forall \ \mathbf{v} \in \mathcal{V}$, $\|T\mathbf{v}\|^2 = \langle T\mathbf{v}, T\mathbf{v}\rangle = \langle T^*T\mathbf{v}, \mathbf{v}\rangle = \langle P^2\mathbf{v}, \mathbf{v}\rangle = \langle P\mathbf{v}, P\mathbf{v}\rangle = \|P\mathbf{v}\|^2$. Hence $\|T\mathbf{v}\| = \|P\mathbf{v}\| \ \forall \ \mathbf{v} \in \mathcal{V}$.
Define $S_1 : \text{range}(P) \to \text{range}(T)$ by $S_1(P\mathbf{v}) = T\mathbf{v}$.
First check this is well-defined. If $P\mathbf{v}_1 = P\mathbf{v}_2$, then $P(\mathbf{v}_1 - \mathbf{v}_2) = 0$, thus $\|T(\mathbf{v}_1 - \mathbf{v}_2)\| = \|P(\mathbf{v}_1 - \mathbf{v}_2)\| = 0$, so $T\mathbf{v}_1 = T\mathbf{v}_2$. Also, $\|S_1(P\mathbf{v})\| = \|T\mathbf{v}\| = \|P\mathbf{v}\|$, so S_1 is an isometry between range(P) and range(T).
Since these subspaces have the same dimension, S_1 extends to an isometry $S \in \mathcal{L}(\mathcal{V})$ by defining it arbitrary as an isometry from range$(P)^\perp$ onto range$(T)^\perp$.
Finally, for every $\mathbf{v} \in \mathcal{V}$, $SP\mathbf{v} = S_1(P\mathbf{v}) = T\mathbf{v}$. Hence $T = SP = S\sqrt{T^*T}$. □

The operator $\sqrt{T^*T}$ contains all metric information of T. It reproduces exactly the same norms as T. Therefore the remaining part of T can only be an isometry, giving the factorisation $T =$ (isometry)(positive operator).

Definition 1.2.124. Suppose $T \in \mathcal{L}(\mathcal{V})$. The *singular values* of T are eigenvalues of $\sqrt{T^*T}$, with each eigenvalue λ repeated $\dim(\text{null}(\sqrt{T^*T} - \lambda I))$ times, all nonnegative as these are eigenvalues of positive operator $\sqrt{T^*T}$.

Theorem 1.2.125. *Singular Value Decomposition*
If $T \in \mathcal{L}(\mathcal{V})$ has singular values $s_1, \ldots, s_n$, then $\exists$ orthonormal bases $(\mathbf{e}_1, \ldots, \mathbf{e}_n)$ and $(\mathbf{f}_1, \ldots, \mathbf{f}_n)$ of $\mathcal{V}$ s.t.

$$T\mathbf{v} = s_1 \langle \mathbf{v}, \mathbf{e}_1\rangle \, \mathbf{f}_1 + \cdots + s_n \langle \mathbf{v}, \mathbf{e}_n\rangle \, \mathbf{f}_n$$

for every $\mathbf{v} \in \mathcal{V}$.

Proof. Let $|T| = \sqrt{T^*T}$. Since $|T|$ is positive and self-adjoint, by Spectral Theorem 1.2.106, $\exists$ an orthonormal basis $(\mathbf{e}_1, \dots, \mathbf{e}_n)$ of $\mathcal{V}$ of $\mathcal{V}$ such that $|T|\,\mathbf{e}_j = s_j \mathbf{e}_j$, where $s_1, \dots, s_n$ are the singular values of T.
Thus $\forall\ \mathbf{v} \in \mathcal{V}$, $|T|\,\mathbf{v} = \sum_{j=1}^{n} s_j \langle \mathbf{v}, \mathbf{e}_j \rangle \mathbf{e}_j$. By Polar Decomposition Theorem 1.2.123, $\exists$ an isometry $S \in \mathcal{L}(\mathcal{V})$ such that $T = S\,|T|$. Therefore $T\mathbf{v} = S(|T|\,\mathbf{v}) = \sum_{j=1}^{n} s_j \langle \mathbf{v}, \mathbf{e}_j \rangle S\mathbf{e}_j$.

Define $\mathbf{f}_j = S\mathbf{e}_j$. As S is an isometry, $(\mathbf{f}_1, \dots \mathbf{f}_n)$ is an orthonormal basis of $\mathcal{V}$. Hence $T\mathbf{v} = \sum_{j=1}^{n} s_j \langle \mathbf{v}, \mathbf{e}_j \rangle \mathbf{f}_j$ for every $\mathbf{v} \in \mathcal{V}$, completing the proof. □

The SVD decomposes T into a positive stretching operator $\sqrt{T^*T}$ and an isometric rotation/reflection. Diagonalising the stretching part yields the singular values and orthogonal directions.

1.2.5 Operators on Complex Vector Spaces

Definition 1.2.126. Given $T \in \mathcal{L}(\mathcal{V})$, and λ an eigenvalue of T, a vector $\mathbf{v} \in \mathcal{V}$ is a *generalised eigenvector* of T corresponding to λ if $(T - \lambda I)^j \mathbf{v} = \mathbf{0}$ for some positive integer j.

Suppose $T \in \mathcal{L}(\mathcal{V})$ and $k \geq 0$, $k \in \mathbb{Z}$. If $T^k\mathbf{v} = \mathbf{0}$, then $T^{k+1}\mathbf{v} = T(T^k\mathbf{v}) = T(\mathbf{0}) = \mathbf{0}$, hence,

$$\{\mathbf{0}\} = \text{null}(T^0) \subset \text{null}(T^1) \subset \cdots \subset \text{null}(T^k) \subset \cdots$$

Proposition 1.2.127. *If $T \in \mathcal{L}(\mathcal{V})$ and $m \geq 0$, $m \in \mathbb{Z}$ such that $null(T^m) = null(T^{m+1})$, then*

$$null(T^0) \subset null(T^1) \subset \cdots \subset null(T^m) = null(T^{m+1}) = null(T^{m+2}) = \cdots$$

Proof. Let $T \in \mathcal{L}(\mathcal{V})$ and suppose $\text{null}(T^m) = \text{null}(T^{m+1})$.
Since $\text{null}(T^j) \subseteq \text{null}(T^{j+1})$ for every $j \geq 0$, it suffices to prove $\text{null}(T^{m+k+1}) \subseteq \text{null}(T^{m+k})$ for all $k \geq 0$.
Take $\mathbf{v} \in \text{null}(T^{m+k+1})$, then $0 = T^{m+k+1}\mathbf{v} = T^{m+1}(T^k\mathbf{v})$, so $T^k\mathbf{v} \in \text{null}(T^{m+1}) = \text{null}(T^m)$.
Hence $0 = T^m(T^k\mathbf{v}) = T^{m+k}\mathbf{v}$, which shows $\mathbf{v} \in \text{null}(T^{m+k})$, therefore $\text{null}(T^{m+k+1}) \subseteq \text{null}(T^{m+k})$.
Combine with reverse inclusion to get $\text{null}(T^{m+k}) = \text{null}(T^{m+k+1})$ for all $k \geq 0$. Thus the chain stabilises. □

The kernels of powers of T form an increasing chain. Once two consecutive kernels coincide, no new vectors can enter later kernels, because any vector killed by a higher power would already produce a vector in the stabilised kernel after applying a suitable power of T.

Proposition 1.2.128. *If $T \in \mathcal{L}(\mathcal{V})$, then*

$$null(T^{dim(\mathcal{V})}) = null(T^{dim(\mathcal{V}+1)}) = null(T^{dim(\mathcal{V}+2)}) = \cdots$$

Proof. Let $n = \dim(\mathcal{V})$. Suppose for contradiction that $\text{null}(T^n) \neq \text{null}(T^{n+1})$. By Proposition 1.2.127,

$$\{\mathbf{0}\} = \text{null}(T^0) \subsetneq \text{null}(T^1) \subsetneq \cdots \subsetneq \text{null}(T^n) \subsetneq \text{null}(T^{n+1})$$

is strictly increasing. Each strict inclusion increases dimension by at least 1.
Since $\dim(\text{null}(T^0)) = 0$, it follows that $\dim(\text{null}(T^{n+1})) \geq n + 1$. But $\text{null}(T^{n+1})$ is a subspace of $\mathcal{V}$, so its dimension cannot exceed $n = \dim(\mathcal{V})$, a contradiction.
Hence $\text{null}(T^n) = \text{null}(T^{n+1})$. Applying Proposition 1.2.127 again yields $\text{null}(T^n) = \text{null}(T^{n+1}) = \cdots$. □

The kernels $\text{null}(T), \text{null}(T^2), \dots$ form an increasing chain of subspaces. In a finite-dimensional space, such a chain cannot strictly increase more than $\dim(\mathcal{V})$ times, so the sequence must eventually stabilise.

Corollary 1.2.129. *Suppose $T \in \mathcal{L}(\mathcal{V})$ and λ is an eigenvalue of T, then the set of generalised eigenvectors of T corresponding to λ equals $null(T - \lambda I)^{dim(\mathcal{V})}$.*

Proof.

(⇒) Let $n = \dim(\mathcal{V})$. If $\mathbf{v} \in \text{null}((T - \lambda I)^n)$, then $\mathbf{v}$ is a generalised eigenvector corresponding to λ.

(⇐) Suppose $\mathbf{v}$ is a generalised eigenvector corresponding to λ, then $\exists\ j \geq 0$ such that $\mathbf{v} \in \text{null}((T - \lambda I)^j)$. By Proposition 1.2.128, $\text{null}(T - \lambda I) \subseteq \text{null}((T - \lambda I)^2) \subseteq \cdots$ is an increasing sequence of subspaces. Proposition 1.2.127, applied to $T - \lambda I$, implies that $\text{null}((T - \lambda I)^n) = \text{null}((T - \lambda I)^{n+1}) = \cdots$, hence $\mathbf{v} \in \text{null}((T - \lambda I)^n)$. Therefore the set of generalised eigenvectors corresponding to λ equals $\text{null}((T - \lambda I)^n)$. □

The kernels of successive powers of $T - \lambda I$ form an increasing chain of subspaces. In finite dimensions, this must stabilise by $\exp(\dim(\mathcal{V}))$, so every generalised eigenvector is already annihilated by the $\dim(\mathcal{V})$-th power.

Definition 1.2.130. An operator is *nilpotent* if some power of it equals 0.

Corollary 1.2.131. *Suppose $N \in \mathcal{L}(\mathcal{V})$ is nilpotent, then $N^{dim(\mathcal{V})} = 0$.*

Proof. Let $n = \dim(\mathcal{V})$. As N is nilpotent, for every $\mathbf{v} \in \mathcal{V}$ $\exists\ j > 0$ such that $N^j\mathbf{v} = 0$. Thus every vector in $\mathcal{V}$ is a generalised eigenvector corresponding to the eigenvalue 0.
By Corollary 1.2.129, $\text{null}(N^n) = \mathcal{V}$, therefore $N^n = 0$ □

A nilpotent operator eventually kills every vector. In finite dimensions, orollary 1.2.129 shows that exponent $\dim(\mathcal{V})$ is already sufficient to annihilate the entire space.

Proposition 1.2.132. *If $T \in \mathcal{L}(\mathcal{V})$, then $range(T^{dim(\mathcal{V})}) = range(T^{dim(\mathcal{V}+1)}) = range(T^{dim(\mathcal{V})+2}) = \cdots$*

Proof. Let $n = \dim(\mathcal{V})$, and suppose $m > n$. By Proposition 1.2.128, $\dim(T^m) = \dim(T^n)$.
Hence by Rank-Nullity Theorem 1.2.32,

$$\dim(\text{range}(T^m)) = \dim(\mathcal{V}) - \dim(\text{null}(T^m)) = \dim(\mathcal{V}) - \dim(\text{null}(T^n)) = \dim(\text{range}(T^n))$$

Also $\text{range}(T^n) \subseteq \text{range}(T^m)$. Thus $\text{range}(T^m) = \text{range}(T^{m+1}) = \text{range}(T^{m+2}) = \cdots$. □

The kernels of T^k eventually stabilise, so rank–nullity forces the dimensions of the ranges to stabilise as well. Since the ranges already form a decreasing chain of subspaces, equal dimensions imply the ranges themselves must eventually stop shrinking.

Theorem 1.2.133. *Let $T \in \mathcal{L}(\mathcal{V})$ and $\lambda \in \mathbb{F}$, then for every basis of $\mathcal{V}$ with respect to which T has an upper-triangular matrix, λ appears on diagonal of matrix of T precisely $\dim(null(T - \lambda I))^{\dim(\mathcal{V})}$ times.*

Proof. Let $n = \dim(\mathcal{V})$. Replacing T by $T - \lambda I$ shifts every diagonal entry by $-\lambda$, so it suffices to prove the theorem for $\lambda = 0$. Proceed by induction on n. Result is trivial for $n = 1$.
Assume $n > 1$, and let $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ be a basis with respect to which T has upper-triangular matrix

$$\mathcal{M}(T) = \begin{bmatrix} \lambda_1 & * & \cdots & * \\ 0 & \lambda_2 & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

Let $\mathcal{U} = \text{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{n-1})$. Then $\mathcal{U}$ is invariant under T, and $T|_{\mathcal{U}}$ has upper-triangular matrix with diagonal entries $\lambda_1, \ldots, \lambda_{n-1}$. By induction hypothesis, number of zeroes among $\lambda_1, \ldots, \lambda_{n-1}$ equals $\dim(\text{null}((T|_{\mathcal{U}})^n))$. Now compare this with $\text{null}(T^n)$.

Case 1 : $\lambda_n \neq 0$. Since powers of upper-triangular matrix remain upper triangular, $\mathcal{M}(T^n)$ has diagonal entries $\lambda_1^n, \ldots, \lambda_n^n$. In particular, $T^n\mathbf{v}_n = \mathbf{u} + \lambda_n^n\mathbf{v}_n$ for some $\mathbf{u} \in \mathcal{U}$.
Take $\mathbf{v} \in \text{null}(T^n)$ and write $\mathbf{v} = \tilde{\mathbf{u}} + a\mathbf{v}_n$, $\tilde{\mathbf{u}} \in \mathcal{U}$, then $0 = T^n\mathbf{v} = T^n\tilde{\mathbf{u}} + aT^n\mathbf{v}_n = T^n\tilde{\mathbf{u}} + a\mathbf{u} + a\lambda_n^n\mathbf{v}_n$.
Because $T^n\tilde{\mathbf{u}}, a\mathbf{u} \in \mathcal{U}$ and $\mathbf{v}_n \notin \mathcal{U}$, thus $a\lambda_n^n = 0$. Since $\lambda_n \neq 0$, it follows that $a = 0$. Hence $\text{null}(T^n) \subseteq \mathcal{U}$.
Therefore $\text{null}(T^n) = \text{null}((T|_{\mathcal{U}})^n)$, so the number of zeroes on the diagonal equals $\dim(\text{null}(T^n))$.

Case 2 : $\lambda = 0$. Then $T\mathbf{v}_n \in \mathcal{U}$, so $T^n\mathbf{v}_n = T^{n-1}(T\mathbf{v}_n) \in \text{range}((T|_{\mathcal{U}})^{n-1})$.
By Proposition 1.2.132, $\text{range}((T|_{\mathcal{U}})^{n-1}) = \text{range}((T|_{\mathcal{U}})^n)$. Hence $\exists\ \mathbf{u} \in \mathcal{U}$ such that $T^n\mathbf{u} = T^n\mathbf{v}_n$.
Thus $\mathbf{u} - \mathbf{u}_n \in \text{null}(T^n)$. Because $\mathbf{v}_n \notin \mathcal{U}$, this vector is not in $\mathcal{U}$.
Therefore $\dim(\text{null}(T^n)) = \dim(\text{null}((T|_{\mathcal{U}})^n)) + 1$. Since $\lambda_n = 0$, the number of zeroes on the diagonal also increases by 1. Hence the number of zeroes on the diagonal equals $\dim(\text{null}(T^n))$.

This proofs the theorem for $\lambda = 0$, and therefore for arbitrary λ by replacing T with $T - \lambda I$. □

For an upper-triangular operator, the generalised eigenspace for λ is detected entirely by how many times λ appears on the diagonal. The proof inductively peels off the last basis vector and compares how the nullspace of T^n changes depending on whether the final diagonal entry is zero.

Definition 1.2.134. Suppose $T \in \mathcal{L}(\mathcal{V})$. The *multiplicity* of eigenvalue λ in T is defined to be dimension of subspace of generalised eigenvectors corresponding to λ, which is $\dim(\text{null}(T - \lambda I)^{\dim(\mathcal{V})})$.

Proposition 1.2.135. *If $\mathcal{V}$ is a complex vector space and $T \in \mathcal{L}(\mathcal{V})$, then the sum of multiplicities of all eigenvalues of T equals $dim(\mathcal{V})$.*

Proof. Let $n = \dim(\mathcal{V})$. As $\mathcal{V}$ is a complex vector space, $\exists$ a basis of $\mathcal{V}$ with respect to which T is upper-triangular

$$\mathcal{M}(T) = \begin{bmatrix} \lambda_1 & * & \cdots & * \\ 0 & \lambda_2 & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

The eigenvalues of T are precisely the diagonal entries $\lambda_1, \ldots, \lambda_n$. By Theorem 1.2.133, the multiplicity of an eigenvalue λ equals the number of times λ appears on the diagonal. Hence the sum of the multiplicities of all eigenvalues equals the total number of diagonal entries (n).
Therefore the sum of the multiplicities of all eigenvalues of T equals $\dim(\mathcal{V})$. □

Over $\mathbb{C}$, every operator can be triangularised, so all eigenvalues appear on the diagonal of an upper-triangular matrix. Counting diagonal entries with repetition therefore counts eigenvalues with their algebraic multiplicities.

Definition 1.2.136. Let $\mathcal{V}$ be complex vector space, and $T \in \mathcal{L}(\mathcal{V})$. Let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T, d_j be multiplicity of λ_j as an eigenvalue of T. The *characteristic polynomial* of T is $(z-\lambda_1)^{d_1} \cdots (z-\lambda_m)^{d_m}$.

Theorem 1.2.137. *Cayley-Hamilton Theorem*
Suppose $\mathcal{V}$ is complex vector space and $T \in \mathcal{L}(\mathcal{V})$, and q is characteristic polynomial of T. Then $q(T) = 0$.

Proof. Let $n = \dim(\mathcal{V})$, and choose a basis $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ with respect to which T is upper-triangular,

$$\mathcal{M}(T) = \begin{bmatrix} \lambda_1 & * & \cdots & * \\ 0 & \lambda_2 & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

Then the characteristic polynomial of T is $q(x) = (x-\lambda_1)\cdots(x-\lambda_n)$.
It suffices to show $q(T)\mathbf{v}_j = 0$ for $j = 1, \ldots, n$, because $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ is a basis of $\mathcal{V}$.
Proof by induction on j that $(T-\lambda_1 I)\cdots(T-\lambda_j I)\mathbf{v}_j = 0$.
For $j = 1$, $T\mathbf{v}_1 = \lambda_1\mathbf{v}_1$, so $(T-\lambda_1 I)\mathbf{v}_1 = 0$.
Suppose $j > 1$ and claim holds for $\mathbf{v}_1, \ldots, \mathbf{v}_{j-1}$. As T is upper-triangular, $(T-\lambda_j I)\mathbf{v}_j \in \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_{j-1})$.
Applying $(T-\lambda_1 I)\cdots(T-\lambda_{j-1} I)$ to this vector gives 0 by induction hypothesis.
Hence $(T-\lambda_1 I)\cdots(T-\lambda_j I)\mathbf{v}_j = 0$. Therefore $q(T)\mathbf{v}_j = 0$ for every j, so $q(T) = 0$. □

In an upper-triangular basis, each operator $(T-\lambda_j I)$ lowers vectors one step in the invariant flag generated by the basis vectors. Applying all factors successively eventually kills every basis vector, forcing $q(T) = 0$.

Proposition 1.2.138. *If $T \in \mathcal{L}(\mathcal{V})$ and $p \in \mathcal{P}(\mathbb{F})$, then null$(p(T))$ is invariant under T.*

Proof. Let $\mathbf{v} \in \operatorname{null}(p(T))$, then $p(T)\mathbf{v} = 0$. As $p(T)$ is a polynomial in T, it commutes with T. Hence,

$$p(T)(T\mathbf{v}) = T(p(T)\mathbf{v}) = T(0) = 0$$

Therefore $T\mathbf{v} \in \operatorname{null}(p(T))$. Thus $\operatorname{null}(p(T))$ is invariant under T. □

Any polynomial in T commutes with T. Therefore, if a vector is annihilated by $p(T)$, applying T to that vector preserves the annihilation condition, making the nullspace invariant.

Theorem 1.2.139. *Major Structure Theorem / Primary Decomposition Theorem*
Suppose $\mathcal{V}$ is a complex vector space and $T \in \mathcal{L}(\mathcal{V})$. Let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T, and let $\mathcal{U}_1, \ldots, \mathcal{U}_m$ be corresponding subspaces of generalised eigenvalues. Then

i. $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$
ii. *each $\mathcal{U}_j$ is invariant under T*
iii. *each $(T-\lambda_j I)|_{\mathcal{U}_j}$ is nilpotent*

Proof. Let $n = \dim(\mathcal{V})$.

(i) For each j, let $\mathcal{U}_j$ denote the generalised eigenspace corresponding to λ_j.
By Theorem 1.2.133, the multiplicity of λ_j equals $\dim(\mathcal{U}_j)$. By Proposition 1.2.135, the sum of multiplicity of all distinct eigenvalues equals n, hence $n = \dim(\mathcal{U}_1) + \cdots + \dim(\mathcal{U}_m))$.
Let $\mathcal{U} = \mathcal{U}_1 + \cdots + \mathcal{U}_m$. Then $\mathcal{U}$ is invariant under T. Consider the restriction $S = T|_{\mathcal{U}}$. Every generalised

eigenvector of T lies in $\mathcal{U}$, so the generalised eigenspaces of S are precisely $\mathcal{U}_1, \ldots, \mathcal{U}_m$.
Therefore the eigenvalues of S are $\lambda_1, \ldots, \lambda_m$ with multiplicities $\dim(\mathcal{U}_1), \ldots, \dim(\mathcal{U}_m)$.
Applying Proposition 1.2.135 to S, $\dim(\mathcal{U}) = \dim(\mathcal{U}_1) + \cdots + \dim(\mathcal{U}_m) = n$.
Since $\mathcal{U} \subseteq \mathcal{V}$ and $\dim(\mathcal{U}) = \dim(\mathcal{V})$, hence $\mathcal{V} = \mathcal{U}_1 + \cdots + \mathcal{U}_m$.
As $\dim(\mathcal{V}) = \dim(\mathcal{U}_1) + \cdots + \dim(\mathcal{U}_m)$, the sum is direct. Hence $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$.

(*ii*) By Corollary 1.2.129, $\mathcal{U}_j = \text{null}((T - \lambda_j I)^n)$. Since T commutes with $T - \lambda_j I$, Proposition 1.2.138 implies that $\mathcal{U}_j$ is invariant under T.

(*iii*) On $\mathcal{U}_j$, $(T - \lambda_j I)^n = 0$ because every vector in $\mathcal{U}_j$ lies in $\text{null}((T - \lambda_j I)^n)$. Hence $(T - \lambda_j I)|_{\mathcal{U}_j}$ is nilpotent.

□

The generalised eigenspaces capture all vectors annihilated by powers of $T - \lambda I$. Over $\mathbb{C}$, these generalised eigenspaces partition the entire space into invariant pieces where T behaves as eigenvalue + nilpotent part.

Corollary 1.2.140. *Suppose $\mathcal{V}$ is a complex vector space and $T \in \mathcal{L}(\mathcal{V})$. Then there is a basis of $\mathcal{V}$ consisting of generalised eigenvectors of T.*

Proof. Let $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$ be decomposition from Theorem 1.2.139, where each $\mathcal{U}_j$ is the generalised eigenspace corresponding to each λ_j. Choose a basis for each $\mathcal{U}_j$. Since the sum is direct, the union of these bases form a basis of $\mathcal{V}$. Every vector in each $\mathcal{U}_j$ is a generalised eigenvector of T, so this basis consists entirely of generalised eigenvectors of T. □

The Primary Decomposition Theorem splits the whole space into generalised eigenspaces. Taking bases inside each piece automatically produces a basis of the entire space consisting entirely of generalised eigenvectors.

Lemma 1.2.141. *Suppose N is nilpotent operator on $\mathcal{V}$, then $\exists$ a basis of $\mathcal{V}$ w.r.t. which matrix of N has form*

$$\begin{bmatrix} 0 & & * \\ & \ddots & \\ 0 & & 0 \end{bmatrix}$$

where all entries on and below diagonal are 0s.

Proof. Since N is nilpotent, $\exists\ m \geq 1$ s.t. $N^m = 0$. Hence $\{0\} \subseteq \text{null}(N) \subseteq \text{null}(N^2) \subseteq \cdots \subseteq \text{null}(N^m) = \mathcal{V}$.
Construct a basis of $\mathcal{V}$ adapted to this chain. First choose a basis of $\text{null}(N)$, then extend it to a basis of $\text{null}(N^2)$, then to $\text{null}(N^3)$ and so on until obtaining a basis of $\mathcal{V}$.
Let $\mathbf{v}$ be a basis vector chosen from $\text{null}(N^k) \backslash \text{null}(N^{k-1})$. Then $N\mathbf{v} \in \text{null}(N^{k-1})$ as $N^{k-1}(N\mathbf{v}) = N^k\mathbf{v} = 0$.
By construction of the basis, $\text{null}(N^{k-1})$ is spanned by basis vectors appearing earlier in the ordering. Thus $N\mathbf{v}$ is a linear combination of earlier basis vectors.
Therefore, in the matrix of N with respect to this basis, every column has nonzero entires only above the diagonal. Hence the matrix of N is strictly upper triangular. □

The kernels $\text{null}(N) \subseteq \text{null}(N^2) \subseteq \cdots$ form a natural filtration for a nilpotent operator. Choosing a basis adapted to this filtration forces each vector to map into the span of earlier basis vectors, producing a strictly upper-triangular matrix.

Theorem 1.2.142. *Triangular Form Theorem for Linear Operators over $\mathbb{C}$*
Suppose $\mathcal{V}$ is a complex vector space and $T \in \mathcal{L}(\mathcal{V})$. Let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T. Then there exists a basis of $\mathcal{V}$ with respect to which the matrix of T is block diagonal of form

$$\begin{bmatrix} A_1 & & 0 \\ & \ddots & \\ 0 & & A_m \end{bmatrix}$$

where each A_j is an upper triangular matrix of form

$$A_j = \begin{bmatrix} \lambda_j & & * \\ & \ddots & \\ 0 & & \lambda_j \end{bmatrix}$$

Proof. For $j = 1, \ldots, m$, let $\mathcal{U})_j$ denote the generalised eigenspace corresponding to λ_j. By Major Structure Theorem 1.2.139, $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$ and $(T - \lambda_j I)|_{\mathcal{U}_j}$ is nilpotent.

By Lemma 1.2.141, for each j, $\exists$ a basis of $\mathcal{U}_j$ with respect to which $(T-\lambda_j I)|_{\mathcal{U}_j}$ has a strictly upper-triangular matrix. Therefore the matrix $T|_{\mathcal{U}_j} = \lambda_j I + (T-\lambda_j I)|_{\mathcal{U}_j}$ has the form

$$A_j = \begin{bmatrix} \lambda_j & * & \cdots & * \\ 0 & \lambda_j & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_j \end{bmatrix}$$

Combining the chosen bases of $\mathcal{U}_1, \ldots, \mathcal{U}_m$ gives a basis of $\mathcal{V}$. With respect to this basis, the matrix of T is block diagonal with blocks $A_1, \ldots, A_m$. □

Each generalised eigenspace reduces T to an eigenvalue plus a nilpotent operator. Choosing a basis adapted to the nilpotent part yields an upper-triangular block with that eigenvalue on the diagonal.

Lemma 1.2.143. *Suppose $N \in \mathcal{L}(\mathcal{V})$ is nilpotent, then $I+N$ has a square root.*

Proof. Let $m \geq 1$ such that $N^m = 0$. Seek square root of $I+N$ in form $S = I + a_1 N + a_2 N^2 + \cdots + a_{m-1} N^{m-1}$, where $a_1, \ldots, a_{m-1} \in \mathbb{C}$. Expanding S^2,

$$S^2 = \sum_{k=0}^{m-1} \left(\sum_{r=0}^{k} a_r a_{k-r} \right) N^k$$

where $a_0 = 1$, and all powers N^k with $k \geq m$ vanish because $N^m = 0$.
Seek $S^2 = I + N$, so coefficients must satisfy

$$\sum_{r=0}^{k} a_r a_{k-r} = \begin{cases} 1, & k = 0, 1 \\ 0, & 2 \leq k \leq m-1 \end{cases}$$

For $k \geq 1$, $\sum_{r=0}^{k} a_r a_{k-r} = s a_k + \sum_{r=1}^{k-1} a_r a_{k-r}$, hence the recursive equation is $2a_1 = 1$.
For $2 \leq k \leq m-1$, $2a_k + \sum_{r=1}^{k-1} a_r a_{k-r} = 0$.
These equations uniquely determine $a_1, \ldots, a_{m-1}$, hence $S^2 = I + N$, and $I+N$ has a square root. □

As N is nilpotent, every power series in N truncates to a finite polynomial. Thus one can construct $\sqrt{I+N}$ exactly by solving recursively for polynomial coefficients.

Theorem 1.2.144. *Let $\mathcal{V}$ be a complex vector space and $T \in \mathcal{L}(\mathcal{V})$. If T is invertible, then T has a square root.*

Proof. Let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T, and let $\mathcal{U}_j$ be the corresponding generalised eigenspaces.
By Major Structure Theorem 1.2.139, $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$, and for each j, $T|_{\mathcal{U}_j} = \lambda_j I + N_j$ where N_j is nilpotent.
As T is invertible, 0 is not an eigenvalue, hence each $\lambda_j \neq 0$, thus

$$T|_{\mathcal{U}_j} = \lambda_j \left(I + \frac{N_j}{\lambda_j} \right)$$

Since N_j is nilpotent, so is N_j/λ_j, hence by Lemma 1.2.143, there exists $R_j \in \mathcal{L}(\mathcal{U}_j)$ such that $R^2 = I + \frac{N_j}{\lambda_j}$.
Chooseing $\alpha_j \in \mathbb{C}$ satisfying $\alpha_j^2 = \lambda_j$, and define $S_j = \alpha_j R_j$, then

$$S_j^2 = \alpha_j^2 R_j^2 = \lambda_j \left(I + \frac{N_j}{\lambda_j} \right) = T|_{\mathcal{U}_j}$$

Now define $S \in \mathcal{L}(\mathcal{V})$ by $S(\mathbf{u}_1 + \cdots + \mathbf{u}_m) = S_1 \mathbf{u}_1 + \cdots + S_m \mathbf{u}_m$, $\mathbf{u}_j \in \mathcal{U}_j$. This is well-defined as $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$.
For each $\mathbf{u}_j \in \mathcal{U}_j$, $S^2(\mathbf{u}_1 + \cdots + \mathbf{u}_m) = S_1^2 \mathbf{u}_1 + \cdots + S_m^2 \mathbf{u}_m = T\mathbf{u}_1 + \cdots + T\mathbf{u}_m = T(\mathbf{u}_1 + \cdots + \mathbf{u}_m)$.
Hence $S^2 = T$, and therefore T has a square root. □

On each generalised eigenspace, T is a nonzero scalar plus a nilpotent operator. Taking square roots of these pieces block-wise yields a square root of T.

Definition 1.2.145. A *monic polynomial* is a non-zero univariate polynomial in which the leading coefficient equals 1, i.e., $x^n + c_{n-1}x^{n-1} + \cdots + c_2 x^2 + c_1 x + x_0$ with $n \geq 0$.

Definition 1.2.146. A *minimal polynomial* of T is monic polynomial $\mathbf{p} \in \mathcal{P}(\mathbb{F})$ of smallest degree s.t. $\mathbf{p}(T) = 0$.

Theorem 1.2.147. *Let $T \in \mathcal{L}(\mathcal{V})$ and let $\mathbf{q} \in \mathcal{P}(\mathbb{F})$, then $\mathbf{q}(T) = 0$ if and only if minimal polynomial of T divides $\mathbf{q}$.*

Proof. Let p be the minimal polynomial of T.

($\Rightarrow$) Suppose $q(T) = 0$. By division algorithm, $\exists$ polynomials $s, r \in \mathcal{P}(\mathbb{F})$ such that $q = sp + r$, $\deg(r) < \deg(p)$. Evaluating at T, $0 = q(T) = s(T)p(T) + r(T) = r(T)$ because $q(T) = 0$.
Since p is the minimal polynomial of T, no nonzero polynomial of degree smaller than $\deg(p)$ annihilates T. Hence $r = 0$. Therefore $q = sp$, so $p \mid q$.

($\Leftarrow$) Suppose $p \mid q$. Then $q = sp$ for some polynomial s. Evaluating at T, $q(T) = s(T)p(T) = s(T)0 = 0$.

Therefore $q(T) = 0 \Leftrightarrow p \mid q$. □

The minimal polynomial is the monic polynomial of smallest degree annihilating T. Any other annihilating polynomial must therefore contain it as a factor, which follows directly from polynomial division.

Theorem 1.2.148. *Let $T \in \mathcal{L}(\mathcal{V})$, then the roots of minimal polynomial of T are precisely eigenvalues of T.*

Proof. Let p be the minimal polynomial of T.

($\Rightarrow$) Suppose $\lambda \in \mathbb{F}$ is the root of p. Then $p(z) = (z - \lambda)q(z)$ for some polynomial q.
Since $p(T) = 0$, then $0 = p(T) = (T - \lambda I)q(T)$. Because $\deg(q) < \deg(p)$, the polynomial q cannot annihilate T; otherwise this will contract the minimality of p. Hence $\exists\ \mathbf{v} \in \mathcal{V}$ s.t. $q(T)\mathbf{v} \neq 0$.
Let $\mathbf{u} = q(T)\mathbf{v}$. Then $(T - \lambda I)\mathbf{u} = 0$, so $T\mathbf{u} = \lambda\mathbf{u}$. Because $\mathbf{u} \neq \mathbf{0}$, it follows that λ is an eigenvalue of T.

($\Leftarrow$) Suppose λ is an eigenvalue of T. Let $\mathbf{v} \neq \mathbf{0}$ satisfy $T\mathbf{v} = \lambda\mathbf{v}$.
Then for every non-negative integer j, $T^j\mathbf{v} = \lambda^j\mathbf{v}$. Let $p(z) = a_0 + a_1 z + \cdots + a_m z^m$.
Since $p(T) = 0$, then $0 = p(T) = (a_0 + a_1 T + \cdots + a_m T^m)\mathbf{v} = (a_0 + a_1\lambda + \cdots + a_m\lambda^m)\mathbf{v} = p(\lambda)\mathbf{v}$.
As $\mathbf{v} \neq \mathbf{0}$, $p(\lambda) = 0$. Therefore the roots of minimal polynomial of T are precisely the eigenvalues of T.

□

A root λ of the minimal polynomial produces a nonzero vector annihilated by $T - \lambda I$, hence an eigenvector. Conversely, eigenvectors reduce polynomial evaluation at T to ordinary scalar evaluation at the eigenvalue.

Remark 1.2.149. *Computing Minimal Polynomial*
Let $T \in \mathcal{L}(\mathcal{V})$, and let $A = (\mathcal{M}(T)$ be the matrix of T with respect to some basis.
Consider the sequence $I, A, A^2, A^3, \ldots$. Since the vector space of $n \times n$ matrices is finite-dimensional, this sequence must eventually become linearly dependent. Let $m \geq 1$ be smallest integer such that $I, A, A^2, A^3, \ldots, A^m$ are linearly dependent. Then $\exists$ scalars $a_0, \ldots, a_{m-1} \in \mathbb{F}$ such that $a_0 I + a_1 A + \cdots + a_{m-1}A^{m-1} + A^m = 0$.
Equivalently, $(A^m + a_{m-1}A^{m-1} + \cdots + a_1 A + a_0 I) = 0$.
Hence the monic polynomial $p(z) = a_0 + a_1 z + \cdots + a_{m-1}z^{m-1} + z^m$ satisfies $p(T) = 0$.
By minimality of m, this polynomial is the minimal polynomial of T.

The powers of T cannot remain linearly independent forever in a finite-dimensional matrix space. The first linear dependence relation among $I, T, T^2, \ldots$ products the minimal polynomial.

Lemma 1.2.150. *Let $N \in \mathcal{L}(\mathcal{V})$ be nilpotent. Then $\exists$ vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k \in \mathcal{V}$ such that*

i. $(\mathbf{v}_1, N\mathbf{v}_1, \ldots, N^{m(\mathbf{v}_1)}\mathbf{v}_1, \ldots, \mathbf{v}_k, N\mathbf{v}_k, \ldots, N^{m(\mathbf{v}_k)}\mathbf{v}_k)$ *is a basis of $\mathcal{V}$.*

ii. $(N^{m(\mathbf{v}_1)}\mathbf{v}_1, \ldots, N^{m(\mathbf{v}_k)}\mathbf{v}_k)$ *is a basis of null(N).*

Proof. Proceed by induction on $\dim(\mathcal{V})$. If $\dim(\mathcal{V}) = 0$, the result is trivial.
Assume result holds for all nilpotent operators on smaller dimension cases, and let N be nilpotent on $\mathcal{V}$.
Since N is nilpotent, it is not injective, so $\dim(\text{range}(N)) < \dim(\mathcal{V})$. The restriction $N|_{\text{range}(N)}$ is nilpotent.
By induction, $\exists$ vectors $\mathbf{u}_1, \ldots, \mathbf{u}_j \in \text{range}(N)$ s.t. $(\mathbf{u}_1, N\mathbf{u}_1, \ldots, N^{m(\mathbf{u}_1)}\mathbf{u}_1, \ldots, \mathbf{u}_j, N\mathbf{u}_j, \ldots, N^{m(\mathbf{u}_j)}\mathbf{u}_j)$ is a basis of range(N), and $(N^{m(\mathbf{u}_1)}\mathbf{u}_1, \ldots, N^{m(\mathbf{u}_j)}\mathbf{u}_j)$ is a basis of null$(N) \cap$ range(N).
For each $r = 1, \ldots, j$, choose $\mathbf{v}_r \in \mathcal{V}$ such that $N\mathbf{v}_r = \mathbf{u}_r$. Then $N^s\mathbf{v}_r = N^{s-1}\mathbf{u}_r$ so each chain for $\mathbf{u}_r$ extends upward by one vector. Now choose a subspace $\mathcal{W}$ such that null$(N) =$ (null$(N) \cap$ range$(N)) \oplus \mathcal{W}$, and let $\mathbf{v}_{j+1}, \ldots, \mathbf{v}_k$ be a basis of $\mathcal{W}$. For these vectors, set $m(\mathbf{v}_s) = 0$.

(i) Claim that $(\mathbf{v}_1, N\mathbf{v}_1, \ldots, N^{m(\mathbf{v}_1)}\mathbf{v}_1, \ldots, \mathbf{v}_k, N\mathbf{v}_k, \ldots, N^{m(\mathbf{v}_k)}\mathbf{v}_k)$ is a basis of $\mathcal{V}$.
Linear independence follows as applying N to a linear relation reduces it to a relation among inductively constructed basis vectors in range(N). Remaining terms lie in (null$(N) \cap$ range$(N)) \oplus \mathcal{W}$, forcing all coefficients to vanish. The number of vectors listed equals $\dim(\text{range}(N)) + \dim(\text{null}(N)) = \dim(\mathcal{V})$ by Rank-Nullity Theorem 1.2.32. Hence the list is a basis of $\mathcal{V}$.

(*ii*) The terminal vectors in the chains are

$$(N^{m(\mathbf{v}_1)}\mathbf{v}_1, \ldots, N^{m(\mathbf{v}_k)}\mathbf{v}_k) = (N^{m(\mathbf{u}_1)}\mathbf{u}_1, \ldots, N^{m(\mathbf{u}_j)}\mathbf{u}_j, \mathbf{v}_{j+1}, \ldots, \mathbf{v}_k)$$

The first group forms a basis of $\text{null}(N) \cap \text{range}(N)$, while $\mathbf{v}_{j+1}, \ldots, \mathbf{v}_k$ is a basis of $\mathcal{W}$.
Therefore these vectors form a basis of $\text{null}(N)$.

□

A nilpotent operator naturally organises vectors into chains $\mathbf{v}, N\mathbf{v}, N^2\mathbf{v}, \ldots$. The proof builds a basis of $\mathcal{V}$ from such chains by first constructing chains inside $\text{range}(N)$, then lifting them one level upward and completing with vectors from $\text{null}(N)$.

Definition 1.2.151. Suppose $T \in \mathcal{L}(\mathcal{V})$. A basis of $\mathcal{V}$ is a *Jordan Basis* for T if, with respect to this basis T has a block diagonal matrix

$$\begin{bmatrix} A_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & A_m \end{bmatrix}$$

where each A_j is an upper-triangular matrix of the form

$$A_j = \begin{bmatrix} \lambda_j & 1 & & 0 \\ & \ddots & \ddots & \\ & & \ddots & 1 \\ 0 & & & \lambda_j \end{bmatrix}$$

Each diagonal of A_j is an eigenvalue of λ_j of T, and line above diagonal is 1, while rest of entries are 0.

Theorem 1.2.152. *Let $\mathcal{V}$ be a complex vector space, and $T \in \mathcal{L}(\mathcal{V})$. Then there exists a basis of $\mathcal{V}$ with respect to which the matrix of T is in Jordan form.*

Proof. First, suppose $T = N$ is nilpotent. By Lemma 1.2.150, $\exists$ vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k \in \mathcal{V}$ such that the vectors $\mathbf{v}_r, N\mathbf{v}_r, \ldots, N^{m(\mathbf{v}_r)}\mathbf{v}_r$ form a basis of $\mathcal{V}$.
For each r, reverse the order of the chain, $N^{m(\mathbf{v}_r)}\mathbf{v}_r, N^{m(\mathbf{v}_r)-1}\mathbf{v}_r, \ldots, N\mathbf{v}_r, \mathbf{v}_r$.
With respect to this ordered basis, N acts by sending each vector to the previous one and sending the first vector to 0. Hence the matrix of N on this chain has the form

$$\begin{bmatrix} 0 & 1 & & 0 \\ & \ddots & \ddots & \\ & & \ddots & 1 \\ 0 & & & 0 \end{bmatrix}$$

which is a Jordan block with eigenvalue 0.
Since the chains together forms a basis of $\mathcal{V}$, the matrix of N is block diagonal with Jordan blocks corresponding to eigenvalue 0. Thus every nilpotent operator has Jordan form.
Now let $T \in \mathcal{L}(\mathcal{V})$ be arbitrary. Let $\lambda_1, \ldots, \lambda_m$ be distinct eigenvalues of T, and let $\mathcal{U}_j$ be the corresponding generalised eigenspaces. By Major Structure Theorem 1.2.139 $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$ and $(T - \lambda_j I)|_{\mathcal{U}_j}$ is nilpotent.
Applying the nilpotent case to $(T - \lambda_j I)|_{\mathcal{U}_j}$, $\exists$ a basis of $\mathcal{U}_j$ with respect to which $(T - \lambda_j I)|_{\mathcal{U}_j}$ has Jordan form with eigenvalue 0. Equivalently, $T|_{\mathcal{U}_j}$ has Jordan blocks with eigenvalue λ_j.
Combining these bases for all $\mathcal{U}_j$ gives a basis of $\mathcal{V}$ with respect to which T is in Jordan form. □

Jordan form is obtained by decomposing $\mathcal{V}$ into generalised eigenspaces and then organising the nilpotent part on each generalised eigenspace into chains $\mathbf{v}, N\mathbf{v}, N^2\mathbf{v}, \ldots$. Each chain produces a Jordan block.

1.2.6 Trace and Determinant

2 Time Series Analysis

2.1 Time Series Analysis

Based on the books by James Douglas Hamilton (1994), and ...

2.1.1 Stationary Time Series

Definition 2.1.1. A *linear first-order difference equation* is defined as

$$y_t = \phi y_{t-1} + w_t$$

where y_t is the target variable (with y_{t-1} the lag 1 variable), w_t is an input variable at time t

The difference equation may be solved by recursive substitution to arrive at

$$y_t = \phi^{t+1} y_{-1} + \phi^t w_0 + \phi^{t-1} w_1 + \phi^{t-2} w_2 + \cdots + \phi w_{t-1} + w_t$$

The *dynamic multiplier* calculates effect of w_t on y_{t+j}, and is given by

$$\frac{\partial y_{t+j}}{\partial w_t} = \phi^j$$

If $|\phi| < 1$, the system is stable. If $|\phi| > 1$, then the system is explosive.

We may generalise the process to *p-th order difference equation*, i.e.,

$$y_t = \phi_t y_{t-1} + \phi_2 t_{t-2} + \cdots + \phi_p y_{t-p} + w_t$$

We may rewrite this as a first-order difference equation in a $(p \times 1)$ vector $\boldsymbol{\xi}_t$:

$$\boldsymbol{\xi}_t = \begin{bmatrix} y_t \\ y_{t-1} \\ \vdots \\ y_{t-p+1} \end{bmatrix}$$

Define the $(p \times p)$ matrix $\boldsymbol{F}$ by

$$\boldsymbol{F} = \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 & \cdots & \phi_{p-1} & \phi_p \\ 1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix}$$

Finally, define the $(p \times 1)$ vector $\boldsymbol{v}_t$ by

$$\boldsymbol{v}_t = \begin{bmatrix} w_t \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Then the system of p equations $\boldsymbol{\xi}_t = \boldsymbol{F}\boldsymbol{\xi}_{t-1} + \boldsymbol{v}_t$ is identical to the first order difference equation, and by recursive substitution, we have the following for the case of p-th order difference equation:

$$\boldsymbol{\xi}_{t+j} = \boldsymbol{F}^{j+1}\boldsymbol{\xi}_{t-1} + \boldsymbol{F}^j \boldsymbol{v}_t + \boldsymbol{F}^{j-1}\boldsymbol{v}_{t+1} + \cdots + \boldsymbol{F}\boldsymbol{v}_{t+j-1} + \boldsymbol{v}_{t+j}$$

Hence the dynamic multiplier is then $\frac{\partial y_{t+j}}{\partial w_t} f_{11}^{(j)}$, where $f_{11}^{(j)}$ is the $(1,1)$ element of $\boldsymbol{F}^j$.

2.1.2 Univariate Time Series Models

Trading activities through an exchange can be described by a sequence of time stamps ('ticks') $t_0 < t_1 < \cdots < t_n$, and 'marks' y_i at time t_i, where t_i denote market open, t_n denote market close. The marks y_i is a characteristic of the order book at time of ith activity. Events with marks associated with the ticks can be described mathematically as a marked point process.

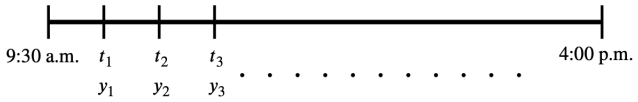


Figure 2: Ticks and Marks

A data aggregation method is where aggregation is conducted when there is a change in marker. An alternative aggregation method would be to divide the time span T for exchange hours into K intervals, so regularly spaced intervals are of size $\Delta t = T/K$. The time series method to be discussed will apply to all aggregated data.

Let $p_{it} = \ln P_{it}$ denote price of ith asset at time t; $p_t = (p_{1t}, p_{2t}, \ldots, p_{nt})$ denote price vector for n assets; y_{it} denote vector of characteristics of ith asset at time t. These quantities are aggregated from high frequency data. Consider r factors $f_t = (f_{1t}, f_{2t}, \ldots, f_{rt})$ that may include market and industry factors, and asset characteristics. Trading rules can be broadly grouped as follows:

i. *Statistical Arbitrage*: $E(p_{i,t+1} \mid p_{i,t}, p_{i,t-1}, \ldots, y_{i,t}, y_{i,t-1}, \ldots)$, that predicts the price of ith stock at $t+1$ based on past trading information (also known as time series momentum)

ii. *Momentum*: $E(p_{t+1} \mid p_t, p_{t-1}, \ldots, y_t, y_{t-1}, \ldots)$, that predicts the cross-sectional momentum of a subset of stocks based on past trading characteristics. For portfolio formation and rebalancing, pairs trading

iii. *Fair Value*: $E(p_{t+1} \mid p_t, p_{t-1}, \ldots, y_t, y_{t-1}, \ldots, f_t, f_{t-1}, \ldots)$, predicts price using all relevant quantities. Factors normally include market, Fama-French; at a more macro level than timescale considered for price prediction, but may still be useful.

Hence price and volatility prediction can be formulated as a time series prediction problem. Autocorrelations and partial autocorrelations can be used to build autoregressive and ARCH models with some predictive power.

Let $Y_1, Y_2, \ldots, Y_T$ be a sequence of random variables with a joint probability distribution. A sequence of observations of stochastic process $\{Y_t, t = 1, \ldots, T\}$ is a realisation of the process.
A time series $\{Y_t\}$ is *stationary* if for every integer m, the set of variables $Y_{t_1}, Y_{t_2}, \ldots, Y_{t_m}$ depends only on the distance between times $t_1, t_2, \ldots, t_m$. Thus $E(Y_t) = \mu$, $Var(Y_t) = \sigma^2$ are constant for all t.
The *auto-covariance* function is defined as

$$\gamma(s) = Cov(Y_t, Y_{t-s}) = E[(Y_t - \mu)(Y_{t-s} - \mu)] \quad \forall s = 0, \pm 1, \ldots$$

The *auto-correlation* of process at lag s is defined as

$$\rho(s) = Corr(Y_t, Y_{t-s}) = \frac{Cov(Y_t, Y_{t-s})}{(Var(Y_t)\ Var(Y_{t-s}))^{1/2}} = \frac{\gamma(s)}{\gamma(0)}, \quad \forall s = 0, \pm 1, \ldots$$

Some examples of stationary stochastic processes are as follows:

Example 2.1.2. *(White Noise)* Sequence of discrete independent random variables (r.v.) $\{\epsilon_t\}$ with $E[\epsilon_t] = 0$ and $E[\epsilon_t^2] = \sigma^2$. Set $Y_t = \mu + \epsilon_t$, then $E[Y_t] = \mu$, $Cov(Y_t, Y_{t-s}) = Var(Y_t) = \sigma^2$ if $s = 0$, and $Cov(Y_t, Y_{t-s}) = 0$ if $s \neq 0$. Hence the process is stationary.

Example 2.1.3. *(Moving Average)* Let $\{\epsilon_t\}$ be independent r.v., with process $\{Y_t\}$ where $Y_t = \mu + \epsilon_t + \epsilon_{t_1}$ for $t = 0, 1, 2, \ldots$, and μ is constant. Then $E[Y_t] = \mu\ \forall t$, and

$$Cov(Y_t, Y_{t-s}) = \gamma(s) = \begin{cases} 2\sigma^2 & \text{if } s = 0 \\ \sigma^2 & \text{if } s = 1 \\ 0 & \text{if } s > 1 \end{cases}$$

Hence $\{Y_t\}$ is stationary, with $\rho(s) = \gamma(s)/\gamma(0)$ such that $\rho(0) = 1$, $\rho(1) = 1/2$, $\rho(s) = 0$ for $|s| > 1$.

Some examples of non-stationary stochastic processes are as follows:

Example 2.1.4. *(Random Walk with Drift)* Let $\{\epsilon_t\}$ be sequence of independent r.v. with $E[\epsilon_t] = 0$, $E[\epsilon_t^2] = \sigma^2$, and define process $\{Y_t\}$ by $Y_t = Y_{t-1} + \delta + \epsilon_t$ with $Y_0 = 0$. Then the process can be summarised as

$$Y_t = \delta t + \sum_{j=1}^{t} \epsilon_j$$

Note that $E[Y_t] = \delta t$ and $Var[Y_t] = t\sigma^2$, hence process $\{Y_t\}$ is not stationary.

Given T observations from stationary process $\{Y_t\}$, the *sample mean* is $\overline{Y} = \frac{1}{T}\sum_{t=1}^{T} Y_t$. The *sample auto-covariance* function is defined by $\hat{\gamma}(s) = \frac{1}{T}\sum_{t=1}^{T-s}(Y_t - \overline{Y})(Y_{T-s} - \overline{Y})$ for $s = 0, 1, \ldots$. The *sample auto-correlation* function (ACF) is defined as $\hat{\rho}(s) = \frac{\hat{\gamma}(s)}{\hat{\gamma}(0)} = r(s)$. The *sample variance* is $Var(\overline{Y}) = \frac{\gamma(0)}{T}\left[1 + 2\sum_{u=1}^{Y-1}\left(\frac{T-u}{T}\right)\rho(u)\right]$; note that it has to account for auto-correlations.

Definition 2.1.5. A stochastic process $\{Y_t\}$ is a *linear process* if it can be represented as

$$Y_t = \mu + \sum_{j=0}^{\infty} \Psi_j \epsilon_{t-j}$$

where ϵ_t are independent with mean 0, variance σ_ϵ^2, and $\sum_{j=0}^{\infty} < \infty$.

3 Mathematical and Statistical Foundation

3.1 Classical Machine Learning

3.1.1 Ensemble Methods

3.2 Deep Learning

3.2.1 Deep Feedforward Networks

Remark 3.2.1. *Deep Feedforward Networks (DFNs)*
Defines a mapping $\boldsymbol{y} = f(\boldsymbol{x}; \boldsymbol{\theta})$ and learns value of parameters $\boldsymbol{\theta}$ that results in best function approximation.

Remark 3.2.2. *Linear Models*
Linear models may fit data efficiently and reliably, either in closed form or with convex optimisation.
Model capacity limited to linear functions, does not model interaction between any two input variables

Remark 3.2.3. *Nonlinear Models*
To represent nonlinear functions of $\boldsymbol{x}$, apply linear model to transformed input $\phi(\boldsymbol{x})$ where ϕ is a nonlinear transformation. Kernel trick may be applied to obtain nonlinear learning algorithm. To choose mapping ϕ:

i. Choose generic ϕ, such as that used by kernel machines based on RBF kernel.
If $\phi(\boldsymbol{x})$ is of high enough dimension, can find enough capacity to fit training set, but generalisations to test set remains poor. Mappings are based on principle of local smoothness and do not encode enough prior information to solve advanced problems.

ii. Manually engineer ϕ. Requires decades of human effort for each task, and practitioners specialising in different domains (i.e, speech recognition, computer vision) with little transfer between domains.

iii. Learn ϕ through deep learning. Model is $y = f(\boldsymbol{x}; \boldsymbol{\theta}, \boldsymbol{w}) = \phi(\boldsymbol{x}; \boldsymbol{\theta})^T \boldsymbol{w}$, where parameters $\boldsymbol{\theta}$ can be used to learn ϕ from broad class of functions, and parameters $\boldsymbol{w}$ that map from $\phi(\boldsymbol{x})$ to desired output.
Do not require training problem to be convex, and only require finding the right general function.

Definition 3.2.4. *Cost Functions*

i. Learning Conditional Distributions with Maximum Likelihood: cost function is negative log-likelihood, which is the cross-entropy between training data and model distribution:

$$J(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim\hat{p}_{\text{data}}} \log p_{\text{model}}(\boldsymbol{y} \mid \boldsymbol{x})$$

Specific form of cost function changes from model to model, depending on form of $\log p_{\text{model}}$.
Method removes the burden of designing cost functions for each model, as specifying a model $p(\boldsymbol{y} \mid \boldsymbol{x})$ automatically determines a cost function $\log p(\boldsymbol{y} \mid \boldsymbol{x})$.
Gradient of cost function for neural networks must be large and predictable enough to serve as good guide for the learning algorithm. The negative log-likelihood helps to avoid saturation of the function.

ii. Learning Conditional Statistics: to learn just one conditional statistic of $\boldsymbol{y}$ given $\boldsymbol{x}$.
With sufficiently powerful neural network, this can represent any function f from a wide class of function, limited by features of continuity and boundedness. Hence the cost function is a functional. Learning is choosing a functional rather than a set of parameters.
By calculus of variation , solving the optimisation problem yields the below function,

$$f^* = \arg\min_f \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim p_{\text{data}}} \|\boldsymbol{y} - f(\boldsymbol{x})\|^2$$
$$f^* = \mathbb{E}_{\boldsymbol{y}\sim p_{\text{data}}(\boldsymbol{y} \mid \boldsymbol{x})}[\boldsymbol{y}]$$

If infinitely many samples from the true data-generating distribution could be trained, then minimising the mean squared error cost function gives a function that predicts mean of $\boldsymbol{y}$ for each value of $\boldsymbol{x}$.
A second result from calculus of variations is:

$$f^* = \arg\min_f \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim p_{\text{data}}} \|\boldsymbol{y} - f(\boldsymbol{x})\|_1$$

which is a function that predicts the median value of $\boldsymbol{y}$ for each $\boldsymbol{x}$. This is the mean absolute error.
Note that mean squared error and mean absolute error often lead to poor results when used with gradient-based optimisation. Output units that saturate may produce very small gradients when combined with these cost functions. Hence the reason that cross-entropy cost function is more popular.

Definition 3.2.5. *Output Units*

i. Linear Units: base on affine transformation with no nonlinearity.
Given features $\boldsymbol{h}$, a layer of output units produces vector $\hat{\boldsymbol{y}} = \boldsymbol{W}^T \boldsymbol{h} + \boldsymbol{b}$. Linear output layers are used to produce mean of conditional Gaussian distribution:

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y}; \hat{\boldsymbol{y}}, \boldsymbol{I})$$

Maximising log-likelihood is equivalent to minimising mean squared error.
Linear units do not saturate, hence may be used for wide variety of optimisation algorithms.

ii. Sigmoid Units: define Bernoulli distribution y conditioned on $\boldsymbol{x}$. Neural net to predict $P(y = 1 \mid \boldsymbol{x})$, which lies in interval $[0, 1]$. The sigmoid unit is defined by

$$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{h} + b)$$

where σ is the logistic sigmoid function.
Note that the cost function used with maximum likelihood is $-\log P(y \mid \boldsymbol{x})$, preventing saturation.
The loss function for MLE of Bernoulli parametrised by sigmoid is

$$J(\boldsymbol{\theta}) = -\log P(y \mid \boldsymbol{x}) = -\log \sigma((2y-1)z) = \zeta((1-2y)z)$$

Function saturates only when $(1-2y)z$ is very negative, i.e., when model has the right answer.

iii. Softmax Units: used to represent probability distribution over n different classes.
A linear layer predicts unnormalised log probabilities:

$$\boldsymbol{z} = \boldsymbol{W}^T \boldsymbol{h} + \boldsymbol{b}$$

where $z_i = \log \tilde{P}(y = i \mid \boldsymbol{x})$. Softmax function is then

$$\text{softmax}(\boldsymbol{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

The function is to maximise $\log P(y = i; \boldsymbol{z}) = \log \text{softmax}(\boldsymbol{z})_i = z_i - \log \sum_j \exp(z_j)$.
Note that the input z_i always has direct contribution to cost function, and the term cannot saturate.
Un-regularised maximum likelihood will drive the model to learn parameters that drive the softmax to predict fraction of counts for each outcome observed in the training set:

$$\text{softmax}(z(\boldsymbol{x}; \boldsymbol{\theta}))_i \approx \frac{\sum_{j=1}^{m} \mathbf{1}_{y^{(j)}=i, \boldsymbol{x}^{(j)}=x}}{\sum_{j=1}^{m} \mathbf{1}_{\boldsymbol{x}^{(j)}=x}}$$

Note that objective functions other than log-likelihood does not work well with softmax function. Squared error is poor loss function for softmax units, and can fail to train the model to change its output.
Softmax function can saturate, and many functions based on softmax also saturate, unless they are able to invert the saturating activating function.

iv. Other Output Types: generally, given a conditional distribution $p(\boldsymbol{y} \mid \boldsymbol{x};\ \boldsymbol{\theta})$, principle of maximum likelihood suggests using $-\log p(\boldsymbol{y} \mid \boldsymbol{x};\ \boldsymbol{\theta})$ as the cost function.
Neural networks represent a function $f(\boldsymbol{x}; \boldsymbol{\theta}) = \boldsymbol{\omega}$. The outputs of the function are not direct predictions of value $\boldsymbol{y}$, but the parameters for distribution over $\boldsymbol{y}$. The loss function is then $-\log p(\boldsymbol{y}; \boldsymbol{\omega}(\boldsymbol{x}))$.

Remark 3.2.6. *Learning Distribution Parameters*

i. Heteroscedastic Model: to predict different variance in $\boldsymbol{y}$ for different values of $\boldsymbol{x}$, formulate the Gaussian distribution using precision rather than variance. In multi-variate case, the diagonal precision matrix is used, $\text{diag}(\boldsymbol{\beta})$. The log-likelihood of Gaussian distribution parametrised by $\boldsymbol{\beta}$ involves only multiplication by β_i and addition of $\log \boldsymbol{\beta}_i$. The gradient of these operations are well-behaved.
Let $\boldsymbol{\alpha}$ be raw activation of the model used to determine diagonal precision. The softplus function may be used to obtain a positive precision vector $\boldsymbol{\beta} = \zeta(\boldsymbol{\alpha})$. Same strategy applies equally if using variance variance or standard deviation rather than precision.
If covariance is full and conditional, then parametrisation must be chosen that guarantees positive-definiteness of predicted covariance matrix.

$$\boldsymbol{\Sigma}(\boldsymbol{x}) = \boldsymbol{B}(\boldsymbol{x})\boldsymbol{B}^T(\boldsymbol{x})$$

where $\boldsymbol{B}$ is unconstrained square matrix. Note that if matrix is full rank, then computing likelihood requires $O(d^3)$ a $d \times d$ matrix for the determinant and inverse of $\boldsymbol{\Sigma}(\boldsymbol{x})$.

ii. Mixture Density Networks: to perform multimodal regression (predict real values that come from condi-

tional distribution $p(\boldsymbol{y} \mid \boldsymbol{x})$) that can have several different peaks in $\boldsymbol{y}$ for the same $\boldsymbol{x}$.

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \sum_{i=1}^{n} p(c = i \mid \boldsymbol{x}) \mathcal{N}(\boldsymbol{y}; \boldsymbol{\mu}^{(i)}(\boldsymbol{x}), \boldsymbol{\Sigma}^{(i)}(\boldsymbol{x}))$$

The neural network will have three outputs:

1. Mixture components $p(c = i \mid \boldsymbol{x})$, forming a multinoulli distribution over n different components with latent variable c, obtained by softmax over n-dimensional vector.
2. Means $\boldsymbol{\mu}^{(i)}(\boldsymbol{x})$
3. Covariances $\boldsymbol{\Sigma}^{(i)}(\boldsymbol{x})$

Gradient-based optimisation of conditional Gaussian mixtures can be unreliable as the divisions can be numerically unstable. May be solved by clipping gradients, or scaling gradients heuristically.

Definition 3.2.7. *Hidden Units*
Even if hidden units are not differentiable at all input points, gradient descent still performs well enough as the training algorithms do not usually arrive at local minimum of cost function, but reduce its value significantly. Most hidden units has input vector $\boldsymbol{x}$, computes an affine transformation $\boldsymbol{z} = \boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b}$, then applying element-wise non-linear function $g(\boldsymbol{z})$.

i. Rectified Linear Units (ReLUs): uses activation function $g(z) = \max\{0, z\}$.
Typically used on top of an affine transformation $\boldsymbol{h} = g(\boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b})$. In initialisation, set all elements of $\boldsymbol{b}$ to small positive values, so that ReLUs will be initially active for most inputs in training set.
Generalisations of ReLUs have non-zero slope α_i for $z_i < 0$: $h_i = \max(0, z_i) + \alpha_i \min(0, z_i)$.

   1. Absolute Value Rectification: sets $\alpha_i = -1$ to obtain $g(z) = |z|$. Used for object recognition from images, to seek features that are invariant under polarity reversal of input illumination.
   2. Leaky ReLU: fixes α_i to small positive value
   3. Parametric ReU: treats α_i as a learnable parameter

ii. Maxout Units: divide $\boldsymbol{z}$ into groups of k values. Each maxout unit then outputs maximum element of one of these groups: $g(\boldsymbol{z})_i = \max_{j \in \mathbb{G}^{(i)}} z_j$, where $\mathbb{G}^{(i)}$ is indices of inputs for group i, which is $\{(i-1)k+1, \ldots, ik\}$.
This allows learning of piecewise linear function that responds to multiple directions in input $\boldsymbol{x}$ space .
Maxout units learn the activation function itself. With large k, maxout unit can learn to approximate any convex function. Each maxout unit is parametrised by k weight vectors, hence need more regularisation than ReLUs. Benefits include:

   1. Can work well without regularisation if training set is large and number of pieces per unit is low.
   2. Can gain statistical and computation advantages by requiring fewer parameters.
   3. Have redundancy that resists catastrophic forgetting, where neural networks forgot how to perform tasks that were trained on in the past.

iii. Logistic Sigmoid and Hyperbolic Tangent: the logistic sigmoid activation function is $g(z) = \sigma(z)$, and the hyperbolic tangent activation function is $g(z) = \tanh(z)$. Note that $\tanh(z) = 2\sigma(2z) - 1$.
Sigmoidal units saturate across most of the domain, which makes gradient-based learning very difficult. Hence the use in hidden units in feedforward networks is now discouraged.
If sigmoidal activation function must be used, hyperbolic tangent activation function performs better, as it resembles identity function more closely. Training $\hat{y} = \boldsymbol{w}^T \tanh(\boldsymbol{U}^T \tanh(\boldsymbol{V}^T\boldsymbol{x}))$ resembles training a linear model $\hat{y} = \boldsymbol{w}^T\boldsymbol{U}^T\boldsymbol{V}^T\boldsymbol{x}$ as long as the activations of the network can be kept small.
Sigmoidal functions are more common in recurrent networks, probabilities models, auto-encoders

iv. Linear Unit: consider neural network layer with n inputs, p outputs, $\boldsymbol{h} = g(\boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b})$. Replace with two layers, one using weight matrix $\boldsymbol{U}$ and the other using weight matrix $\boldsymbol{V}$. If first layer has no activation function, then the factored approach is to compute $\boldsymbol{h} = g(\boldsymbol{V}^T\boldsymbol{U}^T\boldsymbol{x} + \boldsymbol{b})$. If $\boldsymbol{U}$ produces q outputs, then $\boldsymbol{U}$ and $\boldsymbol{V}$ together only contains $(n + p)q$ parameters, while $\boldsymbol{W}$ contains np parameters. For small q, this is considerable saving in parameters, while the cost is constraining linear transformation to be low rank. This is an efficient way of reducing number of parameters in the model.

v. Softmax: naturally represent probability distribution over discrete variable with k possible values. Used only in more advanced architectures that explicitly learn to manipulate memory.

vi. Radial Basis Function (RBF): function becomes more active as $\boldsymbol{x}$ approaches a template $\boldsymbol{W}_{:,i}$. As it saturates to 0 for most $\boldsymbol{x}$, it can be difficult to optimise.

$$h_t = \exp\left(-\frac{1}{\sigma_i^2}\|\boldsymbol{W}_{:,i} - \boldsymbol{x}\|^2\right)$$

vii. Softplus: smooth version of rectifier for functional approximation and for conditional distributions of undirected probabilistic models. Usage is generally discouraged.

$$g(a) = \zeta(a) = \log(1 + e^a)$$

viii. Hard tanh: shaped similarly to tanh but bounded.

$$g(a) = \max(-1, \min(1, a))$$

Theorem 3.2.8. *Universal Approximation Theorem*
A feedforward network with linear output layer and at least one hidden layer with any 'squashing' activation function can approximate any Borel measurable function from one finite-dimensional space to another with any desired non-zero amount of error, provided the network is given enough hidden units.

Method 3.2.9. *Architecture Design*
Neural network layers are arranged in a chain structure, with each layer being a function of preceding layer.

$$\boldsymbol{h}^{(1)} = g^{(1)}(\boldsymbol{W}^{(1)T}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{h}^{(k)} = g^{(k)}(\boldsymbol{W}^{(k)T}\boldsymbol{h}^{(k-1)} + \boldsymbol{b}^{(k)}), \quad k \geq 2$$

The main considerations are the depth of network and width of each layer. Deeper networks use far fewer units per layers and far fewer parameters, and often generalise to the test set, but are harder to optimise.

Remark 3.2.10. *Depth of Network and Universal Approximation Theorem*
A feedforward network with single layer is sufficient to represent any function, but the layer may be infeasibly large and fail to learn and generalise correctly. Using deeper models can reduce number of units required to represent the desired function and can reduce generalisation error.
Shallow networks with broad family of non-polynomial activation functions have universal approximation properties. Piecewise linear networks can represent functions with number of regions that is exponential to depth. The number of linear regions carved out by deep rectifier network with d inputs, depth l, and n units per hidden layer is $O(\binom{n}{d}^{d(d-1)} n^d)$. For maxout networks with k filters per unit, this is $O(k^{(k-1)+d})$.

Remark 3.2.11. *Backpropagation*
To calculate the gradient of loss function with respect to each of individual parameters of the neural network. Model training begins with random initialisation of weights and biases.

i. Forward pass: input is sampled from training data. Nodes receive input vector and passes their value (multiplied by random initial weight) to nodes of first hidden layer. The hidden units take weighted sum of these output values as an input to the activation function, whose output is used for next hidden layer.

ii. Error computation: the final output of the network is compared to the ground truth, difference is calculated for the error value.

iii. Backwards pass: the error value computed earlier is used to compute the gradient of loss function. The gradient is then propagated back through the network, and the weights are updated according to their contribution to the error. The learning rate determines the size of weight updates.

iv. Weights update: the weights are updated in opposite direction of the gradient

Algorithm 1: Backpropagation Learning Algorithm

Require:
A set of training examples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$
A multilayer network with L layers, weights w_{ij}^l, and activation function f
Loss function $J(y, o)$
Learning rate $0 < \alpha < 1$
Number of epochs *epochs*.

Initialize all weights:
for w_{ij}^l in the network **do**
  $w_{ij}^l \leftarrow$ small random number
end for

for $i = 1$ to *epochs* **do**
  for all training examples $(x, y) \in \mathcal{D}$ **do**

Propagate the inputs forward to compute the outputs
for all neurons i in the input layer **do**
  $a_i^0 \leftarrow x_i$
end for

for $l = 2$ to L **do**
  for all neurons i in layer l **do**
    $z_i^l \leftarrow \sum_j w_{ij}^l \, a_j^{l-1}$
    $a_i^l \leftarrow f(z_i^l)$
  end for
end for

Propagate deltas backward from output layer to input layer
for all neurone i in the output layer **do**
  Let $o_i \equiv a_i^L$ *be output of neurone* i
  $\delta_i^L \leftarrow \dfrac{\partial J(y_i, o_i)}{\partial o_i} \, f'(z_i^L)$
end for

for $l = L - 1$ to 1 **do**
  for all neurons i in layer l **do**
    $\delta_i^l \leftarrow f'(z_i^l) \sum_j (w_{ji}^{l+1} \, \delta_j^{l+1})$
  end for
end for

Update weights using the deltas
for all w_{ij}^l in the network **do**
  $w_{ij}^l \leftarrow w_{ij}^l - \alpha \, \delta_i^l \, a_j^{l-1}$
end for
end for
end for

Example 3.2.12. *Backpropagation Computation Example (Matt Mazur (2015))*

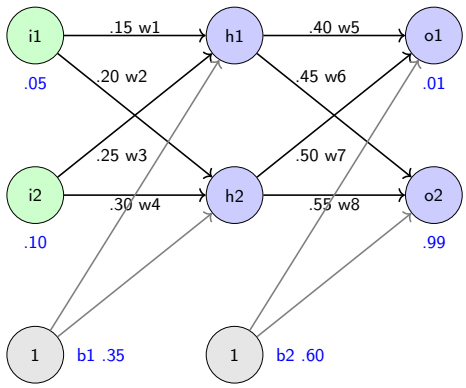


Figure 3: Backpropagation Example

For the forward pass, compute the total net input to each hidden layer neurone, squash total net input with activation function (logistic function), then repeat process with output layer neurones.
The total net input for h_1 is:

$$\begin{aligned} \text{net}_{h1} &= w_1 \cdot i_1 + w_2 \cdot i_2 + b_1 \cdot 1 \\ &= 0.15 \cdot 0.05 + 0.2 \cdot 0.1 + 0.35 \cdot 1 = 0.3775 \end{aligned}$$

Squash with logistic function to get output of h_2:

$$\text{out}_{h1} = 1/(1 + e^{-\text{net}_{h1}}) = 1/(1 + e^{-0.3775}) = 0.593269992$$

For h_2, repeating the same process, the output is then $\text{out}_{h2} = 0.596884378$.
Repeating the process for outer layer neurones,

$$\begin{aligned}\text{net}_{o1} &= w_5 \cdot \text{out}_{h1} + w_6 \cdot \text{out}_{h2} + b_2 \cdot 1 \\ &= 0.4 \cdot 0.593269992 + 0.45 \cdot 0.596884378 + 0.6 \cdot 1 = 1.105905967 \\ \text{out}_{o1} &= 1/(1 + e^{-\text{net}_{o1}}) = 1/(1 + e^{-1.105905967}) = 0.75136507\end{aligned}$$

For o_2, repeating the same process, the output is then $\text{out}_{o2} = 0.772928465$.
The total error for each output neurone may be computed using squared error function $E_{\text{total}} = \sum \frac{1}{2}(Y - \hat{Y})^2$.
For o_1, the target output is 0.01, neurone output is 0.75136507, hence error is

$$E_{o1} = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

Repeating the process for o_2, the total error for neural network is then

$$E_{\text{total}} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

For the backpropagation step, update each of the weights in network to reduce the error for each output neurone.

$$\begin{aligned}E_{\text{total}} &= \frac{1}{2}(Y_{o1} - \hat{Y}_{o1})^2 + \frac{1}{2}(Y_{o2} - \hat{Y}_{o2})^2 \\ \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} &= 2 \cdot \frac{1}{2}(Y_{o1} - \hat{Y}_{o1}) \cdot -1 + 0 = -(0.01 - 0.75136507) = 0.74136507\end{aligned}$$

Also, note that

$$\begin{aligned}\text{out}_{o1} &= 1/(1 + e^{-\text{net}_{o1}}) \\ \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} &= \text{out}_{o1}(1 - \text{out}_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602 \\ \text{net}_{o1} &= w_5 \cdot \text{out}_{h1} + w_6 \cdot \text{out}_{h2} + b_2 \cdot 1 \\ \frac{\text{net}_{o1}}{w_5} &= 1 \cdot \text{out}_{h1} \cdot w_5^0 + 0 + 0 = \text{out}_{h1} = 0.593269992\end{aligned}$$

Putting it all together,

$$\begin{aligned}\frac{\partial E_{\text{total}}}{\partial w_5} &= \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} \cdot \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} \cdot \frac{\partial \text{out}_{o1}}{\partial w_5} \\ &= -(Y_{o1} - \hat{Y}_{o1}) \cdot \text{out}_{o1}(1 - \text{out}_{o1}) \cdot textout_{o1} \\ &= 0.74136507 \cdot 0.186815602 \cdot 0.593269992 = 0.0821267041\end{aligned}$$

To decrease the error, subtract the value from current weight multiplied by learning rate (i.e., $\alpha = 0.5$).

$$w_5^+ = w_5 - \alpha \frac{\partial E_{\text{total}}}{\partial w_5} = 0.4 - 0.5 \cdot 0.0821267041$$

Repeating for the rest of the weights, we get

$$w_6^+ = 0.408666186, \quad w_7^+ = 0.511301270, \quad w_8^+ = 0.561370121$$

For the hidden layers, note that

$$\begin{aligned}\frac{\partial E_{\text{total}}}{\partial w_1} &= \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1} \\ &= \left(\frac{\partial E_{o1}}{\partial \text{out}_{h1}} + \frac{\partial E_{o2}}{\partial \text{out}_{h1}}\right) \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1}\end{aligned}$$

Note that out_{h1} affects both out_{o1} and out_{o2}.

$$\frac{\partial E_{o1}}{\partial \text{out}_{h1}} = \frac{\partial E_{o1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial \text{out}_{h1}}$$
$$\frac{\partial E_{o1}}{\partial \text{net}_{h1}} = \frac{\partial E_{o1}}{\partial \text{out}_{h1}} \cdot \frac{\partial \text{out}_{o1}}{\partial \text{net}_{h1}} = 0.74136507 \cdot 0.186815602 = 0.138498562$$
$$\frac{\partial \text{net}_{h1}}{\partial \text{out}_{h1}} = w_5 = 0.4$$
$$\frac{\partial E_{o1}}{\partial \text{out}_{h1}} = 0.138498562 \cdot 0.4 = 0.055399425$$

Following same process, $\frac{\partial E_{o2}}{\partial \text{out}_{h1}} = -0.019049119$. Hence

$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} = \frac{\partial E_{o1}}{\partial \text{out}_{h1}} + \frac{\partial E_{o2}}{\partial \text{out}_{h1}} = 0.05539425 - 0.019049119 = 0.036350306$$

Next, figure out $\frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}}$ and $\frac{\partial \text{net}_{h1}}{\partial w_1}$.

$$\frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} = \text{out}_{h1} \cdot (1 - \text{out}_{h1}) = 0.59326999(1 - 0.59326999) = 0.2410.241300709$$
$$\frac{\partial \text{net}_{h1}}{\partial w_1} = i_1 = 0.05$$

Putting it together,

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1} = 0.036350306 \cdot 0.241300709 \cdot 0.05 = 0.000438658$$

This is also written as

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \left(\sum_o \frac{\partial E_{\text{total}}}{\partial \text{out}_o} \cdot \frac{\partial \text{out}_o}{\partial \text{net}_o} \cdot \frac{\partial \text{net}_o}{\partial \text{out}_{h_1}} \right) \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1}$$

Next, update w_1 and rest of the weights

$$w_1^+ = w_1 - \alpha \frac{\partial E_{\text{total}}}{\partial w_1} = 0.15 - 0.5 \cdot 0.000438658 = 0.149780716$$
$$w_2^+ = 0.19956143, \quad w_3^+ = 0.24975114, \quad w_4^+ = 0.29950229$$

Hence all the weights has been updated. After the first round of backpropagation, total error has decreased to 0.291027924. Repeating the process $10,000$ times, the error reduces to 0.0000351085.

3.2.2 Regularisation for Deep Learning

Definition 3.2.13. *Regularisation* refers to adding a parameter norm penalty $\Omega(\boldsymbol{\theta})$ to the objective function J. The regularised objective function is then

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta})$$

where $\alpha in [0, \infty)$ is a hyper-parameter that weights the contribution of norm penalty term.
For neural networks, the parameter norm penalty Ω is chosen such that it penalises only the weights of the affine transformation at each layer and leaves the biases un-regularised.

Definition 3.2.14. *L^2 (Ridge) Regularisation*
The regularisation term added to objective function is

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{w}\|_2^2$$

Remark 3.2.15. *Behaviour of Weight Decay (L^2) Regularisation*
Assuming no bias parameter, a model have the following objective function and parameter gradient:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \frac{\alpha}{2}\boldsymbol{w}^T\boldsymbol{w} + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$
$$\nabla_{\boldsymbol{w}}\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\boldsymbol{w} + \nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

On a single gradient step, the update is as follows:

$$\boldsymbol{w} \leftarrow (1 - \epsilon\alpha)\boldsymbol{w} - \epsilon\nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

The addition of weight decay has modified learning rule to multiplicatively shrink weight vector by constant factor on each step before gradient update.
Using quadratic approximation to objective function at minimal unregularised training cost, approximation is

$$\hat{J}(\overline{\theta}) = J(\boldsymbol{w}^*) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}^*)\boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*), \qquad \boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} J(\boldsymbol{w})$$

where $\boldsymbol{H}$ is Hessian matrix of J with respect to $\boldsymbol{w}$ evaluated at $\boldsymbol{w}^*$. Minimum of $\hat{J}$ occurs where gradient is

$$\nabla_{\boldsymbol{w}}\hat{J}(\boldsymbol{w}) = \boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*) = \boldsymbol{0}$$

Adding weight decay gradient, solve for minimum of regularised version of $\hat{J}$. Let $\tilde{\boldsymbol{w}}$ be minimum, then

$$\alpha\tilde{\boldsymbol{w}} + \boldsymbol{H}(\tilde{\boldsymbol{w}} - \boldsymbol{w}^*) = 0$$
$$(\boldsymbol{H} + \alpha\boldsymbol{I})\tilde{\boldsymbol{w}} = \boldsymbol{H}\boldsymbol{w}^*$$
$$\tilde{\boldsymbol{w}} = (\boldsymbol{H} + \alpha\boldsymbol{I})^{-1}\boldsymbol{H}\boldsymbol{w}^*$$

As $\alpha \to 0$, regularised solution $\tilde{\boldsymbol{w}} \to \boldsymbol{w}^*$. Note $\boldsymbol{H}$ is real and symmetric, hence decompose into diagonal matrix $\boldsymbol{A}$ and orthonormal basis of eigenvectors $\boldsymbol{Q}$ such that $\boldsymbol{H} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$, to get

$$\tilde{\boldsymbol{w}} = (\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T + \alpha\boldsymbol{I})^{-1}\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T\boldsymbol{w}^* = \boldsymbol{Q}(\boldsymbol{\Lambda} + \alpha\boldsymbol{I})^{-1}\boldsymbol{\Lambda}\boldsymbol{Q}^T\boldsymbol{w}^*$$

The effect of weight decay rescale $\boldsymbol{w}^*$ along axes defined by eigenvectors of $\boldsymbol{H}$. The component of $\boldsymbol{w}^*$ aligned with i-th eigenvector of $\boldsymbol{H}$ is rescaled by factor $\frac{\lambda_i}{\lambda_i + \alpha}$. For components where $\lambda_i \gg \alpha$, the effects of regularisation is relatively small. For components where $\lambda_i \ll \alpha$, components will be shrunk to nearly zero magnitude.

Definition 3.2.16. *L^1 Regularisation*
The L^1 regularisation is the sum of absolute values of individual parameters.

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{w}\|_1 = \sum_i |w_i|$$

Definition 3.2.17. *Behaviour of L^1 Regularisation*
Assuming no bias parameter, a model has the following objective function and parameter gradient:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\|\boldsymbol{w}\|_1 + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$
$$\nabla_{\boldsymbol{w}}\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\ \text{sign}(\boldsymbol{w}) + \nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

Note that the regularisation contribution to gradient is a constant factor with sign equal to $\text{sign}(w_i)$. Minimum of $\tilde{J}$ occurs at where

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}) = \boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*)$$

Assuming the Hessian is diagonal, $\boldsymbol{H} = \text{diag}([H_{1,1}, \dots, H_{n,n}])$, where each $H_{i,i} > 0$. The quadratic approximation of regularised objective function is then

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{w}^*; \boldsymbol{X}, \boldsymbol{y}) + \sum_i \left[\frac{1}{2} H_{i,i} (\boldsymbol{w}_i - \boldsymbol{w}_i^*)^2 + \alpha |w_i| \right]$$

$$w_i = \text{sign}(w_i^*) \max \left\{ |w_i^*| - \frac{\alpha}{H_{i,i}}, 0 \right\}$$

For the case where $w_i^* > 0$ for all i, then

i. if $w_i^* \leq \frac{\alpha}{H_{i,i}}$, the optimal value is $w_i = 0$ as contribution of $J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$ to regularised objective $\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$ is overwhelmed in direction i by L^1 regularisation which pushes w_i to zero.

ii. if $w_i^* > \frac{\alpha}{H_{i,i}}$, regularisation shifts optimal value of w_i in the direction by $\frac{\alpha}{H_{i,i}}$.

For the case where $w_i^* < 0$, this happens similarly, but with L^1 penalty decreasing w_i by $\frac{\alpha}{H_{i,i}}$, with min value 0. Note that L^1 produces a more sparse solution, which is a feature selection mechanism.

Remark 3.2.18. *Norm Penalties as Constrained Optimisation*
Let cost function regularised by parameter norm penalty be

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha \Omega(\boldsymbol{\theta})$$

Construct a generalised generalised Lagrange function,

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha(\Omega(\boldsymbol{\theta}) - k)$$

The solution to the constrained problem is then

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \max_{\alpha, \alpha \geq 0} \mathcal{L}(\boldsymbol{\theta}, \alpha)$$

Note that optimal value α^* will shrink $\Omega(\boldsymbol{\theta})$, but not such that it is less than k.
Fixing α^*, the problem is then a function of $\boldsymbol{\theta}$,

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}, \alpha^*) = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha^* \Omega(\boldsymbol{\theta})$$

This is the regularised training problem of minimising $\tilde{J}$. The parameter norm penalty is imposing a constraint on the weights. If explicit constraints are to be used rather than penalties, use stochastic gradient descent on $J(\boldsymbol{\theta})$, then take the projected $\boldsymbol{\theta}$ to the nearest point that satisfies $\Omega(\boldsymbol{\theta}) < k$. This method is used if k is predefined by user, and time is not to be spent on searching for α value that corresponds to this k.
Explicit constraints and re-projection work better in circumstances where non-convex optimisation is involved, and this may avoid getting stuck in local minima. Explicit constraints also impose stability, and together with high learning rate allows rapid exploration of parameter space.

Remark 3.2.19. *Dataset Augmentation*
Effective technique for object classification. Operations such as translation of images by few pixels in each direction can greatly improved generalisation, even if the model has been designed to be partially translation invariant using convolution and pooling techniques. Rotation and scaling of images are also quite effective.
For speech recognition tasks, noice injection will improve performance. To improve robustness, train models with random noise applied to inputs. Noise injection also works when applied to the hidden units (data augmentation at multiple levels of abstraction). Enabling dropout constructs new inputs by multiplying by noise.

Remark 3.2.20. *Noise Robustness*
For some models, addition of noise with infinitesimal variance at input of model is equivalent to imposing penalty on norm of the weights. Generally, noise injection can be much more powerful than simply shrinking the parameters, especially when noise is added to hidden units.
Noise may also be added to weights, which is primarily used in recurrent neural networks. This is a stochastic implementation of Bayesian inference over the weights (model weights are uncertain and representable via a probability distribution that reflects this uncertainty).

Adding noise to weights may also result in more stability in function to be learned. Let the cost function be

$$J = \mathbb{E}_{p(x,y)}[(\hat{y}(\boldsymbol{x}) - y)^2]$$

where $\hat{y}(\boldsymbol{x})$ is function to be trained that maps set of features $\boldsymbol{x}$ to a scalar using least-squares cost function J. Assume each input presentation include a random perturbation $\epsilon_{\boldsymbol{W}} \sim \mathcal{N}(\boldsymbol{\epsilon}; \boldsymbol{0}, \eta \boldsymbol{I})$ in the network weight of a standard l-layer MLP. Let the perturbed model be $\hat{y}_{\boldsymbol{\epsilon_W}}(\boldsymbol{x})$. With injection o noise, objective function becomes

$$\tilde{J}_{\boldsymbol{W}} = \mathbb{E}_{p(\boldsymbol{x},y,\boldsymbol{\epsilon_W})}\left[(\hat{y}_{\boldsymbol{\epsilon_W}}(\boldsymbol{x}) - y)^2\right] = \mathbb{E}_{p(\boldsymbol{x},y,\boldsymbol{\epsilon_W})}\left[\hat{y}^2_{\boldsymbol{\epsilon_W}}(\boldsymbol{x}) - 2y\hat{y}_{\boldsymbol{\epsilon_W}} + y^2\right]$$

For small η, the minimisation of J with added noise (with covariance $\eta \boldsymbol{I}$) is equivalent to minimisation of J with additional regularisation term $\eta \mathbb{E}_{p(\boldsymbol{x}),y}[\|\nabla_{\boldsymbol{W}} \hat{y}(\boldsymbol{x})\|^2]$. This encourages parameters to go to regions of parameter space where small perturbations of weights have relatively small influence on output.

Remark 3.2.21. *Label Smoothing*
Datasets may have some mistake in y labels. Regularise a model based on softmax with k output values by replacing hard 0 and 1 classification targets with targets of $\frac{\epsilon}{k-1}$ and $1-\epsilon$ respectively, assuming that the training set label y is correct with probability $1 - \epsilon$. Cross-entropy can then be used on these soft targets.

Remark 3.2.22. *Semi-Supervised Learning*
Unlabelled examples $P(\boldsymbol{x})$ and $P(\boldsymbol{x}, \boldsymbol{y})$ are used to estimate $P(\boldsymbol{y} \mid \boldsymbol{x})$ or predict $\boldsymbol{y}$ from $\boldsymbol{x}$.
Construct a generative model of either $P(\boldsymbol{x})$ or $P(\boldsymbol{x}, \boldsymbol{y})$ which shares parameters with a discriminative model of $P(\boldsymbol{y} \mid \boldsymbol{x})$. The supervised criterion $\log P(\boldsymbol{y} \mid \boldsymbol{x})$ may then be traded with the unsupervised or generative one ($\log P(\boldsymbol{x})$ or $\log P(\boldsymbol{x}, \boldsymbol{y})$). The generative criterion then expresses a particular form of prior belief about the solution to the supervised learning problem (structure of $P(\boldsymbol{x})$ is connected to structure of $P(\boldsymbol{y} \mid \boldsymbol{x})$ such that is is captured by the shared parametrisation).

Remark 3.2.23. *Multi-Task Learning*
Improve generalisations by pooling examples (soft constraints imposed on parameters) from several tasks.
Different supervised tasks (predicting $\boldsymbol{y}^{(i)}$ given $\boldsymbol{x}$) share the same input $\boldsymbol{x}$, as well as some intermediate-level representation $\boldsymbol{h}^{(\text{shared})}$ capturing a common pool of factors. Model can be divided in two parts:

i. Task-specific parameters
ii. Generic parameters, shared across all the tasks

Improved generalisation and associated error bounds can be achieved due to shared parameters, but this only happens if some assumptions about statistical relationship between different tasks are valid.

Remark 3.2.24. *Early Stopping*
To obtain a model with better validation set error by returning to the parameter setting at the point in time with lowest validation set error. Model validation is done in parallel to model training.
Early stopping has the effect of restricting the optimisation procedure to a relatively small volume of parameter space in neighbourhood of initial parameter value $\boldsymbol{\theta}_o$. Assume τ optimisation steps are taken with learning rate ϵ. The product $\epsilon\tau$ is a measure of effective capacity. Assuming gradient is bounded, restricting $\epsilon\tau$ limits the volume of parameter space reachable from $\boldsymbol{\theta}_o$, hence $\epsilon\tau$ behaves as the reciprocal of weight decay coefficients.

Algorithm 2: Early Stopping Algorithm

Require:
Number of steps n between evaluations
Patience p, the number of times to observe worsening validation error before stopping
Initial parameters $\boldsymbol{\theta}_o$

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_o$, $i \leftarrow 0$, $j \leftarrow 0$, $v \leftarrow \infty$, $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$, $i^* \leftarrow i$
while $j < p$ **do**
  Update $\boldsymbol{\theta}$ by running training algorithm for n steps
  $i \leftarrow i + n$, $v' \leftarrow$ ValidationSetError($\boldsymbol{\theta}$)
  if v' < v **then**
    $j \leftarrow 0$, $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$, $i^* \leftarrow i$, $v \leftarrow v'$
  end if
  $j \leftarrow j + 1$
end while
Best parameters are $\boldsymbol{\theta}^*$, best number of training steps is i^*

Remark 3.2.25. *Parameter Tying and Parameter Sharing*
Used to express prior knowledge on suitable values of model parameters (precise values are not known, but there should be some dependencies between parameters). To regularise parameters to be close, may use:

i. Parameter norm penalty: assume model A with parameters $\boldsymbol{w}^{(A)}$ and model B with parameters $\boldsymbol{w}^{(B)}$. Two models map input to two different but related outputs $\hat{y}^{(A)} = f(\boldsymbol{w}^{(A)}, \boldsymbol{x})$ and $\hat{y}^{(B)} = g(\boldsymbol{w}^{(B)}, \boldsymbol{x})$. The model parameters $w_i^{(A)}$ should be close to $w_i^{(B)}$ $\forall i$. Using regularisation, a parameter norm penalty may be imposed, such as using L^2 penalty to get $\Omega(\boldsymbol{w}^{(A)}, \boldsymbol{w}^{(B)}) = \left\|\boldsymbol{w}^{(A)} - \boldsymbol{w}^{(B)}\right\|_2^2$.
ii. Parameter sharing: using constraints. Only a subset of the parameters (input set) need to be stored in memory. May lead to significant reduction in memory footprint of the model (i.e., in CCNs)

Remark 3.2.26. *Sparse Representations*
Representation sparsity is where elements of the representation are zero, accomplished by same sort of mechanisms used in parameter regularisation.
Given a linear regression $\boldsymbol{y} = \boldsymbol{B}\boldsymbol{h}$, where $\boldsymbol{y} \in \mathbb{R}^m, \boldsymbol{B} \in \mathbb{R}^{m \times n}, \boldsymbol{h} \in \mathbb{R}^n$, and $\boldsymbol{h}$ is the representation of data $\boldsymbol{x}$, add to loss function J a norm penalty on the representation ($\Omega(\boldsymbol{h})$)):

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{h})$$

where $\alpha \in [0, \infty)$ weights relative contribution of norm penalty, with larger values of α for more regularisation. L^1 penalty on elements of representation induces sparsity, $\Omega(\boldsymbol{h}) = \|\boldsymbol{h}\|_1 = \sum_i |h_i|$. For representations with elements constrained to lie on unit interval, may use Student-t prior or KL divergence penalties.
Other approaches such as hard constraint on activation values may also work. *Orthogonal matching pursuit* encodes an input $\boldsymbol{x}$ with representation $\boldsymbol{h}$ that solves the constrained optimisation problem

$$\arg \min_{\boldsymbol{h}, \|\boldsymbol{h}\|_0 < k} \|\boldsymbol{x} - \boldsymbol{W}\boldsymbol{h}\|^2$$

where $\|\boldsymbol{h}\|_0$ is the number of non-zero entries of $\boldsymbol{h}$. If $\boldsymbol{W}$ is constrained to be orthogonal, this can be solved efficiently with OMP-k method, where k is number of non-zero features allowed.

Remark 3.2.27. *Bootstrap Aggregating (Bagging)*
Construct k different datasets, each with same number of examples as original dataset, but constructed by sampling with replacement from original dataset. Each dataset may miss some examples but also contain duplicate examples. Each model i is then trained on the dataset i. Differences in random initialisation, random selection of mini-batches, differences in hyperparameters, or different outcomes of non-deterministic implementation of neural networks are enough to allow each model to make partially independent errors.

Remark 3.2.28. *Boosting*
Construct k different datasets from original dataset. Using ith dataset, train ith weak learner. Test the ith weak learner with validation dataset, and each datapoint with wrong prediction is sent to $(i+1)$th dataset. The $(i+1)$th dataset is then used to train the $(i+1)$th weak learner. Continue until reaching the total number of subsets for the total prediction.

Remark 3.2.29. *Dropout*
Dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from the base network by multiplying its output value by zero. To train a neural network with dropout, use minibatch-based learning algorithms (i.e., Stochastic Gradient Descent). Each time an example is loaded into minibatch, randomly (and independently) sample a different binary mask $\boldsymbol{\mu}$ to apply to all of input and hidden units in the network, where the probability of sampling a mask value of one is the dropout rate.
Let $J(\boldsymbol{\theta}, \boldsymbol{\mu})$ be cost of model defined by parameters $\boldsymbol{\theta}$ and mask $\boldsymbol{\mu}$. Dropout training is minimising $\mathbb{E}_{\boldsymbol{\mu}} J(\boldsymbol{\theta}, \boldsymbol{\mu})$. Each model inherits different subset of parameters from parent neural network, and a tiny fraction of possible sub-networks are trained for a single step. Parameter sharing allows sub-networks to arrive at good parameter settings. Each sub-network given by $\boldsymbol{\mu}$ defines a probability distribution $p(y \mid \boldsymbol{x}, \boldsymbol{\mu})$. The arithmetic mean over all masks is then given by

$$\sum_{\boldsymbol{\mu}} p(\boldsymbol{\mu}) p(y \mid \boldsymbol{x}, \boldsymbol{\mu})$$

where $p(\boldsymbol{\mu})$ is the distribution used to sample $\boldsymbol{\mu}$ during training.
Using dropout as a regularisation technique reduces effective capacity of a model. To offset this, increase the size of the model. Typically optimal validation error is much lower when using dropout, but this comes at cost of larger model and more iterations of training.
For very large datasets, costs of using dropout and larger models may outweigh benefits of regularisation.

For sparse datasets (extremely few labelled training examples available), Bayesian neural networks outperform dropout. When additional unlabelled data is available, unsupervised learning may gain advantage over dropout.

Remark 3.2.30. *Adversarial Training*
Adversarial example may be constructed by optimisation to search for an input $\boldsymbol{x}'$ near $\boldsymbol{x}$ such that the model output is very different at $\boldsymbol{x}'$.
The primary cause is excessive linearity in some neural networks. Changing input by ϵ causes linear function with weights $\boldsymbol{w}$ to change by as much as $\epsilon \|\boldsymbol{w}\|_1$, which can be very large if $\boldsymbol{w}$ is high-dimensional.
Adversarial examples assists in semi-supervised learning. At point $\boldsymbol{x}$ that is not associated with a label, model assigns label $\hat{y}$ (may not be true label). Seek adversarial example $\boldsymbol{x}'$ that causes classifier to output label $y' \neq \hat{y}$. Classifier may then be trained to assign same label to $\boldsymbol{x}$ and $\boldsymbol{x}'$, learning function robust to small changes.

Remark 3.2.31. *Tangent Distance Algorithm*
Non-parametric nearest-neighbour algorithm, where examples on the same manifold share the same category. The nearest-neighbour distance between points $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ is the distance between manifolds M_1 and M_2. Computationally cheap alternative is to approximate M_i by its tangent plane at $\boldsymbol{x}_i$ and measure the distance between two tangents, or between a tangent plane and a point.

Remark 3.2.32. *Tangent Propagation Algorithm*
Trains neural net classifier with extra penalty to make each output $f(\boldsymbol{x})$ of neural net locally invariant to factors of variation, which correspond to movement along manifold which examples from same class concentrate. Local invariance is achieved by requiring $\nabla_{\mathbf{x}} f(\boldsymbol{x})$ to be orthogonal to known manifold tangent vectors $\boldsymbol{v}^{(i)}$ at $\boldsymbol{x}$, or equivalently the directional derivative of f at $\boldsymbol{x}$ in directions $\boldsymbol{v}^{(i)}$ be small by adding regularisation penalty:

$$\Omega(f) = \sum_i \left(\left(\nabla_{\boldsymbol{x}} f(\boldsymbol{x}) \right)^T \boldsymbol{v}^{(i)} \right)^2$$

Tangent vectors are derived a priori from effect of transformations (i.e., translation, rotation, scaling).
Note this regularises model to resist infinitesimal perturbation, and the infinitesimal approach poses difficulties for models based on rectified linear units.
Data augmentation is the non-infinitesimal version of Tangent Propagation Algorithm, where network is explicitly trained to correctly classify distinct inputs created from more than infinitesimal transformation.

Remark 3.2.33. *Double Back-propagation Algorithm*
Algorithm regularises Jacobian to be small. Require model to be invariant to all directions of change in input so long as the change is small. Adversarial training is the non-infinitesimal version of Double Back-propagation Algorithm, where inputs were found near original inputs and model is trained to produce same input.

Remark 3.2.34. *Manifold Tangent Classifier*
Eliminates need to know tangent vectors a priori. Auto-encoders may estimate manifold tangent vectors, and the classifier makes use of this technique to avoid requiring user-specified tangent vectors. The steps are:

i. Use auto-encoder to learn manifold structure by unsupervised learning
ii. Use these tangents to regularise a neural net classifier as in Tangent Propagation Algorithm.

3.2.3 Optimisation for Deep Learning

Remark 3.2.35. *Optimisation*
A cost function $J(\boldsymbol{\theta})$ is reduced to improve some performance measure P.
Optimisation algorithms include some specialisation on specific structure of machine learning objective functions.
Cost function is average over training set,

$$J(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim\hat{p}_{\text{data}}} L(f(\boldsymbol{x};\boldsymbol{\theta}), y)$$

where L is per-example loss function, $f(\boldsymbol{x};\boldsymbol{\theta})$ is predicted output given input $\boldsymbol{x}$, $\hat{p}_{\text{data}}$ is empirical distribution.
For supervised learning, y is target output.
The goal is to usually to minimise objective function where expectation is taken on data generating distribution p_{data} rather than over the finite training set:

$$J^*(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim p_{\text{data}}} L(f(\boldsymbol{x};\boldsymbol{\theta}), y)$$

Note that machine learning algorithm usually minimises a surrogate loss function, and halts when convergence criterion based on early stopping is satisfied.

Definition 3.2.36. *Empirical Risk Minimisation*
To minimise expected loss on training set, where $\hat{p}(\boldsymbol{x}, y)$ is the empirical distribution. The empirical risk is

$$\mathbb{E}_{\boldsymbol{x},y\sim\hat{p}_{\text{data}}(\boldsymbol{x},y)}[L(f(\boldsymbol{x},\boldsymbol{\theta}), y)] = \frac{1}{m}\sum_{i=1}^{m} L(f(\boldsymbol{x}^{(i)};\boldsymbol{\theta}), y^{(i)})$$

where m is number of training samples. Note that empirical risk minimisation is prone to overfitting, as models with high capacity may memorise the training set.

Remark 3.2.37. *Batch and Minibatch Algorithms*
Objective algorithms for machine learning computes each update to the parameters based on expected value of cost function using only a subset of terms of full cost function.
For maximum likelihood estimation problems, note that

$$\boldsymbol{\theta}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\text{model}}(\boldsymbol{x}^{(i)}, y^{(i)};\boldsymbol{\theta})$$
$$J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x},y\sim\hat{p}_{\text{data}}} \log p_{\text{model}}(\boldsymbol{x}, y;\boldsymbol{\theta})$$

The gradient is then

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x},y\sim\hat{p}_{\text{data}}} \nabla_{\boldsymbol{\theta}} \log\log p_{\text{model}}(\boldsymbol{x}, y;\boldsymbol{\theta})$$

If the optimisation algorithm uses the entire training set, then it is a batch gradient method.
If a single sample is used at a time, then this is a stochastic/online gradient method.
For algorithms that uses more than one but less than all samples, then this is a minibatch gradient method.

Remark 3.2.38. *Challenges: Ill-Conditioning of Hessian Matrix H*
May cause stochastic gradient descent to be stuck where very small step increases the cost function.
Note that a gradient descent step of $-\boldsymbol{\epsilon g}$ will add

$$\frac{1}{2}\epsilon^2 \boldsymbol{g}^T \boldsymbol{H}\boldsymbol{g} - \epsilon \boldsymbol{g}^T \boldsymbol{g}$$

to the cost. This will be a problem when $\frac{1}{2}\epsilon^2 \boldsymbol{g}^T \boldsymbol{H}\boldsymbol{g} > \epsilon \boldsymbol{g}^T \boldsymbol{g}$.
Monitor squared gradient norm $\boldsymbol{g}^T\boldsymbol{g}$ and $\boldsymbol{g}^T\boldsymbol{H}\boldsymbol{g}$ term. When ill-conditioning happens, gradient norm does not shrink significantly but $\boldsymbol{g}^T\boldsymbol{H}\boldsymbol{g}$ term grows more than order of magnitude, and learning rate shrinks.

Remark 3.2.39. *Challenges: Local Minimums*
Neural networks and models with multiple parametrised latent variables have multiple local minima due to *model identifiability* problem. Model is identifiable if a sufficiently large training set can rule out all but one setting of model parameters. Models with latent variables are not identifiable as equivalent models are obtained by exchanging latent variables with each other (*weight space symmetry*). If there are m layers with n units each, then there are $n!^m$ ways to arrange the hidden units, making the model unidentifiable.
There may be an extremely large or uncountably infinite amount of local minima in a cost function. If the local minima have a high cost compared to global minimum, then this is problematic.

Remark 3.2.40. *Challenges: Plateaus, Saddle Points, Flat Regions*
For a function $f : \mathbb{R}^n \to \mathbb{R}$, expected ratio of saddle points to local minima grows exponentially with n. Hessian

matrix at a saddle point has a mixture of positive and negative eigenvalues.
Degenerate regions with flat gradient may correspond to high value of objective function.

Remark 3.2.41. *Challenges: Cliffs and Exploding Gradients*
On extremely steep cliff structure, gradient update step may move parameters extremely far, losing most of the optimisation work done previously. The consequences may be mitigated using gradient clipping heuristics, which reduces the step size to be small enough.

Remark 3.2.42. *Challenges: Long-Term Dependencies*
Occurs when computational graph becomes very deep. Let a computational graph contain a path that repeatedly multiply a matrix $\boldsymbol{W}$. After t steps, this is $\boldsymbol{W}^t$. Let the eigen-decomposition of $\boldsymbol{W}$ be $\boldsymbol{W} = \boldsymbol{V}\text{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1}$. Then we can see that $\boldsymbol{W}^t = (\boldsymbol{V}\text{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1})^t = \boldsymbol{V}\text{diag}(\lambda)^t\boldsymbol{V}^{-1}$.
There will be a vanishing and exploding gradient problem if gradients on the graph are scaled according to $\text{diag}(\boldsymbol{\lambda})^t$. Note that recurrent networks uses same $\boldsymbol{W}$ at each time step, hence may face the problem.

Remark 3.2.43. *Challenges: Inexact Gradients*
Objective function for minimisation is intractable due to noisy/biased estimate of these quantities; in this case its gradient is intractable as well. Choose surrogate loss function that is easier to approximate than true loss.

Remark 3.2.44. *Challenges: Poor Correspondence between Local and Global Structure*
As gradient descent algorithms make small local moves, optimisation on local downhill moves may fail if local surface does not point toward the global solution. In higher dimensional space, learning algorithms may incur high costs or long training time to circumnavigate the manifold. Hence to find good initial points for problems.

Remark 3.2.45. *Stochastic Gradient Descent (SGD)*
Crucial parameter is the learning rate, which may be changed over time; at iteration k this is ϵ_k.
The SGD gradient estimation introduces a source of noise that does not vanish even at minimum. A sufficient condition to guarantee converge of SGD is

$$\sum_{k=1}^{\infty} \epsilon_k = \infty, \quad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty$$

In practice, it is common to decay learning rate linearly until iteration τ:

$$\epsilon_k = (1-\alpha)\epsilon_0 + \alpha\epsilon_\tau, \quad \alpha = \frac{k}{\tau}$$

After iteration τ, leave ϵ constant.
Compute time per update does not growth with number of training examples.

Algorithm 3: Stochastic Gradient Descent (SGD) Update at Training Iteration k

Require:
Learning rate ϵ_k, Initial parameter $\boldsymbol{\theta}$

while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient estimate $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon\hat{\boldsymbol{g}}$
end while

Remark 3.2.46. *Momentum Algorithm*
Algorithm introduces variable $\boldsymbol{v}$ that is set to an exponentially decaying average of negative gradient.
Hyperparameter $\alpha \in [0, 1)$ determines how quickly the contributions of previous gradients exponentially decay.
The update rule is given by

$$\boldsymbol{v} \leftarrow \alpha\boldsymbol{v} - \epsilon\nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^{m} L(\boldsymbol{f}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)}) \right)$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$$

The larger α is relative to ϵ, the more previous gradients affect current direction.
If gradient is $\boldsymbol{g}$, it will accelerate in direction of $-\boldsymbol{g}$ until reaching terminal velocity where size of each step is

$$\frac{\epsilon \|\boldsymbol{g}\|}{1-\alpha}$$

Algorithm 4: Stochastic Gradient Descent (SGD) with Momentum

Require:
Learning rate ϵ_k, Momentum parameter α

while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient estimate $\boldsymbol{g} \leftarrow +\frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

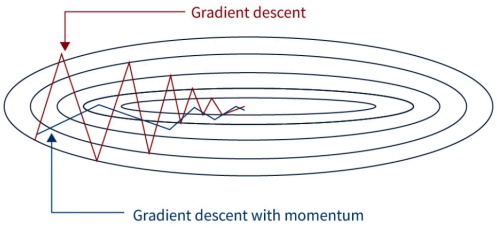


Figure 4: Stochastic Gradient Descent (SGD) Algorithm with and without Momentum

Remark 3.2.47. *Nesterov Momentum*
Inspired by Nesterov's accelerated gradient method, the update rules are:

$$\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^{m} L(\boldsymbol{f}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta} + \alpha \boldsymbol{v}), \boldsymbol{y}^{(i)}) \right)$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$$

Note, gradient is evaluated after current velocity is applied.
Hence, Nesterov momentum is adding a correction factor to the standard method of momentum.

Algorithm 5: Stochastic Gradient Descent (SGD) with Nesterov Momentum

Require:
Learning rate ϵ_k, Momentum parameter α
Initial parameter $\boldsymbol{\theta}$, Initial velocity $\boldsymbol{v}$

while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Apply interim update: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{v}$
Compute gradient (at interim point): $\boldsymbol{g} \leftarrow \frac{1}{m}\nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

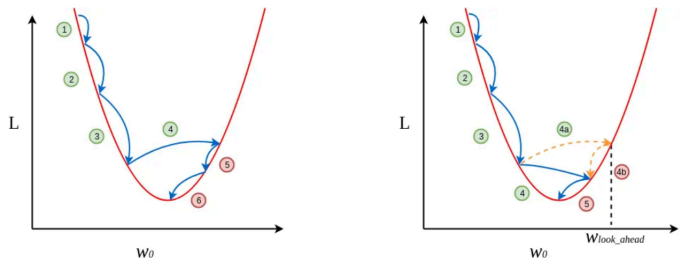


Figure 5: Momentum vs Nesterov Gradient Descent

Remark 3.2.48. *Parameter Initialisation Challenges*
Training deep neural networks is difficult as most algorithms are strongly affected by choice of initialisation.

Designing initialisation strategies is difficult as neural network optimisation is not yet well understood, as the nice properties from initialisation may not be preserved as training proceeds.
Some initial points may be beneficial for optimisation, but detrimental for generalisation.

Remark 3.2.49. *Parameter Initialisation Strategies*
Initial parameters need to 'break symmetry' between different units. May explicitly search for large set of basis functions mutually different from each other, but this may incur noticeable computational cost.
Random initialisation from high-entropy distribution over high-dimensional space is computationally cheaper.
Larger initial weights will yield stronger symmetry breaking effect and avoid redundant units. However, if weights are set too large, may result in chaos (extreme sensitivity to small perturbations of input). Exploding gradient may be mitigated by gradient clipping. The bias (i.e., parameters encoding conditional variance of a prediction) for each unit is set to heuristically chosen constraints.

Remark 3.2.50. *Weight Initialisation Heuristics*
Initialising parameters of neural network $\boldsymbol{\theta}$ to $\boldsymbol{\theta}_0$ is similar to imposing a Gaussian prior $p(\boldsymbol{\theta})$ with mean $\boldsymbol{\theta}_0$, hence it make sense to choose $\boldsymbol{\theta}_0$ near 0. Initialisation of $\boldsymbol{\theta}_0$ to large values means prior specifies which units should interact with each other, and how they should interact.
For a fully connected neural network with m inputs and n outputs, weights may be initialised by:

i. Sampling from $U\left(-\frac{1}{\sqrt{m}}, \frac{1}{\sqrt{m}}\right)$
ii. Sampling from normalised initialisation $W_{i,j} \sim U\left(-\sqrt{\frac{6}{m+n}}, \sqrt{\frac{6}{m+n}}\right)$ (Glorot and Bengio). Compromise between goal of initialising all layers to have same activation variance and that of same gradient variance. Derived from assumption that network is chain of matrix multiplications with no nonlinearities.
iii. Sampling from random orthogonal matrices, with scaling or gain factor g that accounts for nonlinearity at each layer, with specific values for different types of nonlinear activation functions. Assumes neural network is sequence of matrix multiplications without nonlinearities. Guarantees the total number of training iterations required to reach convergence is independent of depth.
Increasing scaling factor g pushes network toward regime where activations increase in norm in forward propagation, and gradients increase in norm in backward propagation.
For scaling rules that set all of initial weights to have same standard deviation, individual weight becomes extremely small when layers become large.
iv. Sparse initialisation (Martens), where each unit is initialised to have exactly k non-zero weight. Keep total number of input to unit independent from number of inputs m without making magnitude of individual weight elements shrink with m.
Imposes very strong prior on weights that are chosen to have large Gaussian values.

If computational resources allow, treat initial scale of weights for each layer as a hyper-parameter, and use hyper-parameter search algorithm. Choice of dense or sparse initialisation may also be a hyper-parameter.

Remark 3.2.51. *Bias Initialisation Heuristics*
Setting bias to zero is compatible with most weight initialisation schemes. The few other situations are:

i. If bias is for output unit, initialise bias to obtain right marginal statistics of the output. Assume initial weights are small enough that output of unit is determined only by bias. The bias will be inverse of activation function applied to marginal statistics of the output in training set.
If output is a highly skewed distribution over classes with marginal probability of class i given by element c_i of some vector $\boldsymbol{c}$, then bias vector $\boldsymbol{b}$ is such that $\text{softmax}(\boldsymbol{b}) = \boldsymbol{c}$. The models have output layers that should resemble input data $\boldsymbol{x}$, hence bias should match marginal distribution over $\boldsymbol{x}$.
ii. If bias is to be set to avoid causing too much saturation at initialisation (i.e., with ReLU). Not compatible with weight initialisation schemes that do not expect strong input form bias (i.e., with random walk init).
iii. If a unit controls whether other units are able to participate in a function. There is unit with output u and another unit $h \in [0, 1]$, multiplied together to produce output uh. Then h is a gate that determines if $uh \approx u$ or $uh \approx 0$. Hence, to set bias for h such that $h \approx 1$ at initialisation, else u does not learn.

Remark 3.2.52. *Variance or Precision Parameter Initialisation Heuristics*
For linear regression with conditional variance estimate using model

$$p(y \mid \boldsymbol{x}) = \mathcal{N}(y \mid \boldsymbol{w}^T\boldsymbol{x} + b, 1/\beta)$$

where β is a precision parameter. The variance of precision parameters may be initialised to 1.
Another approach is to assume initial weights are close enough to zero, and bias may be set while ignoring effect of the weights, then set bias to produce the correct marginal mean of the output, and set variance parameters to the marginal variance of the output in the training set.

Remark 3.2.53. *Model Parameter Initialisation with Machine Learning*
Initialise supervised model with parameters learned by unsupervised model trained on same inputs. Alternatively, perform supervised training on a related task, which may yield initialisation that offers faster convergence than random initialisation. Information on distribution is encoded into initial parameters of model.

Remark 3.2.54. *Adaptive Gradient (AdaGrad) Algorithm*
Algorithm individually adapts learning rates of all model parameters by scaling them inversely proportional to square root of sum of all of their historical squared values. Has greater progress in more gently sloped directions of parameter space. On deep neural networks, accumulation of squared gradients from beginning of training can result in premature and excessive decrease in effective learning rate.

Algorithm 6: Adaptive Gradient (AdaGrad) Algorithm

Require:
Global Learning rate ϵ
Initial parameter $\boldsymbol{\theta}$
Small constant δ, perhaps 10^{-7} for numerical stability

Initialise gradient accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$
Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$ (Division and square root applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Remark 3.2.55. *Root Mean Squared Propagation (RMSProp) Algorithm*
Modifies AdaGrad to perform better in non-convex setting by changing gradient accumulation into exponentially weighted moving average, which discards history from extreme past so it can converge rapidly after finding a convex bowl. Use of moving average introduces a new hyper-parameter ρ that controls length scale of the moving average. RMSProp is effective and practical for deep neural networks.

Algorithm 7: Root Mean Squared Propagation (RMSProp) Algorithm

Require:
Global Learning rate ϵ
Decay rate ρ
Initial parameter $\boldsymbol{\theta}$
Small constant δ, perhaps 10^{-6} to stabilise division by small numbers

Initialise accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho\boldsymbol{r} + (1-\rho)\boldsymbol{g} \odot \boldsymbol{g}$
Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$ ($\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Algorithm 8: Root Mean Squared Propagation (RMSProp) Algorithm

Require:
Global Learning rate ϵ
Decay rate ρ
Momentum coefficient α
Initial parameter $\boldsymbol{\theta}$
Initial velocity $\boldsymbol{v}$

Initialise accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute interim update $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha\boldsymbol{v}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$

Accumulate gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1-\rho)\boldsymbol{g} \odot \boldsymbol{g}$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \frac{\epsilon}{\sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$ ($\frac{1}{\sqrt{\boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

Remark 3.2.56. *Adaptive Moment Estimation (Adam) Algorithm*
Variant on combination of RMSProp and momentum. Note, momentum is incorporated directly as an estimate of first order moment of gradient; bias corrections are made to estimates of both first-order moments and second-order moments to account for initialisation at origin.
Algorithm is fairly robust to choice of hyper-parameters.

Algorithm 9: Adaptive Moment Estimation (Adam) Algorithm

Require:
Step size ϵ (Suggested default: 0.001)
Exponential decay rates for moment estimates ρ_1, ρ_2 in $[0, 1)$ (Suggested defaults: 0.9 and 0.999)
Small constant δ used for numerical stabilisation (Suggested default: 10^{-8})
Initial parameters $\boldsymbol{\theta}$

Initialise 1st and 2nd momentum variables $\boldsymbol{s} = \boldsymbol{0}$, $\boldsymbol{r} = \boldsymbol{0}$
Initialise time step $t = 0$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Update time step $t \leftarrow t + 1$
Update biased first and second moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1-\rho_1)\boldsymbol{g}$ and $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1-\rho_2)\boldsymbol{g} \odot \boldsymbol{g}$
Correct bias in first and second moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1-\rho_1^t}$ and $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1-\rho_2^t}$
Compute update: $\Delta\boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (Operations applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Remark 3.2.57. *Newton's Method*
Based on second-order Taylor's series approximating $J(\boldsymbol{\theta})$ near the point $\boldsymbol{\theta}_0$, ignoring higher derivatives

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^T \boldsymbol{H} (\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

where $\boldsymbol{H}$ is the Hessian of J with respect to $\boldsymbol{\theta}$ evaluated at $\boldsymbol{\theta}_0$.
Solving for critical point will get the Newton parameter update rule

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Note that for deep learning, the surface of objective function is non-convex. Regularising the Hessian will solve many of the issues such as saddle points etc.

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - [H(f(\boldsymbol{\theta}_0)) + \alpha \boldsymbol{I}]^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Due to requirement for inversion of the $k \times k$ matrix with time complexity $O(k^3)$ at every training iteration, only networks with very small number of parameters may be trained with this method.

Algorithm 10: Newton's Method with Objective $J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$

Require:
Initial parameter $\boldsymbol{\theta}_0$
Training set of m examples

while stopping criterion not met **do**
Compute gradient $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute Hessian: $\boldsymbol{H} \leftarrow \frac{1}{m} \nabla^2_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute Hessian Inverse: $\boldsymbol{H}^{-1}$
Compute update: $\Delta\boldsymbol{\theta} = -\boldsymbol{H}^{-1}\boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Remark 3.2.58. *Conjugate Gradient Method*
Method effectively avoids computation of inversion Hessian by descending conjugate directions, which is a search direction conjugate to the previous line search direction (will not undo progress made in that direction).
Let current and previous search directions be $\boldsymbol{d}_t, \boldsymbol{d}_{t-1}$. At training iteration t, the next search direction $\boldsymbol{d}_t$ is

$$\boldsymbol{d}_t = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) + \beta_t \boldsymbol{d}_{t-1}$$

note that β_t may be computed via Fletcher Reeves:

$$\beta_t = \frac{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)}{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})}$$

or may be computed with Polak-Ribiere:

$$\beta_t = \frac{(\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t) - \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1}))^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)}{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})}$$

The non-linear conjugate gradient method is a variation which uses occasional resets where the method is restarted with line search along with the unaltered gradient.

Algorithm 11: Conjugate Gradient Method

Require:
Initial parameter $\boldsymbol{\theta}_0$
Training set of m examples
Initialise $\boldsymbol{\rho}_0 = 0$
Initialise $g_0 = 0$
Initialise $t = 1$

while stopping criterion not met **do**
Initialise the gradient $\boldsymbol{g}_t = \boldsymbol{0}$
Compute gradient $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute Polak-Ribiere metric: $\beta_t = \frac{(\boldsymbol{g}_t - \boldsymbol{g}_{t-1})^T \boldsymbol{g}_t}{\boldsymbol{g}_{t-1}^T \boldsymbol{g}_{t-1}}$
(Nonlinear conjugate gradient: optionally reset $\beta_t = 0$, i.e., if t is a multiple of some constant k)
Compute search direction: $\boldsymbol{\rho}_t = -\boldsymbol{g}_t + \beta_t \boldsymbol{\rho}_{t-1}$
Perform line search to find: $\epsilon^* = \arg\min_\epsilon \frac{1}{m} \sum_{i=1}^m L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}_t + \epsilon \boldsymbol{\rho}_t), \boldsymbol{y}^{(i)})$
(On truly quadratic cost function, analytically solve for ϵ^*)
Apply update: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \epsilon^{\theta} \boldsymbol{\rho}_t$
Apply time step $t \leftarrow t + 1$
end while

Remark 3.2.59. *Nonlinear Conjugate Gradients*
Method of conjugate gradients is still applicable for non-linear objective function.
Conjugate directions are no longer assumed to remain at minimum of the objective for previous directions. Has occasional resets where method of conjugate gradients is restarted with line search along unaltered gradient.

Remark 3.2.60. *Broyden-Fletcher-Goldfarb-Shanno (BGFS) Algorithm*
Similar to conjugate gradient, but uses more direct approach to approximation of newton's update.
Method approximate the inverse of hessian $\boldsymbol{H}$ with matrix $\boldsymbol{M}_t$, iteratively refined by low rank updates to better approximate $\boldsymbol{H}^{-1}$. Direction of gradient descent is then determined by $\boldsymbol{\rho}_t = \boldsymbol{M}_t \boldsymbol{g}_t$. Line search is performed in this direction to determine step size ϵ^*. Final update of parameter is then $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \epsilon^* \boldsymbol{\rho}_t$.
Note, BGFS store matrix $\boldsymbol{M}$ which requires $O(n^2)$ memory, making it impractical for most modern models.

Remark 3.2.61. *Limited Memory BFGS (L-BFGS) Algorithm*
Memory costs of BFGS may be decreased by not storing complete inverse Hessian approximation $\boldsymbol{M}$. L-BFGS computes approximation $\boldsymbol{M}$ with same method as BFGS, but assumes at beginning that $\boldsymbol{M}^{(t-1)}$ is the identity matrix, rather than storing the approximation between steps.
If used with exact line searches, directions defined by L-BFGS are mutually conjugate, and this procedure remains well-behaved when minimum of line search is reached only approximately.
L-BFGS may be generalised to store some vectors used to update $\boldsymbol{M}$ at each time step, costing $O(n)$ per step.

Remark 3.2.62. *Meta-Algorithm: Batch Normalisation*
An update from optimisation algorithm may result in multiple-order effects, and in very deep networks, even higher-order interactions can be significant.

Batch normalisation provides an elegant way of re-parametrising any deep network, which reduces the problem of coordinating updates across multiple layers. Batch normalisation may be applied to any input/hidden layer. Let $\boldsymbol{H}$ be minibatch of activations of layer to normalise, with activations for each example appearing in a row. To normalise $\boldsymbol{H}$, replace it with $\boldsymbol{H}' = \frac{\boldsymbol{H}-\boldsymbol{\mu}}{\boldsymbol{\sigma}}$ where $\boldsymbol{\mu}$ is a vector of mean of each unit and σ is a vector of standard deviation of each unit. Then H_{ij} is normalised by subtracting μ_j and dividing σ_j. Rest of network then operates on $\boldsymbol{H}'$ just as in original network on $\boldsymbol{H}$.
At training, the mean and standard deviations are computed as

$$\boldsymbol{\mu} = \frac{1}{m}\sum_i \boldsymbol{H}_{i,;}, \quad \boldsymbol{\sigma} = \sqrt{\delta + \frac{1}{m}\sum_i (\boldsymbol{H}-\boldsymbol{\mu})_i^2}$$

where δ is a small positive value (i.e., 10^{-8}) to avoid undefined gradient of $\sqrt{z}$ at $z = 0$.
These are then back-propagated to normalise $\boldsymbol{H}$, hence gradient will never increase the standard deviation or mean of h_i. Batch normalisation re-parametrises the model to make some units always standardised.
At test time, μ and σ may be replaced by running averages from training time. This allows model to be evaluated on a single example, without needing to use definitions of μ and $\boldsymbol{\sigma}$ that depends on entire minibatch.

Remark 3.2.63. *Batch Normalisation but Maintain Network Express Power*
Replace batch of hidden activations $\boldsymbol{H}$ with $\boldsymbol{\gamma}\boldsymbol{H}' + \boldsymbol{\beta}$ rather than $\boldsymbol{H}'$, where $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ are learned parameters to allow new variable to have any mean and standard deviation. With original $\boldsymbol{H}'$, mean of $\boldsymbol{H}$ was determined by complicated interaction between parameters in layers below $\boldsymbol{H}$. With $\boldsymbol{\gamma}\boldsymbol{H}' + \boldsymbol{\beta}$, the mean is determined solely by $\boldsymbol{\beta}$, which is much easier to learn with gradient descent.
Most neural network layers are of form $\phi(\boldsymbol{X}\boldsymbol{W} + \boldsymbol{b})$, where ϕ is some fixed nonlinear activation function. Then $\boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}$) should be replaced by normalised version of $\boldsymbol{X}\boldsymbol{W}$, and bias term omitted as it becomes redundant with β parameter applied from batch normalisation re-parametrisation. Statistics of input are non-Gaussian and less amenable to standardisation by linear operations.

Remark 3.2.64. *Meta-Algorithm: Coordinate Descent*
Solving optimisation problems by minimising the objective function $f(\boldsymbol{x})$ with respect to a single variable x_i, and rapidly cycling through all variables. *Block coordinate descent* refers to minimising with respect to a subset of variables simultaneously.
Coordinate descent works when different variables can be clearly separated into groups that play relatively isolated roles, or when optimisation with respect to one group of variables is significantly more efficient than optimisation with respect to all of the variables.

Example 3.2.65. *Sparse Encoding with Coordinate Descent*
Consider the cost function

$$J(\boldsymbol{H}, \boldsymbol{W}) = \sum_{i,j} |H_{ij}| + \sum_{i,j} (\boldsymbol{X} - \boldsymbol{W}^T\boldsymbol{H})_{i,j}^2$$

To find weight matrix $\boldsymbol{W}$ that can linearly decode a matrix of activation values $\boldsymbol{H}$ to reconstruct training set $\boldsymbol{X}$. Most applications also include weight decay or constraint on norms of columns of $\boldsymbol{W}$ to prevent the pathological solution with extremely small $\boldsymbol{H}$ and large $\boldsymbol{W}$.
Note J is not convex, but inputs may be divided into the dictionary parameters $\boldsymbol{W}$ and the code representations $\boldsymbol{H}$. Minimising object function with respect to either set is a convex problem.

Remark 3.2.66. *Polyak Averaging*
Average several points in trajectory through parameter space visited by optimisation algorithm.
If t iterations of gradient descent visit points $\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(t)}$, then the output is $\hat{\boldsymbol{\theta}}^{(t)} = \frac{1}{t}\sum_i \boldsymbol{\theta}^{(i)}$.
The optimisation algorithm may leap back and forth across a valley several times without visiting the minimum point, and the average of all points on either side of the valley should be close to minimum value.
This is developed solely for stochastic gradient descent (SGD) to stabilise the estimates (Granziol et al. (2020)).
For non-convex problems, an exponentially decaying running average is used:

$$\hat{\boldsymbol{\theta}}^{(t)} = \alpha\hat{\boldsymbol{\theta}}^{(t-1)} + (1-\alpha)\boldsymbol{\theta}^{(t)}$$

Remark 3.2.67. *Supervised Pre-training*
Pre-training refers to training simple models on simple tasks before training the desired model on desired task. Greedy algorithms break problem into many components, then solve for optimal version of each component in isolation, which is must cheaper than algorithms that solve for best joint solution. Fine-tuning on joint optimisation algorithm searches for optimal solution to the full problem.
Greedy supervised training breaks supervised learning problems into other simpler supervised learning problems.

Transfer learning refers to pre-training a deep network on a set of tasks, then initialise a same-size network with first k layers of first net. The layers of second networks (upper layers initialised randomly) are then jointly trained to perform a different set of tasks with fewer training examples than for first set of tasks.
FitNets approach begins by training a network with low depth and high width, which becomes a teacher for a second network (the student), which is much deeper and thinner. Student network is trained to not only predict the output on original task, but also on value of middle layer of teacher network.

Remark 3.2.68. *Neural Network Design Choice: Linear Transformation*
Model units have generally moved towards using more linear functions, which allow model to have nice properties that make optimisation easier. The model's local gradient information corresponds reasonably well to moving towards a distant solution.

Remark 3.2.69. *Neural Network Design Choice: Linear Paths and Skip Connections*
Linear paths and skip connections between layers reduce length of shortest path from lower layer's parameters to the output, mitigating vanishing gradient problem.
Adding extra copies of output that are attached to intermediate hidden layers is an alternative to pre-training, where these 'auxiliary heads' are trained to perform the same task as primary output at the top of the network to ensure that the lower layers receive a large gradient. After training, these may be discarded.

Remark 3.2.70. *Continuation Methods*
Family of strategies that can make optimisation easier by choosing initial points to ensure that local optimisation spends most of its time in well-behaved regions. Method constructs series of objective functions $\{J^{(0)}, \dots, J^{(n)}\}$ which are increasingly difficult to minimise (well-behaved over less regions of $\boldsymbol{\theta}$ space) for the true cost function $J(\boldsymbol{\theta})$. The cost functions are designed such that a solution to one is a good initial point for the next.
Continuation methods aim to reach a global minimum despite many local minima. Easier cost functions are constructed through the blurring operation, which approximates

$$J^{(i)}(\boldsymbol{\theta}) = \mathbb{E}_{\theta' \sim \mathcal{N}(\boldsymbol{\theta}'; \boldsymbol{\theta}, \sigma^{(i)2})} J(\boldsymbol{\theta})'$$

via sampling. Some non-convex functions become approximately convex when blurred, but preserved enough information about local global minimum. However, approach might not work when

i. so many incremental cost functions are required that cost of entire procedure remains high;
ii. NP-hard optimisation problems remain NP-hard, even when continuation methods are applicable;
iii. function might not become convex, no matter how much it is blurred;
iv. function may become convex due to blurring, but minimum of this blurred function may track a local rather than global minimum of original cost function.

Remark 3.2.71. *Curriculum Learning*
Planning a learning process to begin by learning simple concepts, and progress to learning more complex topics that depend on these simpler concepts. Earlier objective functions $J^{(i)}$ are made easier by increasing the influence of simpler examples.
Stochastic curriculum is a variation of the approach where a random mix of easy and difficult examples is always presented to the learner, but average proportion of more difficult examples is gradually increased.

3.2.4 Convolutional Neural Networks

Definition 3.2.72. *Convolutional Neural Networks (CNNs)*
Neural network for processing data with a known, grid-like topology. Uses convolution instead of general matrix multiplication in at least one of their layers.

Definition 3.2.73. *Convolution*
The convolution of two functions f and g, denoted by $*$, is the integral of product of two functions after one is reflected about the y-axis and shifted.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

The first argument $f(\tau)$ is the input, and second argument $g(t-\tau)$ is the kernel. The output is feature map.

Definition 3.2.74. *Discrete Convolution*
For complex-valued functions f and g defined on set of $\mathbb{Z}$ integers, the discrete convolution of f and g is

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n-m]$$

If function g_N is periodic with period N, then the convolution is also periodic and identical to

$$(f * g_N)[n] = \sum_{m=0}^{N-1} \left(\sum_{k=-\infty}^{\infty} f[m+kN] \right) g_N[n-m]$$

Note that convolution is commutative, as the kernel is flipped relative to the input (as m increases, the index into input increases, but index into kernel decreases), but this is not an important property of neural network implementation. Neural network libraries implement cross-correlation instead.

Definition 3.2.75. *Cross-Correlation*
A measure of similarity of two series as a function of the displacement of one relative to the other. Also known as *sliding inner or dot product*. This is defined as

$$(f \star g)(\tau) = \int_{-\infty}^{\infty} \overline{f(\tau)}g(t+\tau)dt$$

This is related to convolution by

$$[f(t) \star gt](t) = [\overline{f(-t)} * g(t)](t)$$

Remark 3.2.76. *Motivation: Sparse Interactions*
The kernel is smaller than input, but is able to detect meaningful features. Hence fewer parameters are to be stored, reducing memory requirements of the model. Units in deeper layers may indirectly interact with a larger portion of the input, hence efficiently describe complicated interactions between many variables.

Remark 3.2.77. *Motivation: Parameter Sharing*
Each member of kernel is used at every position of the input except at boundary, which depends on design decisions regarding the boundary. Only one set of parameter is required to be learned. The runtime of forward propagation is at $O(k \times n)$ for $k \lll m$ connections for each input, given m input and n outputs.

Remark 3.2.78. *Motivation: Equivariant Representations*
An *equivariant* function is such that if the input changes, the output changes in the same way.
Applying a transformation, then convolution, is equivalent to applying convolution, then the transformation.

Remark 3.2.79. *Typical CNN Layers*
The layers of a typical CNN are:

i. Convolution: layer performs several convolutions in parallel to produce set of linear activations
ii. Detector: each linear activation is run through a non-linear activation function
iii. Pooling: modify the output of the layer further with summary statistics of nearby outputs

Example 3.2.80. *Example Types of Pooling*

i. Max Pooling: reports maximum output for each region of the feature map

ii. Average Pooling: reports average for each region of the feature map

iii. L^2 Pooling: calculates Euclidean norm (L^2 norm) for each region of the feature map

Remark 3.2.81. *Pooling on Invariance*
Pooling makes the representation approximately invariant to small translations of input.
Use of pooling is analogous to adding an infinitely strong prior that the function the layer learns must be invariant to small translations. As pooling summarises responses over whole neighbourhood, fewer pooling units than detector units may be used. To handle inputs of varying size, the size of offset between pooling regions may be changed so that the classification layer always receives the same number of summary statistics.

Remark 3.2.82. *Multi-Channel Operations*
Not commutative unless each operation has the same number of output and input channels.
Assume there exists 4-D kernel tensor $\mathcal{K}$ with element $K_{i,j,k,l}$ giving connection between strength between a unit in channel i of the output and a unit in channel j of the input, with offset of k rows and l columns between the output unit and input unit. Assume input consists of observed data $\mathcal{V}$ with element $V_{i,j,k}$ giving the value of input unit within channel i at row j and column k. Assume output consists of $\mathcal{Z}$ in same format as $\mathcal{V}$. If $\mathbb{Z}$ is produced by convolving $\mathcal{K}$ across $\mathcal{V}$ without flipping $\mathcal{K}$, then

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$

where summation over l, m, n is over all values for which tensor indexing operations inside summation is valid.

Remark 3.2.83. *Downsampled Convolution*
To reduce computational cost by sampling only every s pixels (*stride*) in each direction of output, then

$$Z_{i,j,k} = c(\mathcal{K}, \mathcal{V}, s)_{i,j,k} = \sum_{l,m,n} [\mathcal{V}_{l,(j-1)\times s+m,(k-1)\times s+n} \mathcal{K}_{i,l,m,n}]$$

Remark 3.2.84. *Zero Padding*
Zero-padding is adding zeroes to end of input sequence $\mathcal{V}$ so that the total number of samples is equal to the next higher multiple of two.

i. Valid Convolution: no zero-padding used. If input image has width m and kernel has width k, the output will be width $m - k + 1$. Rate of shrinkage can be dramatic if kernel used are large.

ii. Same Convolution: just enough zero-padding is added to keep size of output equal to size of input. Border pixels may be underrepresented in the model.

iii. Full Convolution: zeroes are added for every pixel to be visited k times in each direction, resulting in an input image of width $m + k - 1$.

Remark 3.2.85. *Locally Connected Layers (Unshared Convolution)*
Similar operation to discrete convolution with small kernel, but without parameter sharing. Adjacency matrix of MLP is the same, but every connection has its own weight as specified by 6-D tensor $\mathcal{W}$, where the indices of $\mathcal{W}$ are the output channel i, the output row j, the output column k, the input channel l, the row offset within the the input m, and the column offset within the input n. The liner part of the layer is then

$$Z_{i,j,k} = \sum_{l,m,n} [\mathcal{V}_{l,j+m-1,k+n-1} w_{i,j,k,l,m,n}]$$

Locally connected layers are useful when it is known that each feature should be function of a small part of space but would not occur across all of space.

Remark 3.2.86. *Tiled Convolution*
Compromise between convolutional layer and locally connected layer. A set of kernels will be rotated through space. Immediate neighbouring locations will have different filters, but memory requirements for storing the parameters will increase only by a factor of the size of this set of kernels.
Let k be a 6-D tensor, where two of the dimensions correspond to different locations in output matrix. Output locations cycle through a set of t different choices of kernel stack in each direction.

$$Z_{i,j,k} = \sum_{l,m,n} \mathcal{V}_{l,j+m-1,k+n-1} \mathcal{K}_{i,l,m,n,j \bmod t+1,k \bmod t+1}$$

4 Time Series Analysis

4.1 Time Series Analysis

Based on the books by James Douglas Hamilton (1994), and ...

4.1.1 Stationary Time Series

Definition 4.1.1. A *linear first-order difference equation* is defined as

$$y_t = \phi y_{t-1} + w_t$$

where y_t is the target variable (with y_{t-1} the lag 1 variable), w_t is an input variable at time t

The difference equation may be solved by recursive substitution to arrive at

$$y_t = \phi^{t+1} y_{-1} + \phi^t w_0 + \phi^{t-1} w_1 + \phi^{t-2} w_2 + \cdots + \phi w_{t-1} + w_t$$

The *dynamic multiplier* calculates effect of w_t on y_{t+j}, and is given by

$$\frac{\partial y_{t+j}}{\partial w_t} = \phi^j$$

If $|\phi| < 1$, the system is stable. If $|\phi| > 1$, then the system is explosive.

We may generalise the process to *p-th order difference equation*, i.e.,

$$y_t = \phi_t y_{t-1} + \phi_2 t_{t-2} + \cdots + \phi_p y_{t-p} + w_t$$

We may rewrite this as a first-order difference equation in a $(p \times 1)$ vector $\boldsymbol{\xi}_t$:

$$\boldsymbol{\xi}_t = \begin{bmatrix} y_t \\ y_{t-1} \\ \vdots \\ y_{t-p+1} \end{bmatrix}$$

Define the $(p \times p)$ matrix $\boldsymbol{F}$ by

$$\boldsymbol{F} = \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 & \cdots & \phi_{p-1} & \phi_p \\ 1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix}$$

Finally, define the $(p \times 1)$ vector $\boldsymbol{v}_t$ by

$$\boldsymbol{v}_t = \begin{bmatrix} w_t \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Then the system of p equations $\boldsymbol{\xi}_t = \boldsymbol{F}\boldsymbol{\xi}_{t-1} + \boldsymbol{v}_t$ is identical to the first order difference equation, and by recursive substitution, we have the following for the case of p-th order difference equation:

$$\boldsymbol{\xi}_{t+j} = \boldsymbol{F}^{j+1}\boldsymbol{\xi}_{t-1} + \boldsymbol{F}^j \boldsymbol{v}_t + \boldsymbol{F}^{j-1}\boldsymbol{v}_{t+1} + \cdots + \boldsymbol{F}\boldsymbol{v}_{t+j-1} + \boldsymbol{v}_{t+j}$$

Hence the dynamic multiplier is then $\frac{\partial y_{t+j}}{\partial w_t} f_{11}^{(j)}$, where $f_{11}^{(j)}$ is the $(1,1)$ element of $\boldsymbol{F}^j$.

4.1.2 Univariate Time Series Models

Trading activities through an exchange can be described by a sequence of time stamps ('ticks') $t_0 < t_1 < \cdots < t_n$, and 'marks' y_i at time t_i, where t_i denote market open, t_n denote market close. The marks y_i is a characteristic of the order book at time of ith activity. Events with marks associated with the ticks can be described mathematically as a marked point process.

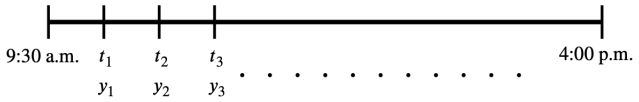


Figure 6: Ticks and Marks

A data aggregation method is where aggregation is conducted when there is a change in marker. An alternative aggregation method would be to divide the time span T for exchange hours into K intervals, so regularly spaced intervals are of size $\Delta t = T/K$. The time series method to be discussed will apply to all aggregated data.

Let $p_{it} = \ln P_{it}$ denote price of ith asset at time t; $p_t = (p_{1t}, p_{2t}, \ldots, p_{nt})$ denote price vector for n assets; y_{it} denote vector of characteristics of ith asset at time t. These quantities are aggregated from high frequency data. Consider r factors $f_t = (f_{1t}, f_{2t}, \ldots, f_{rt})$ that may include market and industry factors, and asset characteristics. Trading rules can be broadly grouped as follows:

i. *Statistical Arbitrage*: $E(p_{i,t+1} \mid p_{i,t}, p_{i,t-1}, \ldots, y_{i,t}, y_{i,t-1}, \ldots)$, that predicts the price of ith stock at $t+1$ based on past trading information (also known as time series momentum)

ii. *Momentum*: $E(p_{t+1} \mid p_t, p_{t-1}, \ldots, y_t, y_{t-1}, \ldots)$, that predicts the cross-sectional momentum of a subset of stocks based on past trading characteristics. For portfolio formation and rebalancing, pairs trading

iii. *Fair Value*: $E(p_{t+1} \mid p_t, p_{t-1}, \ldots, y_t, y_{t-1}, \ldots, f_t, f_{t-1}, \ldots)$, predicts price using all relevant quantities. Factors normally include market, Fama-French; at a more macro level than timescale considered for price prediction, but may still be useful.

Hence price and volatility prediction can be formulated as a time series prediction problem. Autocorrelations and partial autocorrelations can be used to build autoregressive and ARCH models with some predictive power.

Let $Y_1, Y_2, \ldots, Y_T$ be a sequence of random variables with a joint probability distribution. A sequence of observations of stochastic process $\{Y_t, t = 1, \ldots, T\}$ is a realisation of the process.
A time series $\{Y_t\}$ is *stationary* if for every integer m, the set of variables $Y_{t_1}, Y_{t_2}, \ldots, Y_{t_m}$ depends only on the distance between times $t_1, t_2, \ldots, t_m$. Thus $E(Y_t) = \mu$, $Var(Y_t) = \sigma^2$ are constant for all t.
The *auto-covariance* function is defined as

$$\gamma(s) = Cov(Y_t, Y_{t-s}) = E[(Y_t - \mu)(Y_{t-s} - \mu)] \quad \forall s = 0, \pm 1, \ldots$$

The *auto-correlation* of process at lag s is defined as

$$\rho(s) = Corr(Y_t, Y_{t-s}) = \frac{Cov(Y_t, Y_{t-s})}{(Var(Y_t)\ Var(Y_{t-s}))^{1/2}} = \frac{\gamma(s)}{\gamma(0)}, \quad \forall s = 0, \pm 1, \ldots$$

Some examples of stationary stochastic processes are as follows:

Example 4.1.2. *(White Noise)* Sequence of discrete independent random variables (r.v.) $\{\epsilon_t\}$ with $E[\epsilon_t] = 0$ and $E[\epsilon_t^2] = \sigma^2$. Set $Y_t = \mu + \epsilon_t$, then $E[Y_t] = \mu$, $Cov(Y_t, Y_{t-s}) = Var(Y_t) = \sigma^2$ if $s = 0$, and $Cov(Y_t, Y_{t-s}) = 0$ if $s \neq 0$. Hence the process is stationary.

Example 4.1.3. *(Moving Average)* Let $\{\epsilon_t\}$ be independent r.v., with process $\{Y_t\}$ where $Y_t = \mu + \epsilon_t + \epsilon_{t_1}$ for $t = 0, 1, 2, \ldots$, and μ is constant. Then $E[Y_t] = \mu\ \forall t$, and

$$Cov(Y_t, Y_{t-s}) = \gamma(s) = \begin{cases} 2\sigma^2 & \text{if } s = 0 \\ \sigma^2 & \text{if } s = 1 \\ 0 & \text{if } s > 1 \end{cases}$$

Hence $\{Y_t\}$ is stationary, with $\rho(s) = \gamma(s)/\gamma(0)$ such that $\rho(0) = 1$, $\rho(1) = 1/2$, $\rho(s) = 0$ for $|s| > 1$.

Some examples of non-stationary stochastic processes are as follows:

Example 4.1.4. *(Random Walk with Drift)* Let $\{\epsilon_t\}$ be sequence of independent r.v. with $E[\epsilon_t] = 0$, $E[\epsilon_t^2] = \sigma^2$, and define process $\{Y_t\}$ by $Y_t = Y_{t-1} + \delta + \epsilon_t$ with $Y_0 = 0$. Then the process can be summarised as

$$Y_t = \delta t + \sum_{j=1}^{t} \epsilon_j$$

Note that $E[Y_t] = \delta t$ and $Var[Y_t] = t\sigma^2$, hence process $\{Y_t\}$ is not stationary.

Given T observations from stationary process $\{Y_t\}$, the *sample mean* is $\overline{Y} = \frac{1}{T}\sum_{t=1}^{T} Y_t$. The *sample auto-covariance* function is defined by $\hat{\gamma}(s) = \frac{1}{T}\sum_{t=1}^{T-s}(Y_t - \overline{Y})(Y_{T-s} - \overline{Y})$ for $s = 0, 1, \ldots$. The *sample auto-correlation* function (ACF) is defined as $\hat{\rho}(s) = \frac{\hat{\gamma}(s)}{\hat{\gamma}(0)} = r(s)$. The *sample variance* is $Var(\overline{Y}) = \frac{\gamma(0)}{T}\left[1 + 2\sum_{u=1}^{Y-1}\left(\frac{T-u}{T}\right)\rho(u)\right]$; note that it has to account for auto-correlations.

Definition 4.1.5. A stochastic process $\{Y_t\}$ is a *linear process* if it can be represented as

$$Y_t = \mu + \sum_{j=0}^{\infty} \Psi_j \epsilon_{t-j}$$

where ϵ_t are independent with mean 0, variance σ_ϵ^2, and $\sum_{j=0}^{\infty} < \infty$.

References

Axler, S. (2024). *Linear Algebra Done Right* (4 ed.). Undergraduate Texts in Mathematics. Cham: Springer.

Garcia, S. R. and R. A. Horn (2017). *A Second Course in Linear Algebra*. Cambridge: Cambridge University Press.

Granziol, D., X. Wan, and S. Roberts (2020). Gadam: Combining adaptivity with iterate averaging gives greater generalisation. *arXiv preprint arXiv:2003.01247*.

Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press.

Mazur, M. (2015). A step by step backpropagation example.